



# INSTITUTO POLITÉCNICO NACIONAL

## ESCUELA SUPERIOR DE CÓMPUTO

**ESCOM**

*Trabajo Terminal I*

**“Sistema de comparación de código en C/C++  
mediante similitud semántica para cursos  
introdutorios de programación”**

*2026-B032*

*Presentan:*

**Erick Daniel García Rodríguez  
Claudia Emilia Sánchez García**

*Directores:*

**M. en C. Manuel Portillo Cedillo  
Dra. Maribel Aragón García**

INSTITUTO POLITÉCNICO NACIONAL



*13 de Mayo del 2026*



**INSTITUTO POLITÉCNICO NACIONAL**  
**ESCUELA SUPERIOR DE CÓMPUTO**  
**SUBDIRECCIÓN ACADÉMICA**



No. de TT: 2026-B032

-13 de Mayo del 2026

Documento Técnico

**“Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación”**

*Presentan:*

**Erick Daniel García Rodríguez**  
**Claudia Emilia Sánchez García**

*Directores:*

**M. en C. Manuel Portillo Cedillo**

**Dra. Maribel Aragón García**

**RESUMEN**

Este Trabajo Terminal propone el desarrollo de un sistema de comparación de código en C/C++ diseñado como una herramienta de apoyo a la evaluación del docente en tareas de alumnos para cursos introductorios, como Fundamentos de Programación y Algoritmos. El sistema utiliza técnicas de Procesamiento de Lenguaje Natural (PLN) y representaciones vectoriales para realizar una comparación semántica de las entregas respecto al material de referencia. Además, se integra un análisis estilométrico para identificar patrones de escritura y detectar la similitud con código generado por Inteligencia Artificial (IA), permitiendo una revisión de tareas más objetiva y detallada.

**Palabras clave:** Procesamiento de Lenguaje Natural, Similitud semántica de código, Análisis estilométrico, Aprendizaje profundo, Herramienta de apoyo a la decisión docente.

# Índice general

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                                           | <b>1</b>  |
| 1.1      | Planteamiento del problema . . . . .                                          | 2         |
| 1.2      | Justificación . . . . .                                                       | 4         |
| 1.3      | Motivación . . . . .                                                          | 6         |
| 1.4      | Objetivos . . . . .                                                           | 7         |
| 1.4.1    | Objetivo General . . . . .                                                    | 7         |
| 1.4.2    | Objetivos específicos . . . . .                                               | 7         |
| <b>2</b> | <b>Estado del Arte</b>                                                        | <b>8</b>  |
| 2.1      | Problema de la detección de similitud de código . . . . .                     | 8         |
| 2.2      | Enfoques tradicionales basados en texto . . . . .                             | 8         |
| 2.3      | Herramientas basadas en fingerprinting . . . . .                              | 9         |
| 2.3.1    | MOSS . . . . .                                                                | 9         |
| 2.4      | Herramientas basadas en análisis sintáctico y evaluación automática . . . . . | 10        |
| 2.4.1    | JPlag . . . . .                                                               | 10        |
| 2.4.2    | Gradescope . . . . .                                                          | 10        |
| 2.4.3    | CodeGrade . . . . .                                                           | 10        |
| 2.5      | Representaciones estructurales del código . . . . .                           | 11        |
| 2.5.1    | Árboles de sintaxis abstracta (AST) . . . . .                                 | 11        |
| 2.5.2    | Grafos de flujo de control y datos . . . . .                                  | 12        |
| 2.5.3    | Embeddings de código . . . . .                                                | 12        |
| 2.6      | Modelos basados en aprendizaje profundo . . . . .                             | 12        |
| 2.7      | Detección de código generado por inteligencia artificial . . . . .            | 13        |
| 2.8      | Análisis y brecha de investigación . . . . .                                  | 14        |
| <b>3</b> | <b>Marco Teórico</b>                                                          | <b>17</b> |
| 3.1      | Introducción al análisis automático de código . . . . .                       | 17        |
| 3.2      | Problema de la detección de similitud de código . . . . .                     | 17        |
| 3.3      | Clones de código . . . . .                                                    | 18        |
| 3.4      | Enfoques para la detección de similitud de código . . . . .                   | 18        |
| 3.4.1    | Enfoques basados en análisis textual . . . . .                                | 19        |
| 3.4.2    | Enfoques basados en estructura . . . . .                                      | 19        |
| 3.4.3    | Enfoques basados en semántica . . . . .                                       | 19        |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| 3.4.4    | Enfoques basados en aprendizaje automático . . . . .                        | 20        |
| 3.4.5    | Comparación de enfoques . . . . .                                           | 20        |
| 3.5      | Representaciones del código fuente . . . . .                                | 21        |
| 3.5.1    | Representaciones textuales . . . . .                                        | 22        |
| 3.5.2    | Representaciones sintácticas: Árboles de Sintaxis Abstracta . . . . .       | 22        |
| 3.5.3    | Representaciones basadas en grafos . . . . .                                | 23        |
| 3.5.4    | Comparación de representaciones . . . . .                                   | 23        |
| 3.5.5    | Representaciones distribuidas: embeddings . . . . .                         | 24        |
| 3.6      | Medidas de similitud en código fuente . . . . .                             | 25        |
| 3.7      | Modelos para análisis de código . . . . .                                   | 26        |
| 3.7.1    | Evolución del análisis automático de código . . . . .                       | 26        |
| 3.7.1.1  | Modelos clásicos de aprendizaje automático . . . . .                        | 26        |
| 3.7.1.2  | Redes neuronales (RNN y CNN) . . . . .                                      | 26        |
| 3.7.1.3  | Transformers . . . . .                                                      | 27        |
| 3.7.2    | Modelos basados en texto y tokens . . . . .                                 | 27        |
| 3.7.2.1  | RoBERTa para código . . . . .                                               | 27        |
| 3.7.2.2  | CodeBERT . . . . .                                                          | 28        |
| 3.7.3    | Modelos basados en Árboles de Sintaxis Abstracta . . . . .                  | 28        |
| 3.7.3.1  | Code2Vec . . . . .                                                          | 29        |
| 3.7.3.2  | ASTNN . . . . .                                                             | 29        |
| 3.7.4    | Modelos basados en grafos . . . . .                                         | 30        |
| 3.7.4.1  | GGNN (Gated Graph Neural Networks) . . . . .                                | 30        |
| 3.7.4.2  | GraphCodeBERT . . . . .                                                     | 30        |
| 3.7.5    | Comparación de modelos . . . . .                                            | 31        |
| 3.8      | Análisis estilométrico . . . . .                                            | 32        |
| 3.8.1    | Taxonomía de rasgos estilométricos . . . . .                                | 33        |
| 3.8.2    | Enfoques de estilometría . . . . .                                          | 34        |
| 3.8.2.1  | Métodos estadísticos . . . . .                                              | 34        |
| 3.8.2.2  | Machine Learning clásico . . . . .                                          | 34        |
| 3.8.2.3  | Deep Learning . . . . .                                                     | 34        |
| 3.8.3    | Modelos para el análisis estilométrico . . . . .                            | 35        |
| 3.8.3.1  | Modelos basados en n-gramas . . . . .                                       | 35        |
| 3.8.3.2  | CharCNN . . . . .                                                           | 36        |
| 3.8.3.3  | RoBERTa . . . . .                                                           | 36        |
| 3.8.3.4  | Comparación de modelos . . . . .                                            | 37        |
| 3.8.4    | Métricas de evaluación basadas en la matriz de confusión . . . . .          | 37        |
| 3.9      | Integración y fusión de características ( <i>Feature Fusion</i> ) . . . . . | 39        |
| 3.10     | Conclusión del marco teórico . . . . .                                      | 39        |
| <b>4</b> | <b>Propuesta de solución</b>                                                | <b>41</b> |
| 4.1      | Estructura Funcional del Sistema . . . . .                                  | 42        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 4.1.1    | Entrada de datos y fuentes de referencia . . . . .              | 42        |
| 4.1.2    | Módulo de Preprocesamiento y Normalización . . . . .            | 43        |
| 4.1.3    | Canal A: Comparación Semántica . . . . .                        | 43        |
| 4.1.3.1  | Justificación y Selección del Modelo . . . . .                  | 43        |
| 4.1.3.2  | Extracción de Grafo de Flujo de Datos (DFG) . . . . .           | 44        |
| 4.1.3.3  | Procesamiento en GraphCodeBERT . . . . .                        | 44        |
| 4.1.3.4  | Embedding Lógico . . . . .                                      | 45        |
| 4.1.4    | Canal B: Comparación Estilométrica . . . . .                    | 45        |
| 4.1.4.1  | Justificación y Selección del Modelo . . . . .                  | 45        |
| 4.1.4.2  | Análisis de Autoría mediante CharCNN . . . . .                  | 45        |
| 4.1.4.3  | Cálculo de probabilidad de generación por IA . . . . .          | 46        |
| 4.1.5    | Fusión de Características . . . . .                             | 46        |
| 4.1.6    | Similitud del Coseno . . . . .                                  | 47        |
| 4.1.6.1  | Importancia de la métrica en la fusión de características . . . | 47        |
| 4.2      | Conclusión . . . . .                                            | 48        |
| <b>5</b> | <b>Análisis del Sistema</b>                                     | <b>50</b> |
| 5.1      | Análisis de Requerimientos . . . . .                            | 50        |
| 5.1.1    | Requerimientos Funcionales . . . . .                            | 50        |
| 5.1.2    | Requerimientos No Funcionales . . . . .                         | 51        |
| 5.1.3    | Reglas de Negocio . . . . .                                     | 52        |
| 5.2      | Identificación y descripción de actores . . . . .               | 53        |
| 5.2.1    | Docente / Evaluador . . . . .                                   | 53        |
| 5.2.2    | Motor de Inteligencia Artificial Externo (LLM) . . . . .        | 53        |
| 5.3      | Trayectorias principales y alternativas . . . . .               | 54        |
| 5.3.1    | Trayectoria principal (Flujo de Análisis Estándar) . . . . .    | 54        |
| 5.3.2    | Trayectorias alternativas . . . . .                             | 56        |
| 5.4      | Diagrama de Casos de Uso . . . . .                              | 57        |
| 5.4.1    | Descripción de Actores . . . . .                                | 58        |
| 5.4.2    | Descripción de los Casos de Uso . . . . .                       | 58        |
| 5.4.3    | Especificación de Casos de Uso . . . . .                        | 59        |
| 5.5      | Modelado de Procesos de Negocio (BPMN) . . . . .                | 64        |
| 5.5.1    | Descripción del Flujo de Trabajo . . . . .                      | 65        |
| 5.6      | Características semánticas del código . . . . .                 | 66        |
| 5.7      | Características estilométricas del código . . . . .             | 67        |
| 5.8      | Definición de métricas de desempeño . . . . .                   | 68        |
| 5.8.1    | Métricas para el análisis de similitud semántica . . . . .      | 68        |
| 5.8.2    | Métricas para el análisis estilométrico . . . . .               | 68        |
| 5.8.3    | Resumen de métricas utilizadas . . . . .                        | 69        |
| 5.8.4    | Criterios de interpretación de resultados . . . . .             | 69        |
| 5.8.5    | Consideraciones para la experimentación . . . . .               | 70        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 5.8.6    | Justificación de las métricas seleccionadas . . . . .                  | 70        |
| 5.9      | Análisis de riesgos . . . . .                                          | 71        |
| 5.9.1    | Clasificación de riesgos . . . . .                                     | 71        |
| 5.9.2    | Criterios de evaluación de riesgos . . . . .                           | 71        |
| 5.9.3    | Identificación y evaluación de riesgos . . . . .                       | 72        |
| 5.9.4    | Estrategias de mitigación . . . . .                                    | 72        |
| 5.9.5    | Discusión del impacto de los riesgos . . . . .                         | 73        |
| 5.9.6    | Selección y justificación del conjunto de datos . . . . .              | 73        |
| 5.9.6.1  | Código humano: IEEE Programming Homework Dataset . .                   | 74        |
| 5.9.6.2  | Código generado por IA (Clase Sintética) . . . . .                     | 74        |
| 5.9.6.3  | Consideraciones de balance y partición . . . . .                       | 75        |
| <b>6</b> | <b>Diseño del sistema</b>                                              | <b>77</b> |
| 6.1      | Arquitectura (Modelo C4) . . . . .                                     | 77        |
| 6.1.1    | Nivel 1: Contexto . . . . .                                            | 77        |
| 6.1.2    | Nivel 2: Contenedores . . . . .                                        | 78        |
| 6.1.3    | Nivel 3: Componentes . . . . .                                         | 79        |
| 6.1.4    | Nivel 4: Código (Clases y Secuencia) . . . . .                         | 80        |
| 6.1.4.1  | Diagrama de Clases . . . . .                                           | 80        |
| 6.1.4.2  | Diagrama de Secuencia . . . . .                                        | 81        |
| 6.1.5    | Modelado Dinámico: Diagrama de Estados . . . . .                       | 82        |
| 6.2      | Metodología de Desarrollo: Ciclo de Vida Komorebi . . . . .            | 84        |
| 6.3      | Diseño de Base de Datos . . . . .                                      | 87        |
| 6.3.1    | Modelo Relacional (Diagrama Entidad-Relación) . . . . .                | 87        |
| 6.3.2    | Esquema de Colección Vectorial (ChromaDB) . . . . .                    | 88        |
| 6.4      | Diseño de Interfaz de Usuario (UI/UX) . . . . .                        | 89        |
| 6.4.1    | Módulo de Acceso y Registro . . . . .                                  | 89        |
| 6.4.2    | Gestión de Repositorio y Biblioteca . . . . .                          | 90        |
| 6.4.3    | Módulo de Carga y Configuración de Análisis . . . . .                  | 91        |
| 6.4.4    | Visualización de Reportes de Doble Canal . . . . .                     | 92        |
| <b>7</b> | <b>Ingeniería de Datos</b>                                             | <b>94</b> |
| 7.1      | Adquisición de Datos . . . . .                                         | 94        |
| 7.1.1    | Requerimientos del Conjunto de Datos . . . . .                         | 95        |
| 7.1.2    | Selección del Dataset Base: IEEE Programming Homework Dataset . .      | 96        |
| 7.1.3    | Limitaciones del Dataset: Idioma Serbocroata y Falta de Enunciados . . | 96        |
| 7.1.4    | Ingeniería Inversa y Reconstrucción de Enunciados . . . . .            | 97        |
| 7.1.5    | Generación de Código con IA en Serbocroata y Perfiles de Estudiante .  | 98        |
| 7.1.5.1  | Control del Sesgo de Idioma . . . . .                                  | 98        |
| 7.1.5.2  | Perfiles de Estudiante y Cuota de Comentarios . . . . .                | 99        |
| 7.1.5.3  | Variedad de Modelos de Lenguaje . . . . .                              | 99        |

|          |                                                                                 |            |
|----------|---------------------------------------------------------------------------------|------------|
| 7.1.6    | Ejemplo Comparativo: Código Humano vs. Código de IA . . . . .                   | 99         |
| 7.2      | Preparación y Preprocesamiento de Datos . . . . .                               | 101        |
| 7.2.1    | Bifurcación del Pipeline de Datos . . . . .                                     | 102        |
| 7.2.2    | Normalización Selectiva de Indentación . . . . .                                | 102        |
| 7.2.3    | Preprocesamiento para el Canal Semántico (GraphCodeBERT) . . . . .              | 103        |
| 7.2.4    | Preprocesamiento para el Canal Estilométrico (CharCNN) . . . . .                | 104        |
| 7.2.5    | Partición por Problema para Evitar Fuga de Información . . . . .                | 104        |
| 7.3      | Conclusión del capítulo . . . . .                                               | 105        |
| <b>8</b> | <b>Modelado y Experimentación</b>                                               | <b>106</b> |
| 8.1      | Configuración y Entrenamiento de Modelos . . . . .                              | 106        |
| 8.1.1    | Canal Semántico: GraphCodeBERT y Adaptadores LoRA . . . . .                     | 106        |
| 8.1.1.1  | Modelo Base y Grafo de Flujo de Datos . . . . .                                 | 107        |
| 8.1.1.2  | Adaptación Eficiente mediante LoRA . . . . .                                    | 107        |
| 8.1.2    | Canal Estilométrico: Red Convolutiva Multiescala (MultiScaleChar-CNN) . . . . . | 107        |
| 8.1.2.1  | Arquitectura Neuronal y Fundamentación Estilométrica . . . . .                  | 108        |
| 8.1.2.2  | Hiperparámetros y Entrenamiento . . . . .                                       | 109        |
| 8.1.3    | Fusión de Características (BimodalFusion) . . . . .                             | 110        |
| 8.2      | Evaluación y Validación Experimental . . . . .                                  | 112        |
| 8.2.1    | Evolución Iterativa de los Modelos y Comparativa Experimental . . . . .         | 112        |
| 8.2.2    | Curvas de Aprendizaje y Dinámica de Entrenamiento . . . . .                     | 113        |
| 8.2.3    | Matriz de Confusión sobre Problemas No Vistos . . . . .                         | 114        |
| 8.2.4    | Evaluación del Canal Semántico con Casos de Estudio de ESCOM . . . . .          | 115        |
| 8.2.5    | Evaluación de Invarianza Estilométrica . . . . .                                | 116        |
| 8.2.6    | Rendimiento Global de MultiScaleCharCNN en el Conjunto de Prueba . . . . .      | 116        |
| 8.2.7    | Evaluación del Canal Semántico ante Clones de Bellon . . . . .                  | 117        |
| 8.2.8    | Validación con Arquetipos Estudiantiles (Caso Práctico) . . . . .               | 118        |
| 8.2.8.1  | Análisis de los Resultados . . . . .                                            | 118        |
| 8.3      | Conclusión del capítulo . . . . .                                               | 119        |
| <b>9</b> | <b>Despliegue y Monitorización</b>                                              | <b>120</b> |
| 9.1      | Arquitectura del Backend y Servicios de Inferencia . . . . .                    | 120        |
| 9.1.1    | Organización por Arquitectura Limpia ( <i>Clean Architecture</i> ) . . . . .    | 120        |
| 9.1.2    | Pipeline de Inferencia Asíncrono . . . . .                                      | 121        |
| 9.2      | Capa de Persistencia Híbrida: Relacional y Vectorial . . . . .                  | 121        |
| 9.2.1    | Base de Datos Relacional: PostgreSQL . . . . .                                  | 122        |
| 9.2.2    | Base de Datos Vectorial: ChromaDB . . . . .                                     | 122        |
| 9.3      | Capa de Presentación e Interfaz de Usuario . . . . .                            | 123        |
| 9.3.1    | Componentes Principales de la Plataforma . . . . .                              | 123        |
| 9.3.1.1  | Biblioteca de Referencias y Asignaciones . . . . .                              | 123        |

|           |                                                                                            |            |
|-----------|--------------------------------------------------------------------------------------------|------------|
| 9.3.1.2   | Carga y Parametrización de Soluciones . . . . .                                            | 125        |
| 9.3.1.3   | Panel de Reporte de Similitud e Integridad . . . . .                                       | 125        |
| 9.4       | Infraestructura y Despliegue de la Solución . . . . .                                      | 126        |
| 9.4.1     | Interfaz Web en Vercel y Actualización Automática desde GitHub . . .                       | 127        |
| 9.4.2     | Base de Datos en Supabase (PostgreSQL en la Nube) . . . . .                                | 127        |
| 9.4.3     | Servidor de Inteligencia Artificial en Equipo Local con Tarjeta Gráfica<br>(GPU) . . . . . | 128        |
| 9.4.4     | Empaquetamiento en Contenedores con Docker . . . . .                                       | 128        |
| 9.4.5     | Conexión Segura hacia Internet mediante un Túnel de Cloudflare . . . .                     | 128        |
| 9.5       | Pruebas, Verificación y Monitorización del Sistema . . . . .                               | 129        |
| 9.5.1     | Suite de Pruebas Automatizadas con Pytest . . . . .                                        | 129        |
| 9.5.2     | Estrategia de Monitorización y Mitigación de Falsos Positivos . . . . .                    | 130        |
| 9.6       | Conclusión del capítulo . . . . .                                                          | 130        |
| <b>10</b> | <b>Conclusiones y Trabajo Futuro</b>                                                       | <b>131</b> |
| 10.1      | Cumplimiento de Objetivos y Resultados Técnicos . . . . .                                  | 131        |
| 10.2      | Trabajo Futuro y Líneas de Investigación . . . . .                                         | 132        |
|           | <b>Referencias</b>                                                                         | <b>134</b> |

# Índice de figuras

|      |                                                                                                                                             |     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4.1  | Diagrama de bloques funcional del sistema de comparación Graphito. . . . .                                                                  | 42  |
| 5.1  | Diagrama de trayectoria principal del sistema Graphito correspondiente al flujo de análisis estándar. . . . .                               | 55  |
| 5.2  | Diagrama de casos de uso del sistema Graphito. . . . .                                                                                      | 58  |
| 5.3  | Diagrama BPMN de la comparación de código fuente. . . . .                                                                                   | 66  |
| 6.1  | Diagrama de Contexto (Nivel 1) del sistema Graphito. . . . .                                                                                | 78  |
| 6.2  | Diagrama de Contenedores (Nivel 2) del sistema Graphito. . . . .                                                                            | 79  |
| 6.3  | Diagrama de Componentes (Nivel 3) enfocado en el motor back-end en Python. . . . .                                                          | 80  |
| 6.4  | Diagrama de clases del sistema Graphito, ilustrando la arquitectura orientada a objetos correspondientes al Nivel 4 de abstracción. . . . . | 81  |
| 6.5  | Diagrama de secuencia transaccional de <i>Graphito</i> desde el cliente (Nivel 4). . . . .                                                  | 82  |
| 6.6  | Diagrama de estados del proceso de análisis de código en Graphito. . . . .                                                                  | 83  |
| 6.7  | Flujo secuencial del pipeline de modelos basado en la metodología Komorebi. . . . .                                                         | 85  |
| 6.8  | Modelo Entidad-Relación de la base de datos relacional (PostgreSQL). . . . .                                                                | 88  |
| 6.9  | Esquema conceptual de la colección vectorial en ChromaDB. . . . .                                                                           | 89  |
| 6.10 | Interfaz de inicio de sesión (Cumplimiento de RF-01). . . . .                                                                               | 90  |
| 6.11 | Interfaz de registro de nuevos usuarios. . . . .                                                                                            | 90  |
| 6.12 | Repositorio de problemas y gestión de ejercicios (Cumplimiento de RF-02). . . . .                                                           | 91  |
| 6.13 | Interfaz de carga de código de referencia (Cumplimiento de RF-03). . . . .                                                                  | 92  |
| 6.14 | Configuración de parámetros para el inicio del análisis. . . . .                                                                            | 92  |
| 6.15 | Dashboard de resultados híbridos (Cumplimiento de RF-10 y RF-11). . . . .                                                                   | 93  |
| 7.1  | Flujo general de adquisición e ingeniería inversa de datos. . . . .                                                                         | 95  |
| 8.1  | Flujo de inferencia del pipeline bimodal de Graphito. . . . .                                                                               | 111 |
| 8.2  | Curvas de pérdida y métricas de validación en problemas no vistos para MultiScaleCharCNN. . . . .                                           | 114 |
| 8.3  | Matriz de confusión de MultiScaleCharCNN en el conjunto de prueba ciego (13 problemas no vistos). . . . .                                   | 114 |
| 8.4  | Comparativa de similitud semántica en casos de estudio del temario de ESCOM IPN. . . . .                                                    | 115 |
| 8.5  | Invarianza de la probabilidad sintética ante formateo y comentarios. . . . .                                                                | 116 |

|     |                                                                                    |     |
|-----|------------------------------------------------------------------------------------|-----|
| 9.1 | Módulo de biblioteca de referencias y gestión de asignaciones docentes. . . . .    | 124 |
| 9.2 | Interfaz para la carga y parametrización de nuevas soluciones de referencia. . .   | 125 |
| 9.3 | Panel interactivo del reporte de integridad académica con indicadores bimodales.   | 126 |
| 9.4 | Distribución física de los servicios y flujo de comunicación del sistema Graphito. | 127 |

## Índice de tablas

|      |                                                                                |    |
|------|--------------------------------------------------------------------------------|----|
| 2.1  | Comparación de herramientas de análisis de código . . . . .                    | 11 |
| 2.2  | Comparación de representaciones de código . . . . .                            | 12 |
| 2.3  | Modelos basados en Transformers para análisis de código . . . . .              | 13 |
| 3.1  | Tipos de clones de código según Bellon. . . . .                                | 18 |
| 3.2  | Comparación de enfoques para la detección de similitud de código . . . . .     | 21 |
| 3.3  | Comparación de representaciones del código fuente . . . . .                    | 24 |
| 3.4  | Comparación de métricas de similitud en código fuente . . . . .                | 25 |
| 3.5  | Comparación de modelos preentrenados para análisis de código . . . . .         | 31 |
| 3.6  | Taxonomía de rasgos estilométricos en código fuente . . . . .                  | 33 |
| 3.7  | Comparación de enfoques para el análisis estilométrico . . . . .               | 35 |
| 3.8  | Comparación de modelos para el análisis estilométrico de código fuente . . . . | 37 |
| 3.9  | Componentes fundamentales de la matriz de confusión binaria. . . . .           | 38 |
| 3.10 | Métricas de rendimiento derivadas de la matriz de confusión. . . . .           | 38 |
| 5.1  | Especificación del Caso de Uso 1. . . . .                                      | 59 |
| 5.2  | Especificación del Caso de Uso 2. . . . .                                      | 60 |
| 5.3  | Especificación del Caso de Uso 3. . . . .                                      | 60 |
| 5.4  | Especificación del Caso de Uso 4. . . . .                                      | 61 |
| 5.5  | Especificación del Caso de Uso 5. . . . .                                      | 61 |
| 5.6  | Especificación del Caso de Uso 6. . . . .                                      | 62 |
| 5.7  | Especificación del Caso de Uso 7. . . . .                                      | 63 |
| 5.8  | Especificación del Caso de Uso 8. . . . .                                      | 63 |
| 5.9  | Especificación del Caso de Uso 9. . . . .                                      | 64 |
| 5.10 | Especificación del Caso de Uso 10. . . . .                                     | 64 |
| 5.11 | Clasificación sintetizada de características semánticas del código. . . . .    | 66 |
| 5.12 | Clasificación de características estilométricas del código fuente. . . . .     | 67 |
| 5.13 | Métricas de validación de modelos utilizadas en el sistema. . . . .            | 69 |
| 5.14 | Criterios de interpretación para similitud semántica espacial. . . . .         | 69 |
| 5.15 | Análisis de riesgos del sistema . . . . .                                      | 72 |

|      |                                                                                                               |     |
|------|---------------------------------------------------------------------------------------------------------------|-----|
| 5.16 | Composición y origen del conjunto de datos dual . . . . .                                                     | 75  |
| 5.17 | Distribución cuantitativa y partición del conjunto de datos . . . . .                                         | 76  |
| 7.1  | Estructura del conjunto de datos IEEE Plagiarism. . . . .                                                     | 96  |
| 7.2  | Ejemplos de requerimientos algorítmicos recuperados por ingeniería inversa. . . . .                           | 98  |
| 7.3  | Perfiles de simulación de estudiantes en la generación sintética. . . . .                                     | 99  |
| 7.4  | Contraste de características entre implementación humana y sintética (Caso de Estudio). . . . .               | 101 |
| 7.5  | Tratamiento diferencial en el pipeline dual de preprocesamiento. . . . .                                      | 102 |
| 7.6  | Distribución y partición del conjunto de datos consolidado. . . . .                                           | 105 |
| 8.1  | Arquitectura por capas y flujo tensorial de la red MultiScaleCharCNN. . . . .                                 | 109 |
| 8.2  | Hiperparámetros de entrenamiento de la red MultiScaleCharCNN. . . . .                                         | 110 |
| 8.3  | Matriz de decisión y directrices del dictamen heurístico bimodal. . . . .                                     | 112 |
| 8.4  | Evolución histórica de versiones y métricas de desempeño sobre el conjunto de prueba. . . . .                 | 113 |
| 8.5  | Métricas de desempeño consolidadas de MultiScaleCharCNN sobre el conjunto de prueba (3,000 muestras). . . . . | 117 |
| 8.6  | Resultados del análisis bimodal sobre los arquetipos de entrega estudiantil. . . . .                          | 118 |
| 9.1  | Comparativa técnica de persistencia híbrida: PostgreSQL vs ChromaDB. . . . .                                  | 123 |
| 9.2  | Estructura y cobertura de la suite de pruebas automatizadas en Pytest. . . . .                                | 129 |

# Capítulo 1

## Introducción

La enseñanza de programación actualmente enfrenta un cambio de paradigma derivado del uso de herramientas de Inteligencia Artificial (IA) basadas en Modelos de Lenguaje de Gran Escala (LLMs), como ChatGPT, Claude o Gemini [1]. Estas herramientas han demostrado ser altamente eficientes para resolver tareas lógicas y algorítmicas, logrando un desempeño comparable al de un estudiante de programación destacado [1]. No obstante, el uso de estas tecnologías introduce nuevas dinámicas en los procesos de evaluación [1]. Algunas investigaciones han descrito este fenómeno como la “paradoja cognitiva de la inteligencia artificial”, donde la IA puede actuar tanto como un facilitador del razonamiento como un mecanismo de delegación de procesos cognitivos [1]. En el ámbito educativo, esto se relaciona con el concepto de “descarga cognitiva”, el cual modifica la forma en que se construyen y presentan las soluciones en programación [1].

Estudios recientes han señalado que el uso de asistentes de IA influye en la manera en que se abordan tareas técnicas, incluyendo actividades como la lectura, comprensión y depuración de código [2]. Este escenario no solo impacta la dinámica de aprendizaje, sino que introduce un reto adicional para los docentes, quienes deben interpretar el código como evidencia del proceso lógico seguido para resolver un problema [2]. En este contexto, la evaluación en cursos de programación se vuelve particularmente compleja. La revisión manual del código continúa siendo uno de los métodos más confiables para analizar el razonamiento detrás de una solución; sin embargo, en grupos numerosos o en contextos con alta carga académica, este proceso resulta poco escalable y difícil de sostener de manera sistemática [2]. Esta situación genera una sobrecarga operativa para el docente, limitando la profundidad con la que puede realizarse el análisis de cada programa [2].

Debido a estas limitaciones, el problema técnico puede analizarse desde dos dimensiones complementarias. La primera dimensión se relaciona con la dificultad para evaluar la equivalencia lógica entre programas que implementan una misma solución mediante estructuras sintácticas diferentes. Las herramientas tradicionales de detección de similitud, como aquellas basadas en comparaciones textuales o análisis léxico, presentan limitaciones para identificar relaciones se-

mánticas profundas entre programas, especialmente cuando existen modificaciones superficiales en el código o variaciones en la estructura de implementación [3]. Esta problemática se vuelve aún más relevante en escenarios donde las herramientas de inteligencia artificial generativa son capaces de producir soluciones funcionalmente equivalentes, pero sintácticamente distintas [3].

Para abordar esta problemática, el presente trabajo plantea un enfoque de comparación semántica basado en representaciones vectoriales del código fuente. Mediante el uso de modelos especializados en análisis de código, se busca representar las relaciones internas del programa, incluyendo dependencias y estructuras asociadas a su lógica de funcionamiento, con el objetivo de generar embeddings semánticos que permitan comparar programas desde una perspectiva lógica y no únicamente textual [3]. Bajo este enfoque, dos programas que resuelven un mismo problema mediante diferentes estructuras sintácticas pueden ser identificados como semánticamente similares al conservar una lógica de funcionamiento equivalente [4]. A partir de estas representaciones, es posible utilizar métricas de similitud para identificar programas con comportamiento equivalente aun cuando presenten diferencias sintácticas significativas [4].

La segunda dimensión del problema se relaciona con la identificación de patrones asociados al uso de inteligencia artificial generativa y el reto adicional para los docentes, quienes deben interpretar el código como evidencia del proceso lógico seguido para resolver un problema [2]. En este contexto, la evaluación en cursos de programación se vuelve particularmente compleja. La revisión manual del código continúa siendo uno de los métodos más confiables para analizar el razonamiento detrás de una solución; sin embargo, en grupos numerosos o en contextos con alta carga académica, este proceso resulta poco escalable y difícil de sostener de manera sistemática [2]. Esta situación genera una sobrecarga operativa para el docente, limitando la profundidad con la que puede realizarse el análisis de cada programa [2].

Frente a este escenario, el presente trabajo terminal tiene como objetivo desarrollar Graphito, un sistema de comparación de código en C/C++ mediante similitud semántica y análisis estilométrico para cursos introductorios de programación, concebido como una herramienta de apoyo a la decisión docente que facilite evaluaciones más integrales. Mediante este enfoque de doble canal, Graphito contribuye a la reducción de la carga operativa del docente.

## **1.1 Planteamiento del problema**

La enseñanza de la programación en lenguajes como C/C++ se fundamenta en un ideal pedagógico orientado al desarrollo del pensamiento computacional, entendido como la capacidad de descomponer problemas, identificar patrones, diseñar algoritmos y construir soluciones lógicas de manera estructurada [4]. Este enfoque trasciende la simple escritura de código funcional, ya que busca que el estudiante comprenda, interprete y justifique la lógica de cada solución desarrollada, lo cual constituye una competencia esencial en la formación de ingenieros [4].

Sin embargo, este ideal pedagógico se ha visto históricamente comprometido por la dificultad

inherente al aprendizaje de la programación [5]. Diversos estudios han señalado que los estudiantes enfrentan barreras cognitivas significativas al intentar comprender conceptos abstractos como estructuras de control, manejo de memoria o diseño algorítmico [5]. Como consecuencia, se ha observado una tendencia recurrente a evadir el proceso de razonamiento mediante la reutilización de soluciones externas, ya sea a través de la copia directa de compañeros o mediante el uso de recursos disponibles en línea [5]. Este fenómeno pone en evidencia que el problema de entregar código sin comprenderlo no es reciente, sino una problemática estructural [5].

Para verificar que el estudiante ha comprendido la lógica de un programa, la revisión manual o la defensa oral se han considerado tradicionalmente como el método de referencia, ya que permiten evaluar directamente el razonamiento detrás de la solución [6]. Sin embargo, este tipo de evaluación se vuelve difícil de implementar en contextos con alta matrícula, donde la cantidad de estudiantes y las limitaciones de tiempo del docente generan un cuello de botella que impide su aplicación de forma sistemática [6].

Como respuesta a estas limitaciones, se han desarrollado diversas herramientas automatizadas de evaluación [7]. Entre ellas destacan los jueces en línea, los cuales operan bajo un modelo de evaluación estática centrado en la verificación de la salida del programa frente a un conjunto de casos de prueba [7]. Aunque estos sistemas permiten evaluar rápidamente grandes volúmenes de código, su principal limitación radica en que no consideran la lógica interna utilizada para resolver el problema, lo que puede derivar en la aprobación de soluciones funcionales sin una comprensión real por parte del estudiante [7]. De manera complementaria, las herramientas tradicionales de detección de similitud, como MOSS o JPlag, presentan limitaciones al basarse principalmente en comparaciones textuales y análisis léxico [7, 8]. Estos enfoques presentan vulnerabilidades ante modificaciones superficiales del código, tales como cambios en nombres de variables, reordenamiento de instrucciones o alteraciones en el formato [7]. Al no analizar la equivalencia semántica ni el flujo lógico del programa, se dificulta identificar soluciones que conservan la misma lógica algorítmica, pero presentan diferencias sintácticas [8].

En este contexto, la aparición de herramientas de inteligencia artificial generativa introduce una nueva dimensión al problema [9]. A diferencia de la copia tradicional, donde existía un código fuente original que podía ser rastreado y comparado, los modelos de lenguaje actuales son capaces de generar código de manera probabilística, produciendo soluciones funcionalmente equivalentes pero sintácticamente distintas en cada ejecución [9]. Esta característica difumina la noción de un punto de referencia fijo y reduce significativamente la efectividad de los métodos tradicionales de detección basados en similitud textual, ya que las soluciones generadas no presentan coincidencias directas que puedan ser identificadas convencionalmente [9]. Asimismo, los detectores de contenido generado por IA presentan resultados variables en código, ya que han sido diseñados principalmente para texto natural [8].

Como consecuencia directa de esta nueva dimensión, emerge el fenómeno del *outsourcing* cognitivo, en el cual el estudiante delega el proceso de resolución de problemas a herramientas de in-

teligencia artificial, reduciendo su participación activa en la construcción del conocimiento [10]. Este comportamiento impacta negativamente en el desarrollo de habilidades fundamentales, ya que el estudiante puede obtener resultados funcionales sin haber internalizado los conceptos necesarios para su construcción [10]. Estudios recientes han evidenciado que esta dependencia puede afectar el rendimiento real del estudiante, generando una discrepancia entre la calidad del código entregado y el nivel de comprensión del mismo [10].

Adicionalmente, esta situación genera una pérdida de trazabilidad, en la cual el profesor pierde la capacidad de distinguir de manera confiable entre código desarrollado por el estudiante y código generado automáticamente [11]. La combinación de evaluación estática, herramientas de similitud insuficientes y generación probabilística de código provoca que los mecanismos tradicionales resulten ineficaces para respaldar la integridad académica y la correcta medición del aprendizaje [11].

En este sentido, el problema no radica únicamente en el uso de herramientas de inteligencia artificial, sino en la incapacidad de los sistemas actuales para analizar el código desde una perspectiva integral [11]. Por lo tanto, la ausencia de herramientas capaces de integrar el análisis semántico del código con la identificación de patrones de autoría constituye el problema central de investigación [12]. Ante esta nueva dimensión, resulta necesario transitar hacia modelos de evaluación centrados en competencias y desarrollar herramientas híbridas que utilicen modelos avanzados para analizar la lógica del código a gran escala, permitiendo asistir al docente en la toma de decisiones para mitigar estos nuevos retos tecnológicos [12].

## **1.2 Justificación**

El panorama de la evaluación en la enseñanza de la programación se ha transformado profundamente debido a la adopción de la Inteligencia Artificial Generativa. La creciente integración de estos asistentes por parte del alumnado exige a los profesores enfrentarse al desafío de discernir si el código entregado es producto del razonamiento crítico del estudiante o si proviene de una asistencia automatizada [1, 2, 3, 13]. Frente a este escenario, los mecanismos de revisión convencionales se ven rebasados, lo que restringe la capacidad del educador para validar con certeza tanto la autoría como la estructura lógica de los trabajos presentados.

Ante esta situación, el presente trabajo propone el desarrollo de Graphito como un Sistema de Soporte a la Decisión Educativa, orientado a asistir al docente en el análisis del código fuente. Este sistema se propone como una herramienta para reducir la carga operativa en procesos complejos de toma de decisiones, al permitir procesar grandes volúmenes de información y generar indicadores interpretables que facilitan el análisis. En este sentido, Graphito no se plantea como un sistema de calificación automática, sino como una herramienta de análisis estructural y semántico que proporciona métricas objetivas para interpretar el proceso de razonamiento del estudiante, especialmente en escenarios con alta matrícula donde la revisión manual resulta limitada.

Desde el punto de vista metodológico, la propuesta se sustenta en la integración de técnicas avanzadas de análisis de código que han mostrado resultados prometedores en la literatura reciente [14, 15]. Para evaluar la equivalencia semántica, se plantea el uso de un modelo Transformer preentrenado especializado en código, como GraphCodeBERT [15]. Este ha sido seleccionado por su capacidad de incorporar información estructural mediante grafos de flujo de datos, lo que permite capturar relaciones semánticas profundas entre las variables internas del programa, superando las limitaciones de los enfoques basados únicamente en similitud textual [15]. De manera complementaria, se propone integrar un módulo de análisis estilométrico mediante una red neuronal convolucional a nivel de caracteres (CharCNN), entrenada para identificar patrones asociados a diferentes formas de construcción de código. Este componente permite extraer características estructurales y de formato, generando una representación que puede funcionar como un patrón o métrica del código analizado [16]. La combinación de estos enfoques proporciona una base sólida para desarrollar herramientas capaces de analizar el código desde una perspectiva más integral [16].

En el ámbito teórico, esta propuesta se alinea con las tendencias actuales en analítica del aprendizaje, las cuales buscan ir más allá de evaluaciones binarias centradas únicamente en la ejecución del programa [17]. Estudios recientes han señalado la necesidad de incorporar métricas que permitan analizar la lógica y el proceso de construcción del código, con el objetivo de obtener una evaluación más representativa del aprendizaje del estudiante [17]. El sistema propuesto contribuye a este enfoque al integrar análisis semántico y estilométrico, permitiendo aproximarse a la comprensión del razonamiento detrás del código.

Finalmente, la relevancia de este trabajo se enmarca en el contexto actual de la educación superior en México, donde la inteligencia artificial generativa ha pasado de ser una herramienta emergente a formar parte de las prácticas académicas cotidianas [18]. Estudios recientes evidencian que su uso se ha extendido de manera significativa entre estudiantes y docentes, lo que introduce nuevas dinámicas en el proceso de aprendizaje [19]. Esta adopción generalizada implica que los mecanismos tradicionales de evaluación, centrados principalmente en el resultado final, dejan de ser suficientes para interpretar el nivel real de comprensión del estudiante [20].

En este sentido, el análisis del contexto educativo mexicano señala la necesidad de replantear los modelos de evaluación, orientándolos hacia el reconocimiento de procesos, competencias y formas de construcción del conocimiento, en lugar de limitarse únicamente a la verificación de productos terminados [21]. Este cambio responde a una transformación más amplia en la forma en que se concibe el aprendizaje en entornos mediados por inteligencia artificial, donde el énfasis se desplaza hacia la capacidad del estudiante para comprender, justificar y estructurar soluciones [21]. Bajo esta perspectiva, el desarrollo de herramientas como Graphito adquiere relevancia al alinearse con estas nuevas necesidades del entorno educativo, contribuyendo a optimizar el proceso de evaluación docente en el contexto mexicano y favoreciendo evaluaciones más consistentes y acordes con las dinámicas actuales de la educación superior.

### 1.3 Motivación

La motivación detrás de este trabajo terminal surge del interés por las fronteras actuales de la Inteligencia Artificial (IA), y el profundo interés por transformar y optimizar los procesos de evaluación docente en la educación superior, particularmente en disciplinas afines a la computación.

Desde una perspectiva técnica, el proyecto está impulsado por el desafío de aplicar conceptos avanzados de Procesamiento de Lenguaje Natural (PLN), como *embeddings* y mecanismos de atención en el código fuente. La posibilidad de tratar un programa no solo como texto plano, sino como una estructura con semántica profunda, nos permitió explorar cómo *GraphCodeBERT* puede capturar relaciones lógicas subyacentes mediante grafos de flujo de datos. En este sentido, el trabajo de Martínez Gil [6] ha sido una fuente de inspiración fundamental, demostrando que es posible construir puentes entre la algoritmia y el aprendizaje profundo para resolver problemas complejos de similitud y autoría.

En el ámbito académico, nuestra experiencia en las aulas de la ESCOM nos ha permitido observar una marcada discrepancia en cómo el cuerpo docente enfrenta los nuevos retos que presenta la IA generativa en cursos introductorios de programación. Mientras algunos profesores buscan nuevos recursos metodológicos para asegurar que el estudiante realmente haya asimilado el conocimiento, otros se ven obligados a ignorar esta variable debido a la gran carga de trabajo que implica revisar manualmente múltiples entregas o realizar evaluaciones uno a uno para que el estudiante demuestre su conocimiento del tema. Actualmente, realizar este tipo de evaluaciones consume una cantidad de tiempo insostenible, lo que a menudo reduce la calificación a una simple verificación estática de funcionalidad (entradas y salidas), dejando un punto ciego respecto a cómo se construyó la solución.

Resulta paradójico que, mientras los estudiantes adoptan rápidamente modelos de lenguaje para incrementar su productividad y automatizar la resolución de tareas, las metodologías de revisión docente se mantengan, en gran medida, desprovistas de este mismo avance tecnológico. Nuestra motivación radica en equilibrar esta balanza: implementar el poder de la Inteligencia Artificial no solo como un asistente para quien escribe el código, sino como un respaldo robusto para quien lo evalúa.

Consideramos que el tiempo del docente es el recurso más valioso en el proceso educativo. Por lo tanto, el propósito fundamental de **Graphito** es proveer a los profesores de una herramienta que automatice el desgaste operativo del análisis de código. Al delegar la auditoría semántica y estilométrica a un sistema inteligente, buscamos que el docente optimice sus tiempos de revisión y pueda redirigir ese esfuerzo hacia actividades de mayor impacto pedagógico, tales como asesorías personalizadas, planeación de actividades pedagógicas en grupo y el fomento del pensamiento crítico, áreas donde el criterio humano (*Human-in-the-loop*) es verdaderamente irremplazable.

## 1.4 Objetivos

### 1.4.1 Objetivo General

Implementar un sistema de apoyo a la evaluación docente en cursos introductorios de programación en C/C++ basado en comparación semántica de código, que permita cuantificar la similitud lógica entre programas mediante el cálculo de la similitud del coseno e identificar patrones de generación por inteligencia artificial evaluados a través de f1-score y métricas de clasificación, con el fin de optimizar el proceso de evaluación.

### 1.4.2 Objetivos específicos

- Seleccionar los modelos de lenguaje y arquitecturas de red neuronal adecuados mediante una evaluación comparativa del estado del arte, para fundamentar la viabilidad tecnológica de los módulos semántico y estilométrico.
- Diseñar la arquitectura de software del sistema mediante el modelado de diagramas estándar (UML/BPMN), definiendo la orquestación entre el cliente web, la API de servicios y la persistencia de datos.
- Desarrollar el módulo de análisis semántico integrando el modelo preentrenado GraphCodeBERT, automatizando la extracción de Grafos de Flujo de Datos (DFG) y el cálculo de la similitud del coseno entre representaciones vectoriales.
- Entrenar un modelo de clasificación estilométrica (CharCNN), conformando previamente un conjunto de datos conformado por un repositorio de código humano y código generado automáticamente por IA, evaluando su desempeño en la detección de posibles patrones de inteligencia artificial generativa mediante f1-score.
- Integrar los motores de inteligencia artificial en una plataforma web funcional, permitiendo al docente configurar parámetros de evaluación y visualizar gráficamente los índices de similitud.
- Evaluar el desempeño del sistema en un entorno de pruebas controlado, ejecutando simulaciones con un repositorio estático, para medir sus tiempos de respuesta, precisión y exhaustividad (*recall*) previo a un escenario de producción.

# Capítulo 2

## Estado del Arte

### 2.1 Problema de la detección de similitud de código

La detección de similitud y plagio en código fuente es un problema ampliamente estudiado dentro de la ingeniería de software y la educación en ciencias de la computación. Este problema surge principalmente en cursos de programación donde múltiples estudiantes desarrollan soluciones para un mismo problema, lo que puede facilitar la copia parcial o total de código entre estudiantes. El plagio de código se define como el uso de un programa desarrollado por otra persona sin la debida atribución, lo cual dificulta evaluar si el estudiante realmente comprendió el problema o simplemente reutilizó una solución existente. De acuerdo con diversos estudios sobre análisis de similitud de código, la detección automática de plagio constituye un área fundamental dentro del análisis de software, ya que también se relaciona con tareas como detección de clones de código, reutilización de software y mantenimiento de sistemas [22].

### 2.2 Enfoques tradicionales basados en texto

Los primeros enfoques para detectar similitud entre programas se basaban principalmente en comparaciones textuales del código fuente. Estos métodos analizan el código como una secuencia de caracteres y buscan coincidencias exactas o casi exactas entre fragmentos de programas. Aunque este enfoque permite identificar copias directas, presenta limitaciones cuando los programas han sido modificados mediante cambios superficiales como renombramiento de variables, modificación de comentarios o alteraciones en el formato del código. En consecuencia, los investigadores comenzaron a explorar métodos más robustos que pudieran detectar similitudes incluso cuando el código había sido modificado para ocultar su origen [23].

## 2.3 Herramientas basadas en fingerprinting

### 2.3.1 MOSS

Uno de los sistemas más influyentes en la detección de plagio en código es MOSS (Measure of Software Similarity), desarrollado por Alex Aiken en la Universidad de Stanford. Este sistema utiliza técnicas de fingerprinting, un método que consiste en generar una representación compacta del código mediante huellas digitales o identificadores únicos derivados del contenido del programa. En términos generales, el proceso de fingerprinting consiste en dividir el código en fragmentos y aplicar una función hash a cada uno de ellos. Una función hash es un algoritmo matemático que transforma una secuencia de caracteres en un número de longitud fija. Estas huellas permiten representar el código de forma resumida, de modo que dos programas con fragmentos similares generarán hashes similares o idénticos [24].

#### Funcionamiento de MOSS

- División del código en fragmentos
- Aplicación de funciones hash
- Generación de huellas digitales
- Selección de fingerprints mediante winnowing

Para mejorar la precisión de este proceso, MOSS utiliza el algoritmo de winnowing, un método diseñado para seleccionar un subconjunto representativo de las huellas digitales generadas a partir del código. El algoritmo funciona generando múltiples hashes a partir de fragmentos consecutivos del programa y posteriormente seleccionando aquellos que representan la información más relevante dentro de una ventana de tamaño determinado. Esta técnica permite garantizar que cualquier coincidencia significativa entre dos programas sea detectada incluso cuando se realizan modificaciones menores en el código. Gracias a este procedimiento, el sistema puede identificar fragmentos similares de manera eficiente sin necesidad de comparar el código completo carácter por carácter [24].

#### Limitaciones

- Sensible a cambios estructurales del código
- No detecta similitud semántica profunda

A pesar de su amplia adopción en instituciones educativas, el sistema MOSS presenta ciertas limitaciones relacionadas con el tipo de similitud que es capaz de detectar. Debido a que su

funcionamiento se basa en la comparación de huellas digitales generadas a partir de fragmentos del código, el sistema puede verse afectado cuando los programas presentan modificaciones significativas en su estructura, incluso si implementan la misma lógica algorítmica. Por ejemplo, cambios como la reestructuración de bucles, la división de funciones o la reorganización del flujo del programa pueden alterar las huellas digitales generadas, reduciendo la probabilidad de detectar coincidencias. Estas limitaciones han motivado el desarrollo de enfoques más avanzados capaces de analizar la semántica del código en lugar de basarse únicamente en coincidencias textuales o estructurales [25].

## **2.4 Herramientas basadas en análisis sintáctico y evaluación automática**

### **2.4.1 JPlag**

Otra herramienta ampliamente utilizada en entornos educativos es JPlag, un sistema desarrollado en el Karlsruhe Institute of Technology que permite detectar similitudes entre múltiples programas dentro de un conjunto de entregas. A diferencia de MOSS, que se basa principalmente en fingerprinting, JPlag utiliza un enfoque basado en tokenización del código. La tokenización consiste en convertir el código fuente en una secuencia de unidades léxicas llamadas tokens, que representan elementos fundamentales del lenguaje de programación como palabras clave, operadores, identificadores y símbolos. Una vez que el código se transforma en tokens, el sistema compara las secuencias obtenidas utilizando algoritmos de coincidencia para identificar similitudes estructurales entre programas [26].

### **2.4.2 Gradescope**

Otro sistema ampliamente utilizado en entornos educativos para la evaluación automática de programas es Gradescope, una plataforma que permite automatizar la revisión de tareas de programación mediante la ejecución de pruebas sobre el código entregado por los estudiantes. Este sistema se enfoca principalmente en la validación funcional del programa, evaluando si el código produce los resultados esperados para un conjunto de casos de prueba definidos por el instructor. Aunque este enfoque facilita la calificación automática de ejercicios de programación, su capacidad para detectar similitud entre programas es limitada, ya que se centra principalmente en verificar la funcionalidad del código y no necesariamente en analizar la lógica interna de la solución implementada. En consecuencia, dos programas que resuelven el mismo problema mediante estructuras algorítmicas equivalentes pueden obtener resultados idénticos sin que el sistema detecte posibles coincidencias estructurales entre ellos [27].

### **2.4.3 CodeGrade**

De manera similar, herramientas como CodeGrade han sido desarrolladas para apoyar la evaluación automática en cursos de programación mediante la integración de sistemas de ejecución de

pruebas y detección de similitud entre programas. CodeGrade permite automatizar la revisión de tareas y proporcionar retroalimentación inmediata a los estudiantes, lo cual resulta particularmente útil en cursos con un gran número de participantes. Sin embargo, al igual que otras herramientas basadas en análisis sintáctico o textual, su capacidad para identificar similitud profunda entre programas puede verse limitada cuando los estudiantes aplican transformaciones estructurales al código, como el uso de diferentes estructuras de control o la reorganización de funciones dentro del programa [28].

| Herramienta | Enfoque               | Ventaja                       | Limitación           |
|-------------|-----------------------|-------------------------------|----------------------|
| MOSS        | Fingerprinting        | Eficiente y escalable         | No detecta semántica |
| JPlag       | Tokenización          | Detecta similitud estructural | Evadible             |
| Gradescope  | Pruebas               | Automatización                | No analiza lógica    |
| CodeGrade   | Evaluación automática | Retroalimentación             | Limitado             |

Tabla 2.1: Comparación de herramientas de análisis de código

## 2.5 Representaciones estructurales del código

### 2.5.1 Árboles de sintaxis abstracta (AST)

Con el objetivo de mejorar la detección de similitud entre programas, los investigadores han explorado representaciones estructurales del código basadas en árboles de sintaxis abstracta (AST). Un árbol de sintaxis abstracta es una representación jerárquica del programa que describe su estructura gramatical de acuerdo con las reglas del lenguaje de programación. En esta representación, los nodos del árbol corresponden a construcciones del lenguaje como operaciones aritméticas, expresiones lógicas o estructuras de control. El uso de AST permite comparar la estructura del programa en lugar de su representación textual, lo que facilita detectar similitudes entre programas que presentan diferencias sintácticas superficiales [29].

El análisis basado en árboles de sintaxis abstracta ha sido ampliamente utilizado en sistemas de detección de clones de código debido a su capacidad para capturar la estructura gramatical de los programas. A diferencia de los métodos basados en texto, el análisis de AST permite ignorar detalles superficiales como espacios, comentarios o nombres de variables, enfocándose en la estructura lógica del programa. No obstante, este enfoque también presenta ciertas limitaciones. Dos programas que implementan el mismo algoritmo utilizando diferentes construcciones sintácticas pueden producir árboles sintácticos significativamente distintos, lo que dificulta la detección de similitudes semánticas profundas. Por esta razón, muchos trabajos recientes han propuesto combinar el análisis estructural con técnicas de aprendizaje automático que permitan capturar patrones más complejos en el código [29].

### 2.5.2 Grafos de flujo de control y datos

Sin embargo, el análisis estructural mediante árboles sintácticos puede resultar insuficiente para detectar similitudes más profundas relacionadas con la lógica del programa. Por esta razón, algunos enfoques utilizan representaciones basadas en grafos, como los grafos de flujo de control (CFG) y los grafos de dependencia de datos (DFG). Los grafos de flujo de control representan las posibles rutas de ejecución dentro de un programa, mientras que los grafos de dependencia de datos modelan las relaciones entre variables y operaciones. Estas representaciones permiten capturar información semántica más rica del programa, ya que describen cómo fluye la información dentro del código y cómo interactúan sus diferentes componentes [30].

### 2.5.3 Embeddings de código

En los últimos años, el avance del aprendizaje automático ha permitido desarrollar nuevas técnicas para analizar código fuente mediante representaciones vectoriales conocidas como embeddings. Un embedding es una representación numérica de alta dimensión que captura características semánticas de un objeto. En el contexto del código fuente, los embeddings permiten transformar fragmentos de código en vectores dentro de un espacio matemático donde programas similares se ubican cerca entre sí. Para medir la similitud entre estos vectores se utilizan métricas como la similitud del coseno [31].

| Representación | Nivel              | Ventaja            | Limitación       |
|----------------|--------------------|--------------------|------------------|
| Texto          | Superficial        | Simple             | Fácil de evadir  |
| AST            | Sintáctico         | Analiza estructura | No semántico     |
| Grafos         | Semántico          | Captura flujo      | Complejo         |
| Embeddings     | Semántico profundo | Detecta lógica     | Requiere modelos |

Tabla 2.2: Comparación de representaciones de código

## 2.6 Modelos basados en aprendizaje profundo

Una de las principales innovaciones en este campo ha sido la aplicación de modelos basados en la arquitectura Transformer, originalmente desarrollada para tareas de procesamiento de lenguaje natural. Estos modelos utilizan mecanismos de atención que permiten analizar relaciones entre diferentes partes de una secuencia de texto o código. Entre los modelos más relevantes se encuentra CodeBERT, un modelo preentrenado desarrollado por Microsoft que fue entrenado utilizando grandes repositorios de código fuente y documentación que permite generar representaciones semánticas del código que pueden utilizarse para tareas como búsqueda de código, generación de comentarios y detección de similitud entre programas [32].

Una evolución de este enfoque es GraphCodeBERT, que incorpora información estructural adicional mediante grafos de flujo de datos durante el proceso de entrenamiento. A diferencia de

modelos que analizan únicamente secuencias de tokens, este modelo integra relaciones entre variables y dependencias de datos dentro del programa, lo que permite capturar información semántica más profunda sobre el comportamiento del código. Investigaciones recientes han demostrado que este modelo puede mejorar significativamente la detección de similitud semántica entre programas que implementan la misma lógica mediante estructuras sintácticas diferentes [5].

Además de CodeBERT y GraphCodeBERT, investigaciones recientes han propuesto otros modelos basados en arquitecturas Transformer diseñados específicamente para el análisis de código fuente. Uno de estos modelos es UniXcoder, desarrollado por investigadores de Microsoft, el cual busca aprender representaciones unificadas del código mediante diferentes tareas de pre-entrenamiento. Este modelo combina información sintáctica y semántica del programa para generar representaciones que pueden utilizarse en tareas como búsqueda de código, generación automática de documentación y análisis de similitud entre programas. Estudios recientes han demostrado que este tipo de modelos puede capturar patrones complejos en el código y mejorar la capacidad de los sistemas para identificar relaciones semánticas entre programas escritos de manera diferente [33].

### Características de los modelos basados en Transformers

- Capturan relaciones semánticas dentro del código mediante mecanismos de atención
- Permiten representar programas en espacios vectoriales de alta dimensión
- Integran información sintáctica y semántica en una sola representación

| Modelo        | Tipo                 | Característica           | Aplicación          |
|---------------|----------------------|--------------------------|---------------------|
| CodeBERT      | Transformer          | Representación semántica | Búsqueda de código  |
| GraphCodeBERT | Transformer + grafos | Usa flujo de datos       | Similitud semántica |
| UniXcoder     | Multimodal           | Representación unificada | Análisis avanzado   |

Tabla 2.3: Modelos basados en Transformers para análisis de código

### 2.7 Detección de código generado por inteligencia artificial

De manera complementaria, algunos trabajos recientes han propuesto el uso de técnicas de aprendizaje profundo para identificar patrones semánticos dentro del código utilizando redes neuronales y representaciones basadas en grafos. Por ejemplo, investigaciones sobre aprendizaje semántico de código mediante grafos han demostrado que los modelos que analizan relaciones

estructurales entre instrucciones pueden mejorar la detección de clones semánticos, es decir, programas que realizan la misma funcionalidad, aunque presenten diferencias en su estructura textual [30]. Estos enfoques permiten capturar información a múltiples niveles del programa, incluyendo tokens, sentencias y relaciones entre estructuras de control.

En los últimos años también se han realizado diversos estudios que exploran el uso de representaciones semánticas del código para detectar plagio o similitud entre programas. Por ejemplo, investigaciones recientes han propuesto sistemas de búsqueda semántica de código que utilizan modelos preentrenados para generar embeddings de programas y posteriormente comparar estos vectores utilizando métricas de similitud. Estos sistemas han demostrado que las representaciones semánticas pueden capturar relaciones entre programas incluso cuando presentan diferencias significativas en su estructura sintáctica. En particular, algunos trabajos han mostrado que los modelos basados en Transformers pueden correlacionar de manera significativa con evaluaciones humanas al medir la similitud entre programas, lo que sugiere que estas técnicas pueden convertirse en herramientas valiosas para apoyar la evaluación automática en cursos de programación [34].

Finalmente, estudios recientes han destacado que el crecimiento de herramientas de generación automática de código basadas en modelos de lenguaje introduce nuevos desafíos para la detección de plagio en entornos educativos. Los modelos generativos actuales son capaces de producir soluciones funcionalmente correctas para problemas de programación, lo que dificulta determinar si el código fue desarrollado por un estudiante o generado mediante inteligencia artificial. Ante este escenario, algunos trabajos han comenzado a explorar el uso de técnicas de análisis estilométrico y aprendizaje automático para identificar patrones característicos asociados a código generado automáticamente [35].

### **Retos en la detección de código generado por IA**

- Los modelos generativos producen código funcionalmente correcto
- Difícil distinguir entre código humano y generado automáticamente
- Los patrones de generación pueden ser consistentes pero no evidentes

## **2.8 Análisis y brecha de investigación**

En general, la literatura muestra que los sistemas existentes para la detección de similitud de código suelen enfocarse en uno de dos enfoques principales: el análisis estructural del programa o el análisis semántico mediante representaciones aprendidas por modelos de aprendizaje profundo. Sin embargo, cada uno de estos enfoques presenta limitaciones cuando se utiliza de manera aislada. Mientras que los métodos estructurales pueden verse afectados por modificaciones sintácticas significativas, los modelos basados en aprendizaje profundo pueden requerir grandes

cantidades de datos para su entrenamiento y presentar dificultades para explicar sus decisiones. Por esta razón, varios trabajos recientes sugieren que la combinación de diferentes enfoques de análisis puede mejorar la precisión y robustez de los sistemas de detección de similitud de código [22].

### **Limitaciones identificadas en la literatura**

- Los enfoques estructurales no capturan completamente la semántica del código
- Los modelos de aprendizaje profundo requieren grandes volúmenes de datos
- Existe dificultad para interpretar los resultados de modelos complejos

En síntesis, la investigación en detección de similitud de código ha evolucionado desde enfoques basados en comparación textual hacia métodos más avanzados que incorporan representaciones estructurales y semánticas del programa. Mientras que herramientas tradicionales como MOSS y JPlag se enfocan principalmente en coincidencias sintácticas, los modelos modernos basados en aprendizaje profundo y arquitecturas Transformer permiten capturar relaciones semánticas más complejas dentro del código. No obstante, el desarrollo reciente de herramientas de generación automática de código plantea nuevos retos para la evaluación académica, lo que sugiere la necesidad de desarrollar sistemas híbridos que integren múltiples técnicas de análisis para mejorar la detección de similitud y la identificación de código generado mediante inteligencia artificial.

A pesar de los avances en la detección de similitud de código, las herramientas tradicionales como MOSS o JPlag se enfocan principalmente en coincidencias textuales o estructurales, lo que limita su capacidad para detectar equivalencias semánticas profundas entre programas que implementan la misma lógica mediante estructuras diferentes. Por otra parte, los modelos recientes basados en aprendizaje profundo y arquitecturas Transformer han demostrado ser capaces de capturar relaciones semánticas más complejas dentro del código, aunque su aplicación en entornos educativos aún presenta desafíos relacionados con la interpretación de resultados y la integración con herramientas de evaluación académica.

Asimismo, el crecimiento de herramientas de generación automática de código basadas en modelos de lenguaje ha introducido nuevos retos para la detección de plagio en programación. Los sistemas actuales no siempre permiten distinguir de manera confiable entre código escrito por estudiantes y código generado mediante inteligencia artificial, lo que dificulta evaluar adecuadamente el aprendizaje en cursos introductorios de programación.

### **Necesidad identificada**

- Integrar análisis semántico y estilométrico en un mismo sistema

- Mejorar la interpretación de resultados para apoyo docente
- Desarrollar herramientas aplicables en contextos educativos reales

En este contexto, se identifica una brecha en la literatura relacionada con el desarrollo de sistemas que integren simultáneamente análisis semántico del código y técnicas adicionales que permitan identificar patrones asociados al uso de herramientas de inteligencia artificial. Por lo tanto, resulta pertinente explorar enfoques híbridos que combinen representaciones semánticas del código con técnicas complementarias de análisis, con el objetivo de mejorar la evaluación automática en entornos educativos.

# Capítulo 3

## Marco Teórico

### 3.1 Introducción al análisis automático de código

El análisis automático de código fuente es un área interdisciplinaria que integra conceptos de ingeniería de software, procesamiento de lenguaje natural y aprendizaje automático. En el ámbito educativo, particularmente en cursos introductorios de programación, se emplea para evaluar soluciones desarrolladas por estudiantes y detectar similitudes entre programas.

Sin embargo, debido a la complejidad estructural y semántica de los lenguajes de programación, identificar estas similitudes representa un desafío técnico. A lo largo del tiempo, se han propuesto diversas metodologías que abarcan desde comparaciones textuales hasta enfoques basados en grafos y modelos de aprendizaje profundo.

En este contexto, resulta necesario analizar los principales enfoques y técnicas utilizadas en el análisis de código, con el fin de comprender sus características, alcances y limitaciones.

### 3.2 Problema de la detección de similitud de código

Roy y Cordy describen los clones de código como fragmentos de programas que presentan similitudes en su estructura o funcionalidad, y destacan que su detección es una tarea fundamental para el mantenimiento y análisis de software [23]. No obstante, este proceso enfrenta múltiples desafíos técnicos debido a la ambigüedad estructural del código y a la existencia de múltiples implementaciones para un mismo algoritmo. Esto hace más complejo distinguir entre una similitud semántica real y coincidencias sintácticas por el uso de convenciones de programación estándar y buenas prácticas.

Para representar estos obstáculos, la literatura identifica los siguientes retos principales en la detección automática [23, 36]:

- **Variaciones sintácticas:** Cambios en el estilo de programación, nombres de variables o

estructuras de control que no alteran el resultado.

- **Ofuscación de código:** Transformaciones deliberadas para hacer el código ilegible manteniendo su funcionalidad original.
- **Diferentes implementaciones:** El uso de distintos algoritmos o estructuras de datos para resolver el mismo problema lógico.
- **Código generado por IA:** El auge de modelos de lenguaje (LLMs) permite generar múltiples versiones de un mismo programa, introduciendo patrones de similitud difíciles de rastrear mediante métodos tradicionales.

Estos desafíos evidencian la necesidad de una taxonomía que permita categorizar el grado de semejanza entre programas, lo cual se aborda a través de la clasificación técnica de clones.

### 3.3 Clones de código

La similitud entre programas no es un concepto binario, sino que puede presentarse en distintos niveles dependiendo del grado en que comparten estructura o funcionalidad. Para abordar esta variabilidad, la literatura propone clasificaciones que permiten categorizar los diferentes tipos de similitud. Una de las más utilizadas es la taxonomía de Bellon [37], la cual organiza los clones de código en cuatro niveles principales.

Tabla 3.1: Tipos de clones de código según Bellon.

| Tipo     | Nivel de Similitud      | Descripción                                                 |
|----------|-------------------------|-------------------------------------------------------------|
| Tipo I   | Textual / Exacto        | Copias idénticas, excepto por espacios y comentarios.       |
| Tipo II  | Sintáctico / Renombrado | Estructura idéntica con cambios en identificadores o tipos. |
| Tipo III | Sintáctico / Modificado | Código con sentencias añadidas, eliminadas o modificadas.   |
| Tipo IV  | Semántico / Funcional   | Misma lógica implementada con distinta sintaxis.            |

Esta clasificación refleja un incremento progresivo en la complejidad de detección: mientras que los clones de Tipo I y II pueden identificarse mediante análisis superficiales, los clones de Tipo III requieren tolerancia a modificaciones estructurales, y los clones de Tipo IV implican comprender la lógica del programa. Por esta razón, los clones semánticos representan el principal desafío en la detección de similitud de código y motivan el uso de enfoques más avanzados.

### 3.4 Enfoques para la detección de similitud de código

Existen distintos enfoques para la detección de similitud de código, los cuales se diferencian en el nivel de abstracción con el que analizan los programas. Estos van desde comparaciones

textuales superficiales hasta métodos que capturan la estructura, la semántica o representaciones vectoriales mediante aprendizaje automático [23].

Cada enfoque presenta un compromiso entre precisión, capacidad para capturar similitud semántica y costo computacional, lo que determina su aplicabilidad en distintos contextos.

### **3.4.1 Enfoques basados en análisis textual**

Los enfoques textuales tratan el programa como una secuencia de caracteres o cadenas, comparando fragmentos directamente tras aplicar filtros básicos como la eliminación de espacios y comentarios [23]. Su fortaleza radica en la eficiencia computacional y en la alta precisión para detectar copias exactas (clones Tipo I), sin requerir conocimiento gramatical del lenguaje [23, 38]. Sin embargo, al operar sobre la superficie del texto, son vulnerables a cambios menores como el renombrado de variables, la reubicación de instrucciones o la modificación del formato, los cuales alteran la representación textual sin afectar la lógica del programa [23].

### **3.4.2 Enfoques basados en estructura**

Los enfoques estructurales analizan la organización interna del programa en lugar de su forma textual, modelando unidades como funciones, bloques de control y relaciones entre componentes. A partir de estas unidades se derivan métricas como la profundidad de anidamiento, la complejidad ciclomática o el número de llamadas a funciones, lo que permite identificar similitudes dentro de límites sintácticos específicos y a diferentes niveles de granularidad [23, 36].

A diferencia de los métodos textuales, estos enfoques capturan la organización lógica del código y las preferencias de diseño del programador como la utilización de funciones, modularidad, patrones recurrentes con menor sensibilidad a cambios de formato o nombres de variables [36]. No obstante, estas representaciones siguen siendo estáticas, puesto que no modelan el comportamiento dinámico del programa, por lo que cambios equivalentes como sustituir un bucle `for` por un `while` pueden producir estructuras distintas sin alterar la funcionalidad, limitando la detección de clones semánticos [23, 38].

### **3.4.3 Enfoques basados en semántica**

Los enfoques semánticos analizan el código desde la perspectiva de cómo funcionan, ya que en lugar de examinar cómo está escrito, buscan comprender lo que hace, modelando su lógica de ejecución y las relaciones entre datos [23, 30]. Para ello, en lugar de comparar texto o estructura gramatical, abstraen el programa en representaciones que capturan las dependencias entre instrucciones y el flujo de información interno, independientemente de cómo esté implementado sintácticamente.

Esta capacidad de abstracción los convierte en los únicos enfoques capaces de detectar clones de

Tipo IV: programas funcionalmente equivalentes con una estructura sintáctica diferente [23, 30]. Por ejemplo, un bucle `for` y una recursión que cumplen con la misma tarea serían completamente diferentes para métodos textuales o estructurales, pero equivalentes bajo un análisis semántico.

Sin embargo, esta profundidad analítica tiene un costo, puesto que comparar programas a nivel semántico implica resolver problemas de alta complejidad computacional, lo que limita la escalabilidad de los métodos clásicos [30]. Investigaciones recientes han respondido a esta limitación con modelos de aprendizaje profundo que aproximan la semántica del código de manera más eficiente [30, 39]. Las representaciones concretas sobre las que operan estos enfoques como grafos de flujo de control, flujo de datos y dependencia de programas se describen en detalle más adelante.

#### 3.4.4 Enfoques basados en aprendizaje automático

El análisis de código fuente ha evolucionado con la consolidación del área conocida como *Big Code*, que parte de la premisa de que el código puede interpretarse como un lenguaje altamente estructurado [40]. Bajo este enfoque, los modelos de aprendizaje profundo son capaces de analizar la sintaxis y la semántica de los programas sin necesidad de ejecutarlos, transformando el código en representaciones numéricas continuas denominadas *embeddings*, donde fragmentos con comportamientos similares se ubican en regiones cercanas del espacio vectorial [40, 41].

El paradigma dominante es la arquitectura codificador-decodificador (*encoder-decoder*) donde el codificador transforma representaciones de código como secuencia de tokens, árbol sintáctico o grafo en un vector de contexto, y el decodificador utiliza dicha representación para tareas como clasificación, búsqueda o detección de clones [40]. Los mecanismos de atención permiten al modelo ponderar las partes más relevantes del código durante la inferencia, facilitando la captura de relaciones entre elementos distantes dentro del programa [41]. Su principal limitación es estructural, ya que requieren grandes volúmenes de datos de entrenamiento, tienen alto costo computacional y operan como cajas negras, dificultando la interpretación de sus decisiones [40, 41].

#### 3.4.5 Comparación de enfoques

Los enfoques descritos durante esta sección no son excluyentes entre sí, sino que representan niveles progresivos de abstracción sobre el mismo problema. Para facilitar su comparación, en la Tabla 3.2 se sintetizan las dimensiones más relevantes de cada enfoque.

Tabla 3.2: Comparación de enfoques para la detección de similitud de código

| Enfoque     | Abstracción         | Clones | Costo | Ventaja principal                                   | Limitación principal                             |
|-------------|---------------------|--------|-------|-----------------------------------------------------|--------------------------------------------------|
| Textual     | Bajo (Texto)        | I–II   | Bajo  | Alta eficiencia; independiente del lenguaje.        | Vulnerable a cambios de nombres y formato.       |
| Estructural | Medio (AST)         | II–III | Medio | Captura lógica y diseño del programa.               | Sensible a cambios en estructuras equivalentes.  |
| Semántico   | Alto (DF-G/CFG)     | IV     | Alto  | Detecta equivalencias funcionales reales.           | Complejidad algorítmica y baja escalabilidad.    |
| IA (ML/DL)  | Variable (Vectores) | I–IV   | Alto  | Extracción automática de características complejas. | Requiere grandes datasets; modelo de caja negra. |

*Elaboración propia con base en [23, 38, 36, 30, 40, 41].*

Esta limitación demuestra que, si se planea hacer una comparación compleja entre programas, no basta con utilizar un sólo enfoque para el análisis. Antes de determinar cómo afrontar el problema de realizar la comparación semántica mientras se tiene en cuenta los patrones de autoría, resulta necesario analizar las representaciones internas del código sobre las que operan estos enfoques, tema que se aborda en la siguiente sección.

### 3.5 Representaciones del código fuente

El análisis automático de código fuente requiere transformar los programas en estructuras que permitan su procesamiento y comparación. Estas transformaciones determinan el tipo de información que puede ser capturada y, por tanto, el nivel de similitud que el sistema es capaz de identificar. Las representaciones van desde la forma textual del programa hasta estructuras que modelan su comportamiento semántico, cada una con un equilibrio distinto entre expresividad y costo computacional.

### 3.5.1 Representaciones textuales

El nivel más básico consiste en tratar el código como una secuencia lineal de elementos. Existen tres variantes principales según el grado de abstracción aplicado [23]:

- **Nivel de caracteres:** el código se modela como una secuencia continua de caracteres y se compara mediante técnicas como *fingerprinting*. Es simple y eficiente, pero altamente sensible a cambios mínimos de formato como espacios o saltos de línea [23].
- **Nivel de tokens:** un analizador léxico (*lexer*) transforma el código en una secuencia de unidades como palabras clave, operadores e identificadores. Es común aplicar normalización sustituyendo nombres de variables por símbolos genéricos para reducir la variabilidad [23].
- **N-gramas:** se construyen subsecuencias contiguas de longitud  $n$  sobre caracteres o tokens. Valores entre 6 y 8 tokens ofrecen un buen balance entre precisión y generalización [42].

En todos los casos, el preprocesamiento incluye la eliminación de comentarios y espacios en blanco, y la comparación se realiza mediante estructuras eficientes como árboles de sufijos (*suffix trees*) [23]. Estas representaciones son portables entre lenguajes y computacionalmente económicas, pero su naturaleza lineal las hace incapaces de capturar la estructura interna o el comportamiento del programa, lo que motiva el uso de representaciones sintácticas más ricas, descritas en la siguiente sección.

### 3.5.2 Representaciones sintácticas: Árboles de Sintaxis Abstracta

El Árbol de Sintaxis Abstracta (AST) representa el programa como una estructura jerárquica en la que se eliminan elementos superficiales como espacios, comentarios, o formato; conservando únicamente la información gramaticalmente relevante. Los nodos internos corresponden a constructos del lenguaje como condicionales, bucles o declaraciones de funciones, mientras que los nodos hoja representan operandos como identificadores o literales [23, 43].

Su construcción se realiza mediante un analizador sintáctico (*parser*) que organiza la secuencia de tokens conforme a las reglas de una gramática libre de contexto [29]. Herramientas como *Tree-sitter* o *ANTLR* automatizan este proceso generando estructuras navegables que representan fielmente la sintaxis del programa. A diferencia de las representaciones textuales, los AST abstraen detalles como nombres de variables o formato y agrupan instrucciones en subárboles que corresponden a unidades sintácticas bien definidas, lo que los hace inmunes a modificaciones superficiales y adecuados para detectar clones de Tipo II y Tipo III [23, 44].

No obstante, presentan dos limitaciones importantes: requieren analizadores específicos para cada lenguaje, lo que reduce su portabilidad, y la comparación entre árboles mediante distancia de

edición o isomorfismo implica algoritmos de alto costo computacional [23, 44]. Más fundamentalmente, el AST captura la organización gramatical del programa pero no modela cómo fluye la información entre sus instrucciones, por lo que transformaciones equivalentes como sustituir un bucle `for` por un `while` pueden producir árboles distintos sin alterar la funcionalidad [23]. Esta limitación motiva el uso de representaciones basadas en grafos, descritas en la siguiente sección.

### 3.5.3 Representaciones basadas en grafos

Las representaciones basadas en grafos modelan el programa como una red de relaciones donde los nodos representan entidades del código como sentencias o variables y las aristas representan dependencias o relaciones de ejecución [30]. Existen tres variantes principales:

- **Grafo de Flujo de Control (CFG):** representa el orden de ejecución mediante aristas dirigidas que modelan los posibles caminos del programa, incluyendo condicionales, ciclos y bifurcaciones [30].
- **Grafo de Flujo de Datos (DFG):** modela cómo los valores se producen, transforman y consumen a lo largo del programa, representando dependencias entre variables [5].
- **Grafo de Dependencia de Programas (PDG):** integra el flujo de control y de datos en una única representación, ofreciendo una visión semántica completa del comportamiento del programa [30].

A diferencia del AST, que captura únicamente la jerarquía gramatical, estas representaciones modelan explícitamente las dependencias que determinan el comportamiento en tiempo de ejecución, incluyendo relaciones causales entre instrucciones y ciclos como bucles o recursividad [45]. Esto las convierte en las representaciones más adecuadas para detectar clones de Tipo IV, al abstraer completamente las diferencias superficiales de sintaxis o estilo [30, 5].

Su principal desventaja es el costo: la construcción y comparación de grafos implica algoritmos costosos en tiempo y memoria, y requieren herramientas avanzadas de análisis estático adaptadas a cada lenguaje [30]. Esta limitación ha impulsado el desarrollo de representaciones más compactas que proyectan estas estructuras complejas en espacios vectoriales continuos, descritas en la siguiente sección.

### 3.5.4 Comparación de representaciones

Las representaciones descritas en las secciones anteriores no son equivalentes entre sí ya que cada una captura un nivel distinto de información del programa. La Tabla 3.3 sintetiza estas diferencias considerando las dimensiones más relevantes para un sistema de comparación de código.

Tabla 3.3: Comparación de representaciones del código fuente

| Representación   | Abstracción                  | Costo | Ventaja principal                                           | Limitación principal                                               |
|------------------|------------------------------|-------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Textual          | Bajo (cadenas o tokens)      | Bajo  | Alta portabilidad; no requiere conocimiento gramatical [23] | Sensible a cambios de formato; no captura semántica [23]           |
| AST              | Medio (jerarquía sintáctica) | Medio | Abstrae formato; captura organización lógica [23, 44]       | No modela flujo de datos; requiere <i>parser</i> por lenguaje [23] |
| Grafos (CFG/DFG) | Alto (flujo y datos)         | Alto  | Inmune a variaciones sintácticas; captura intención [30, 5] | Alta complejidad de extracción y comparación [30]                  |

Elaboración propia con base en [23, 44, 30, 5].

Como se observa, las representaciones más simples priorizan la eficiencia a costa de profundidad semántica, mientras que las basadas en grafos logran capturar equivalencias funcionales complejas con un costo computacional mayor [23, 30]. Esta falta de equivalencia entre las representaciones ha motivado el desarrollo de un mecanismo que permite proyectar cualquiera de estas representaciones (textual, sintáctica o basada en grafos) en un espacio vectorial continuo donde la similitud puede cuantificarse mediante operaciones geométricas simples. Dicho mecanismo, conocido como *embedding*, no constituye una representación alternativa del código, sino una capa de procesamiento que puede aplicarse sobre las anteriores y que actúa como puente hacia los modelos de aprendizaje profundo descritos en la siguiente sección [40, 41].

### 3.5.5 Representaciones distribuidas: embeddings

Un elemento transversal a todas las representaciones descritas es la posibilidad de proyectarlas en un espacio vectorial continuo mediante modelos de aprendizaje profundo, generando lo que se conoce como *embeddings*. Un embedding no es una representación alternativa del código, sino una transformación numérica que puede aplicarse sobre cualquiera de las anteriores: una secuencia de tokens, un AST o un grafo de dependencias pueden convertirse en vectores densos de dimensión fija [40, 41].

La ventaja de esta proyección es que fragmentos de código con comportamientos similares tienden a ubicarse en regiones cercanas del espacio vectorial, lo que permite cuantificar la similitud mediante operaciones geométricas simples [41]. La siguiente sección se enfoca en presentar distintas medidas de similitud en código fuente, destacando sus usos, fortalezas y limitaciones.

### 3.6 Medidas de similitud en código fuente

Una vez definidas las representaciones del código, es necesario establecer cómo se cuantifica la semejanza entre dos fragmentos. La elección de la métrica está directamente condicionada por la representación utilizada: no es equivalente comparar cadenas de texto que árboles sintácticos, grafos o vectores continuos [23, 45]. La Tabla 3.4 resume las principales métricas empleadas en la literatura según su nivel de abstracción.

Tabla 3.4: Comparación de métricas de similitud en código fuente

| Métrica                  | Nivel      | Clones      | Ventaja                                                  | Limitación                                   |
|--------------------------|------------|-------------|----------------------------------------------------------|----------------------------------------------|
| Similitud de Jaccard     | Léxico     | Tipo I–II   | Simple y eficiente; mide solapamiento de tokens [23]     | No captura orden ni estructura.              |
| Distancia de Levenshtein | Léxico     | Tipo I–II   | Detecta copias cercanas con cambios mínimos [23]         | Sensible a renombrado y reordenamiento.      |
| Tree Edit Distance       | Sintáctico | Tipo II–III | Captura similitud estructural sobre el AST [29]          | Costo crece con la profundidad del árbol.    |
| Graph Edit Distance      | Semántico  | Tipo III–IV | Captura diferencias en flujo de datos [30]               | Complejidad NP-completo; baja escalabilidad. |
| Similitud del coseno     | Vectorial  | Tipo I–IV   | Alta robustez semántica sobre <i>embeddings</i> [41, 45] | Depende de la calidad del modelo de IA.      |
| Distancia euclidiana     | Vectorial  | Tipo I–IV   | Intuitiva en espacios de baja dimensión [45]             | Sensible a la magnitud de los vectores.      |

Como se observa, existe una relación directa entre la profundidad semántica de una métrica y su costo computacional [23]. Las métricas léxicas son eficientes pero frágiles ante modificaciones intencionales del código, mientras que las basadas en grafos ofrecen mayor capacidad de abstracción a costa de una complejidad computacional que las hace inviables a gran escala [30]. La similitud del coseno sobre embeddings representa el punto de equilibrio más adecuado para el contexto de Graphito: combinada con un modelo que incorpore información estructural del código, permite comparar programas con alta robustez semántica y bajo costo de inferencia [41, 45].

En la siguiente sección se exponen distintos modelos que siguen los enfoques y representaciones previamente planteados para evaluar sus fortalezas y limitaciones.

## **3.7 Modelos para análisis de código**

### **3.7.1 Evolución del análisis automático de código**

El análisis automatizado de código fuente ha evolucionado desde enfoques basados en características diseñadas manualmente hacia modelos capaces de aprender representaciones de manera automática a partir de los datos [41]. Esta transición responde a las limitaciones de los métodos tradicionales para capturar la estructura y semántica del código.

En general, los sistemas de análisis de código siguen un pipeline común: el código fuente se transforma en una representación procesable, un modelo analiza dicha representación y, finalmente, se emplea una métrica para cuantificar la similitud entre programas [41].

#### **3.7.1.1 Modelos clásicos de aprendizaje automático**

Los primeros enfoques utilizaron algoritmos como Máquinas de Vectores de Soporte (SVM), Bosques Aleatorios (Random Forest) y k-Vecinos Más Cercanos (kNN), operando sobre vectores de características extraídas manualmente, como frecuencias de tokens o métricas de software [46].

Estos modelos resultan eficientes y relativamente simples de implementar; sin embargo, su desempeño depende directamente de la calidad de las características diseñadas. Al centrarse en propiedades superficiales, presentan dificultades para capturar relaciones estructurales o semánticas del código, lo que los hace sensibles a transformaciones como renombrado de variables o cambios de estilo [47].

#### **3.7.1.2 Redes neuronales (RNN y CNN)**

Las redes neuronales introducen la capacidad de aprender representaciones automáticamente a partir del código, reduciendo la dependencia de la ingeniería manual de características [41].

Las Redes Neuronales Recurrentes (RNN) y sus variantes como LSTM modelan el código como una secuencia de tokens, procesando cada elemento en función del contexto previo mediante un estado interno [48]. Este enfoque permite capturar dependencias locales dentro del programa, aunque presenta dificultades para modelar relaciones a largo alcance debido a su naturaleza secuencial [40].

Por su parte, las Redes Neuronales Convolucionales (CNN), especialmente a nivel de caracteres (CharCNN), aplican filtros sobre fragmentos locales del código para identificar patrones recurrentes [7]. Esta aproximación resulta útil para capturar características estilísticas, pero su

capacidad para representar la estructura global del programa es limitada [40].

En conjunto, estos modelos mejoran la representación del código respecto a los métodos clásicos, aunque aún presentan limitaciones para capturar completamente su semántica.

### **3.7.1.3 Transformers**

Los modelos basados en Transformers representan el estado del arte en el análisis de código, al utilizar mecanismos de atención que permiten relacionar directamente todos los elementos de una secuencia [49]. A través de este mecanismo, el modelo puede identificar qué partes del código son relevantes entre sí, capturando dependencias globales de manera más efectiva que los enfoques secuenciales.

Esta capacidad resulta especialmente relevante en código fuente, donde variables, funciones o estructuras pueden interactuar a lo largo de diferentes partes del programa. Además, la atención multibezel permite modelar distintos tipos de relaciones en paralelo, facilitando la captura tanto de patrones sintácticos como semánticos.

Como resultado, los Transformers han demostrado una alta capacidad para aprender representaciones profundas del código, siendo la base de los modelos modernos en tareas como detección de similitud, búsqueda de código y análisis semántico [49]. No obstante, su uso implica un mayor costo computacional y una menor interpretabilidad en comparación con enfoques más simples.

## **3.7.2 Modelos basados en texto y tokens**

Los modelos basados en texto y tokens constituyen una de las primeras aproximaciones modernas al análisis de código fuente mediante aprendizaje profundo. Su principio fundamental consiste en tratar el código como lenguaje natural, representándolo como una secuencia lineal de tokens sin considerar explícitamente su estructura sintáctica o semántica.

A pesar de esta simplificación, el uso de arquitecturas basadas en Transformers permite capturar dependencias contextuales entre los elementos del código, generando representaciones útiles para diversas tareas. No obstante, su capacidad está limitada por la ausencia de información estructural del programa.

### **3.7.2.1 RoBERTa para código**

RoBERTa es una variante optimizada de BERT (Transformer) que mejora el proceso de preentrenamiento mediante el uso de mayores volúmenes de datos y la eliminación de ciertos objetivos auxiliares [50]. En el análisis de código, se emplea como modelo base para procesar el código fuente como texto plano.

El modelo utiliza tokenización basada en Codificación de Par de Bytes (BPE), lo que le per-

mite manejar eficientemente el vocabulario abierto característico del código fuente, donde los identificadores pueden variar ampliamente.

Su entrenamiento se basa en el objetivo de Modelado de Lenguaje Enmascarado, aprendiendo a predecir tokens a partir de su contexto. Esto le permite capturar patrones léxicos y ciertas dependencias contextuales dentro del código.

No obstante, al igual que otros modelos basados únicamente en texto, su principal limitación radica en la falta de representación explícita de la estructura del programa, lo que reduce su capacidad para distinguir entre similitudes superficiales y equivalencias funcionales.

### **3.7.2.2 CodeBERT**

CodeBERT es un modelo bimodal basado en Transformers, diseñado para aprender representaciones conjuntas de código fuente y lenguaje natural [41]. Su arquitectura se basa en RoBERTa, lo que le permite procesar secuencias de tokens de manera bidireccional, capturando el contexto completo de cada elemento.

El modelo representa el código como una secuencia lineal de tokens, combinándolo opcionalmente con texto natural mediante tokens especiales como *[CLS]* y *[SEP]*. El token *[CLS]* se utiliza como un marcador inicial que permite al modelo generar una representación global de toda la secuencia, mientras que *[SEP]* funciona como un separador entre diferentes segmentos, como una descripción en lenguaje natural y el código correspondiente. Esta organización permite al modelo relacionar ambos tipos de información dentro de una misma entrada, facilitando tareas como la búsqueda de código o la generación de documentación.

En cuanto a su entrenamiento, CodeBERT utiliza una función híbrida que combina el Modelado de Lenguaje Enmascarado (MLM) y la Detección de Tokens Reemplazados (RTD), aprovechando grandes volúmenes de datos provenientes de repositorios públicos. Este enfoque le permite aprender patrones tanto léxicos como contextuales.

En conjunto, estos modelos demuestran que es posible obtener representaciones útiles del código a partir de su forma textual. Sin embargo, sus limitaciones evidencian la necesidad de incorporar información estructural más rica, lo que ha motivado el desarrollo de modelos basados en representaciones sintácticas y semánticas, como se aborda en la siguiente sección.

### **3.7.3 Modelos basados en Árboles de Sintaxis Abstracta**

Los modelos basados en Árboles de Sintaxis Abstracta (AST) representan una evolución respecto a los enfoques puramente textuales, al incorporar explícitamente la estructura gramatical del programa. En lugar de tratar el código como una secuencia lineal de tokens, estos modelos lo organizan en una representación jerárquica que refleja las relaciones sintácticas entre sus componentes.

Esta representación permite capturar patrones estructurales como la anidación de bloques, la organización de expresiones y la descomposición funcional del código, proporcionando una visión más rica que los modelos basados únicamente en texto.

### **3.7.3.1 Code2Vec**

Code2Vec es un modelo basado en una red neuronal diseñado para aprender representaciones vectoriales de fragmentos de código a partir de su estructura sintáctica [41]. Su arquitectura utiliza un mecanismo de atención suave (soft-attention) para combinar múltiples contextos estructurales en un único vector representativo.

El modelo representa el código como un conjunto de rutas extraídas del AST, donde cada ruta conecta pares de tokens a través de nodos intermedios. Esta estrategia permite capturar relaciones estructurales entre elementos del programa, como la interacción entre parámetros, variables y valores de retorno.

Code2Vec fue entrenado sobre grandes colecciones de código en Java, utilizando como objetivo la predicción del nombre de un método a partir de su implementación. Esto le permite aprender patrones semánticos asociados a la funcionalidad del código.

Sin embargo, su desempeño depende en gran medida de los identificadores presentes en el código, lo que lo hace sensible a la ofuscación o al uso de nombres no vistos durante el entrenamiento.

### **3.7.3.2 ASTNN**

ASTNN es un modelo que combina redes neuronales recursivas para capturar tanto la información estructural como la secuencial del código, abordando las limitaciones de procesar árboles completos de gran profundidad [51].

Para ello, el modelo descompone el AST en una secuencia de subárboles correspondientes a sentencias individuales, codificando cada uno mediante redes neuronales y posteriormente modelando sus relaciones mediante una red recurrente bidireccional. Esta estrategia permite manejar programas de mayor tamaño sin perder información estructural relevante.

ASTNN ha demostrado buenos resultados en tareas como clasificación de código y detección de clones, al combinar información local (estructura de cada subárbol) con el contexto global del programa.

No obstante, su evaluación se ha centrado principalmente en conjuntos de datos académicos, lo que puede limitar su generalización a código real más diverso.

En conjunto, estos modelos evidencian que incorporar la estructura sintáctica del código permite mejorar significativamente la representación respecto a los enfoques basados únicamente en texto. Sin embargo, el AST describe la organización gramatical del programa, pero no modela

explícitamente el flujo de datos o el comportamiento en ejecución. Esta limitación ha motivado el desarrollo de modelos basados en grafos de dependencia, los cuales se abordan en la siguiente sección.

### **3.7.4 Modelos basados en grafos**

Los modelos basados en grafos representan un nivel más avanzado en el análisis de código fuente, al enfocarse en capturar el comportamiento del programa en lugar de solo su forma textual o estructura sintáctica. En este enfoque, el código se modela como un grafo donde los nodos representan elementos del programa (como variables o instrucciones) y las aristas representan relaciones entre ellos, como el flujo de datos o de control.

Esta representación permite identificar similitudes funcionales entre programas, incluso cuando su implementación es completamente diferente, lo que resulta fundamental para la detección de clones semánticos.

#### **3.7.4.1 GGNN (Gated Graph Neural Networks)**

GGNN es un modelo basado en Redes Neuronales de Grafos (GNN) que utiliza unidades recurrentes tipo GRU para propagar información entre los nodos del grafo a través de múltiples iteraciones [52]. Esta arquitectura permite que cada nodo actualice su representación considerando el contexto de sus vecinos, capturando dependencias complejas dentro del programa.

Para representar el código, el modelo construye un grafo enriquecido a partir del AST, incorporando tanto relaciones sintácticas como semánticas, incluyendo flujo de datos y control. De este modo, no solo se modela la estructura del programa, sino también cómo se transmiten los valores entre sus componentes.

El modelo fue entrenado de manera supervisada en tareas específicas como predicción de nombres de variables y detección de errores. Su principal fortaleza es la capacidad de capturar relaciones semánticas explícitas; sin embargo, presenta limitaciones en su generalización a nuevos dominios y requiere que el código sea completamente válido para poder construir el grafo.

#### **3.7.4.2 GraphCodeBERT**

GraphCodeBERT es un modelo basado en la arquitectura Transformer que integra información estructural del código mediante el uso de grafos de flujo de datos [5]. A diferencia de los modelos puramente secuenciales, introduce restricciones en el mecanismo de atención para reflejar relaciones entre variables.

El modelo representa el código como una combinación de tokens y nodos de un grafo de flujo de datos, permitiendo que el mecanismo de atención considere únicamente aquellas conexiones que tienen sentido semántico. Esto le permite capturar dependencias relevantes sin necesidad de

procesar estructuras completas como el AST.

Su entrenamiento combina el Modelado de Lenguaje Enmascarado (MLM) con tareas adicionales como la predicción de relaciones entre nodos, lo que favorece el aprendizaje de representaciones más estructuradas.

GraphCodeBERT ha demostrado un alto desempeño en tareas como detección de clones, búsqueda de código y análisis semántico. No obstante, su complejidad computacional es elevada y su funcionamiento interno resulta menos interpretable.

En conjunto, los modelos basados en grafos permiten capturar el comportamiento funcional del código, siendo especialmente efectivos para detectar similitudes semánticas profundas. Sin embargo, al centrarse en la lógica del programa, tienden a ignorar características superficiales del código, como el estilo o la forma en que está escrito, lo que limita su utilidad en tareas donde estos aspectos son relevantes.

### 3.7.5 Comparación de modelos

La evolución de las arquitecturas para el análisis de código refleja un incremento progresivo en la capacidad de abstracción semántica. A continuación, se presenta una síntesis comparativa de los modelos analizados, categorizados por su tipo de representación, destacando sus fortalezas y las limitaciones técnicas que definen su alcance.

Tabla 3.5: Comparación de modelos preentrenados para análisis de código

| Modelo   | Categoría      | Representación   | Ventaja principal                                                  | Limitación principal                                           |
|----------|----------------|------------------|--------------------------------------------------------------------|----------------------------------------------------------------|
| CodeBERT | Texto / tokens | Bimodal (Tokens) | Vincula código y lenguaje natural; eficaz en búsqueda [32].        | Sin estructura jerárquica; requiere mucho <i>fine-tuning</i> . |
| RoBERTa  | Texto / tokens | Tokens BPE       | Robusto ante vocabulario abierto; distingue patrones léxicos [50]. | Puramente textual; impreciso en lógica funcional equivalente.  |
| Code2Vec | Estructural    | Rutas del AST    | Captura relaciones estructurales entre identificadores [41].       | Vocabulario cerrado; sensible a ofuscación de nombres.         |

*Continúa en la siguiente página...*

**Tabla 3.5 – Continuación**

| <b>Modelo</b> | <b>Categoría</b> | <b>Representación</b>    | <b>Ventaja principal</b>                                   | <b>Limitación principal</b>                         |
|---------------|------------------|--------------------------|------------------------------------------------------------|-----------------------------------------------------|
| ASTNN         | Estructural      | Subárboles               | Maneja programas largos sin perder información local [51]. | Validado principalmente en entornos académicos.     |
| GGNN          | Grafos           | AST + Aristas semánticas | Modela dependencias de datos mediante grafos [52].         | Requiere código compilable; difícil de generalizar. |
| GraphCodeBERT | Grafos           | Tokens + DFG             | Integra flujo de datos en el mecanismo de atención [5].    | Opacidad interpretativa; alta demanda de cómputo.   |

*Elaboración propia con base en [32, 50, 41, 51, 52, 5].*

Una vez agotado el análisis de los modelos orientados a la captura de la semántica algorítmica, es imperativo reconocer que el código fuente no es solo una entidad funcional, sino también un producto de autoría humana. Mientras que el flujo de datos y control define la ejecución del programa, existen rasgos intrínsecos en la escritura que no alteran la lógica, pero que actúan como una huella digital del programador. En consecuencia, el análisis automático de código requiere de una perspectiva complementaria que no se centre en el 'qué' hace el algoritmo, sino en el 'cómo' se manifiesta el estilo individual del autor. Esta disciplina, conocida como estilometría de código, proporciona las herramientas necesarias para abordar la dimensión de identidad, la cual se explora en detalle en la siguiente sección.

### 3.8 Análisis estilométrico

La estilometría se define como el estudio cuantitativo del estilo de escritura individual [36]. En el contexto del código fuente, analiza cómo los hábitos de un desarrollador como la nomenclatura, el formato o la organización del código se manifiestan como patrones medibles.

Su objetivo principal es la atribución de autoría, es decir, determinar quién escribió un fragmento de código a partir de características estilísticas. Este problema es relevante en escenarios como detección de plagio, análisis forense de software y seguridad informática. A pesar de las restricciones de los lenguajes de programación, los desarrolladores conservan suficiente libertad

para dejar huellas distintivas en su código [36].

### 3.8.1 Taxonomía de rasgos estilométricos

Para caracterizar estos patrones, Horvath [36] clasifica los rasgos estilométricos en diferentes niveles de abstracción, desde aspectos superficiales hasta decisiones de diseño y comportamiento.

Tabla 3.6: Taxonomía de rasgos estilométricos en código fuente

| Tipo de rasgo      | Qué analiza                                        | Ejemplos                                                                                                           |
|--------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Léxico</b>      | Elementos superficiales del código (tokens).       | Convenciones de nombres ( <i>camelCase</i> vs <i>snake_case</i> ), uso de operadores, longitud de identificadores. |
| <b>Sintáctico</b>  | Organización gramatical y reglas del lenguaje.     | Uso de estructuras de control ( <i>for</i> vs <i>while</i> ), estilo de condiciones booleanas, posición de llaves. |
| <b>Estructural</b> | Decisiones de diseño y organización de alto nivel. | Funciones largas vs. modulares, profundidad de anidamiento, uso de recursividad o iteración.                       |
| <b>Semántico</b>   | Intención y lógica detrás del programa.            | Elección de estructuras de datos específicas, coherencia en nombres de dominio, algoritmos implementados.          |
| <b>Conductual</b>  | Dinámica de desarrollo y comportamiento del autor. | Frecuencia de <i>commits</i> , patrones de edición en el tiempo, dinámica de tecleo ( <i>keystroke dynamics</i> ). |

Elaboración propia con base en [36].

En conjunto, estos rasgos permiten modelar el estilo de un programador desde múltiples niveles. Mientras que los rasgos superficiales son más fáciles de extraer, los niveles estructurales y semánticos ofrecen mayor capacidad discriminativa, aunque requieren representaciones más complejas. Por otro lado, los rasgos conductuales aportan una dimensión temporal que complementa el análisis estático del código.

### 3.8.2 Enfoques de estilometría

El análisis estilométrico puede abordarse mediante distintos enfoques metodológicos que varían en su nivel de complejidad, capacidad de generalización y profundidad semántica. De forma general, estos enfoques han evolucionado desde técnicas estadísticas simples hacia modelos avanzados basados en aprendizaje automático y profundo. Cada uno de estos paradigmas ofrece ventajas y limitaciones específicas, por lo que su elección depende del tipo de rasgos estilísticos que se deseen capturar y del nivel de abstracción requerido.

#### 3.8.2.1 Métodos estadísticos

Los métodos estadísticos analizan el estilo mediante el cálculo de frecuencias sobre elementos superficiales del código: conteo de tokens, métricas de software simples y análisis de n-gramas [36]. Su principal ventaja es la interpretabilidad, ya que las decisiones de similitud se basan en recuentos directamente comprensibles para el analista. Sin embargo, al tratar el programa como una secuencia básica de símbolos, ignoran la intención funcional o lógica del mismo, lo que los hace útiles para capturar el vocabulario superficial de un programador pero insuficientes ante modificaciones estructurales.

#### 3.8.2.2 Machine Learning clásico

El Machine Learning clásico emplea algoritmos como las Máquinas de Vectores de Soporte (SVM), los k-vecinos más cercanos (kNN) y árboles de decisión sobre vectores de características extraídas manualmente del código [36]. Representa una mejora respecto a los métodos estadísticos en capacidad de discriminación y generalización, pero mantiene su limitación fundamental: depende completamente de que expertos definan los atributos relevantes previamente, lo que restringe su alcance a similitudes léxicas o estructurales superficiales.

#### 3.8.2.3 Deep Learning

Las arquitecturas de Deep Learning aprenden representaciones directamente desde los datos crudos, sin depender de características diseñadas manualmente [36]. Al mapear el código fuente en embeddings de alta dimensión, codifican conjuntamente propiedades sintácticas y semánticas, permitiendo que fragmentos con patrones de autoría similares converjan en el espacio vectorial. Esta capacidad de generalización los hace especialmente adecuados para escenarios complejos como la identificación de autoría en código ofuscado o la distinción entre código humano y generado por IA.

La Tabla 3.7 sintetiza las diferencias entre los tres paradigmas en las dimensiones más relevantes para el análisis estilométrico de código fuente.

Una vez conocidos los enfoques para abordar el problema, a continuación se expondrán distin-

Tabla 3.7: Comparación de enfoques para el análisis estilométrico

| Enfoque                  | Capacidad semántica | Costo computacional | Ventaja principal                                                                                | Limitación principal                                                       |
|--------------------------|---------------------|---------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Estadístico              | Baja                | Bajo                | Alta interpretabilidad; no requiere entrenamiento [36]                                           | Incapaz de capturar intención funcional o patrones estructurales complejos |
| Machine Learning clásico | Media               | Medio               | Mejor discriminación que métodos estadísticos; generalizable a múltiples autores [36]            | Dependencia absoluta de ingeniería manual de características               |
| Deep Learning            | Alta                | Alto                | Aprende representaciones automáticamente; robusto ante variaciones imprevistas en el estilo [36] | Alto costo computacional de entrenamiento; comportamiento de caja negra    |

*Elaboración propia con base en [36].*

tos modelos de Inteligencia Artificial aplicados al análisis estilométrico siguiendo los enfoques previamente planteados, analizando sus fortalezas y debilidades.

### 3.8.3 Modelos para el análisis estilométrico

El análisis estilométrico de código fuente requiere modelos capaces de capturar las huellas personales que un programador deja en sus programas, más allá de la lógica del algoritmo. A diferencia de los modelos orientados a similitud semántica, estos enfoques se centran en patrones como convenciones de nomenclatura, formato e incluso hábitos de escritura.

En este contexto, los modelos estilométricos pueden entenderse como una evolución desde métodos estadísticos simples hasta arquitecturas profundas capaces de aprender representaciones complejas del estilo. A continuación, se presentan tres enfoques representativos que ilustran esta progresión.

#### 3.8.3.1 Modelos basados en n-gramas

Los modelos de n-gramas constituyen un enfoque estadístico basado en frecuencia, donde el código se representa como secuencias contiguas de tokens o caracteres [42]. Su “arquitectura” no es neuronal, sino un modelo de perfilado: construyen vectores de frecuencia que describen el

estilo de un autor.

Esta representación es efectiva para estilometría porque captura patrones repetitivos de bajo nivel, como el uso de operadores, palabras clave o convenciones de escritura. Al comparar estos perfiles mediante métricas como similitud del coseno, es posible identificar similitudes entre autores.

Su principal fortaleza es la simplicidad e interpretabilidad. Sin embargo, al depender únicamente de frecuencias locales, no capturan relaciones de largo alcance ni contexto, lo que limita su capacidad cuando el estilo no es superficial o cuando hay pocos datos disponibles.

### **3.8.3.2 CharCNN**

CharCNN es un modelo basado en Redes Neuronales Convolucionales (CNN) que opera directamente sobre secuencias de caracteres [7]. Su arquitectura utiliza convoluciones unidimensionales para detectar patrones locales en el texto, seguidas de operaciones de pooling que resumen la información relevante.

Esta arquitectura resulta adecuada para estilometría porque permite aprender automáticamente patrones de bajo nivel, como indentación, uso de caracteres especiales, convenciones de nombres o formato, sin necesidad de tokenizar el código ni conocer su sintaxis. En otras palabras, captura el estilo como una señal visual.

No obstante, su enfoque local limita la captura de dependencias globales dentro del código, y su desempeño depende fuertemente de grandes volúmenes de datos para aprender representaciones robustas.

### **3.8.3.3 RoBERTa**

RoBERTa es un modelo basado en la arquitectura Transformer bidireccional, que utiliza mecanismos de autoatención para modelar relaciones entre todos los tokens de una secuencia [50]. Esta arquitectura permite capturar dependencias contextuales globales dentro del código.

En estilometría, esta capacidad resulta especialmente valiosa, ya que permite modelar no solo patrones léxicos, sino también decisiones de codificación que dependen del contexto completo del programa. Su tokenización basada en subpalabras (BPE) permite además manejar identificadores arbitrarios sin perder información relevante.

Gracias a su arquitectura, RoBERTa puede aprender representaciones profundas del estilo de programación, capturando patrones complejos que van más allá de lo superficial. Sin embargo, este nivel de abstracción implica un alto costo computacional y menor interpretabilidad, además de no incorporar explícitamente la estructura del código.

### 3.8.3.4 Comparación de modelos

Los modelos presentados reflejan distintos niveles de complejidad y capacidad para capturar el estilo de programación. Mientras que los enfoques estadísticos destacan por su simplicidad, los modelos neuronales permiten capturar patrones más complejos a costa de mayor costo computacional. La Tabla 3.8 sintetiza las diferencias entre los tres modelos en las dimensiones más relevantes para el análisis estilométrico.

Tabla 3.8: Comparación de modelos para el análisis estilométrico de código fuente

| Modelo   | Arquitectura                 | Rasgos                   | Ventaja principal                                                        | Limitación principal                                     |
|----------|------------------------------|--------------------------|--------------------------------------------------------------------------|----------------------------------------------------------|
| N-gramas | Estadística<br>(Frecuencias) | Léxicos<br>superficiales | Alta interpretabilidad; independiente del lenguaje [42].                 | Degrada con múltiples autores; requiere muestras largas. |
| CharCNN  | ConvNet 1D<br>(Caracteres)   | Léxicos y de<br>formato  | Detecta “biomarcadores” de estilo invisibles para modelos de tokens [7]. | Requiere grandes volúmenes de datos para entrenamiento.  |
| RoBERTa  | Transformer<br>(BPE)         | Contexto<br>semántico    | Maneja identificadores arbitrarios sin perder contexto [50].             | Sin comprensión estructural; alto costo computacional.   |

*Elaboración propia con base en [42, 7, 50].*

En conjunto, estos modelos ofrecen distintas formas de capturar el estilo de programación, desde patrones superficiales hasta representaciones profundas basadas en contexto. En la siguiente sección se presenta una síntesis que muestra que mediante la integración de distintos enfoques, se puede conseguir una arquitectura robusta capaz de superar las limitaciones de los modelos previamente presentados.

### 3.8.4 Métricas de evaluación basadas en la matriz de confusión

De acuerdo con [53], la matriz de confusión es una herramienta de evaluación detallada en la ciencia de datos que permite analizar el rendimiento de un clasificador mediante el desglose de predicciones acertadas y erróneas para cada clase. Esta estructura matricial de tamaño  $N \times N$  ofrece una visión clara de cómo el modelo confunde o acierta las etiquetas reales frente a las predichas.

Para el caso de una clasificación binaria, los componentes fundamentales que integran esta ma-

triz se describen en la Tabla 3.9.

Tabla 3.9: Componentes fundamentales de la matriz de confusión binaria.

| Componente                                | Definición técnica                                                                       |
|-------------------------------------------|------------------------------------------------------------------------------------------|
| Verdadero Positivo ( $TP$ )               | Instancias donde tanto el valor real como la predicción del modelo son positivos.        |
| Verdadero Negativo ( $TN$ )               | Instancias donde el valor real es negativo y la predicción es igualmente negativa.       |
| Falso Positivo ( $FP$ )                   | Casos donde la predicción es positiva pero el valor real es negativo (Error de Tipo I).  |
| Falso Negativo ( $FN$ )                   | Casos donde la predicción es negativa pero el valor real es positivo (Error de Tipo II). |
| <i>Elaboración propia basada en [53].</i> |                                                                                          |

A partir de estos cuatro estados, se calculan indicadores de desempeño que permiten cuantificar la eficacia del modelo bajo distintas perspectivas (ver Tabla 3.10).

Tabla 3.10: Métricas de rendimiento derivadas de la matriz de confusión.

| Métrica                                   | Fórmula                         | Propósito analítico                                                                      |
|-------------------------------------------|---------------------------------|------------------------------------------------------------------------------------------|
| Exactitud                                 | $\frac{TP+TN}{Total}$           | Mide la fracción total de predicciones correctas. Es útil en clases balanceadas.         |
| Precisión                                 | $\frac{TP}{TP+FP}$              | Evalúa la exactitud de las predicciones positivas; busca minimizar los falsos positivos. |
| Recall                                    | $\frac{TP}{TP+FN}$              | Mide la capacidad del modelo para capturar la totalidad de los casos positivos reales.   |
| Puntuación $F_1$                          | $\frac{2 \cdot P \cdot R}{P+R}$ | Media armónica que equilibra precisión y exhaustividad, ideal para datos desbalanceados. |
| Especificidad                             | $\frac{TN}{TN+FP}$              | Cuantifica qué tan bien el modelo identifica los resultados negativos reales.            |
| <i>Elaboración propia basada en [53].</i> |                                 |                                                                                          |

Finalmente, el análisis se complementa con la curva ROC y el valor AUC (*Area Under the Curve*). La curva ROC permite visualizar el equilibrio entre la tasa de verdaderos positivos y falsos positivos a través de distintos umbrales, mientras que el AUC proporciona una medida agregada de la capacidad del modelo para distinguir entre clases, donde un valor de 1.0 representa un clasificador perfecto.

### 3.9 Integración y fusión de características (*Feature Fusion*)

La fusión de características es una técnica avanzada que permite mejorar el desempeño de modelos de análisis de código mediante la combinación de múltiples fuentes de información en una representación unificada [6]. A diferencia de los enfoques tradicionales, que dependen de una sola representación del código, este paradigma integra características aprendidas automáticamente por modelos profundos con atributos externos diseñados manualmente o derivados de otras fuentes.

Desde el punto de vista arquitectónico, este proceso se implementa mediante la concatenación del vector de salida global del modelo base (*pooled output*) con vectores adicionales de características, seguido de una capa de proyección que unifica ambas representaciones en un espacio latente común [6]. Esto permite que el modelo combine información heterogénea —como patrones sintácticos, métricas estructurales o datos de ejecución— en una única representación enriquecida.

Esta estrategia ofrece tres ventajas principales:

- **Compensación de limitaciones:** Permite complementar los modelos preentrenados con información específica del dominio que no es capturada durante el aprendizaje automático.
- **Complementariedad analítica:** Facilita la integración de distintos tipos de análisis, como el análisis estático del código y características externas que reflejan su comportamiento o contexto.
- **Representaciones más ricas:** Genera embeddings más completos al combinar múltiples niveles de información, mejorando la capacidad del modelo para capturar tanto la forma como la funcionalidad del código.

Diversos estudios han demostrado la efectividad de este enfoque híbrido, reportando mejoras significativas en tareas de análisis de código, con métricas que alcanzan valores cercanos a 0.98 en precisión y 0.99 en medida F [6].

En conjunto, la fusión de características se posiciona como una estrategia clave para superar las limitaciones individuales de los modelos existentes, al permitir la integración de múltiples perspectivas del código en un único sistema de análisis.

### 3.10 Conclusión del marco teórico

El análisis de la similitud de código fuente constituye un problema complejo que requiere la integración de múltiples perspectivas teóricas y técnicas. A lo largo de este marco teórico se ha evidenciado que no existe un enfoque único capaz de abordar de manera completa todas

las dimensiones del problema, desde la similitud superficial hasta la equivalencia semántica profunda.

En primer lugar, se estableció que la detección de clones de código presenta desafíos inherentes derivados de la diversidad de implementaciones, la ambigüedad estructural y la creciente influencia de herramientas automatizadas como los modelos de lenguaje. Asimismo, la clasificación de clones permitió comprender los distintos niveles de similitud, evidenciando que los enfoques más simples resultan insuficientes para capturar relaciones funcionales complejas.

Posteriormente, se analizaron los principales enfoques para la detección de similitud, observando una evolución progresiva desde métodos textuales y estructurales hasta modelos semánticos y basados en aprendizaje automático. Esta evolución refleja un compromiso constante entre precisión, interpretabilidad y costo computacional, donde cada paradigma aporta ventajas específicas pero también limitaciones significativas.

En este contexto, los modelos modernos basados en aprendizaje profundo, particularmente aquellos sustentados en arquitecturas Transformer y representaciones estructuradas como árboles y grafos, han demostrado una capacidad superior para capturar patrones complejos en el código. Sin embargo, su eficacia depende en gran medida de la representación utilizada y de la información disponible durante el proceso de aprendizaje.

De manera complementaria, el análisis estilométrico introduce una dimensión adicional al problema, al centrarse en la identificación de patrones de autoría más allá de la funcionalidad del código. Este enfoque permite capturar características individuales del programador, aunque enfrenta limitaciones cuando se consideran transformaciones que alteran la superficie del código sin modificar su comportamiento.

Finalmente, la integración de características (*feature fusion*) surge como una estrategia prometedora para superar las limitaciones de los enfoques individuales, al permitir la combinación de múltiples fuentes de información en una representación unificada. Este paradigma refuerza la idea de que las soluciones más efectivas no dependen de un único modelo, sino de la articulación de distintas técnicas de análisis.

En conjunto, el marco teórico desarrollado establece una base sólida para la comprensión del problema y de las herramientas disponibles, evidenciando la necesidad de diseñar soluciones híbridas que equilibren precisión, escalabilidad e interpretabilidad. A partir de este análisis, en las siguientes secciones se aborda el planteamiento del problema, así como la definición de los objetivos y la propuesta de solución, donde se seleccionarán y justificarán las técnicas más adecuadas en función de sus características y limitaciones.

## Capítulo 4

### Propuesta de solución

La propuesta de solución que se detalla a continuación constituye la respuesta técnica a la problemática en el proceso de evaluación analizada en los capítulos precedentes. A partir del planteamiento del problema y su justificación, se ha identificado la necesidad crítica de contar con herramientas de validación capaces de enfrentar los desafíos actuales en la enseñanza de la programación, especialmente ante el auge de soluciones generadas de forma automatizada.

Considerando las limitaciones detectadas en el estudio del estado del arte, donde los métodos de evaluación tradicionales han perdido efectividad frente a las nuevas técnicas de generación de código, y fundamentándose en los conceptos de análisis semántico y estilométrico explorados en el marco teórico, se propone el desarrollo de Graphito.

Graphito se define como un sistema de comparación de código diseñado para actuar como una herramienta de apoyo a la evaluación docente en cursos introductorios de programación en C/C++. Para lograr este objetivo, la solución se estructura mediante un pipeline de procesamiento híbrido de doble canal, el cual transforma el código fuente en representaciones vectoriales densas o embeddings. Por un lado, el análisis semántico es impulsado por el modelo GraphCodeBERT, que extrae el Grafo de Flujo de Datos (DFG) para capturar la esencia algorítmica del programa. Por otro lado, el análisis estilométrico emplea la red neuronal CharCNN, la cual procesa el código a nivel de caracteres para identificar patrones de escritura y detectar posibles indicios de generación mediante inteligencia artificial.

De esta manera, el sistema se constituye como un comparador integral que no solo realiza un contraste superficial, sino que coteja la estructura funcional y los patrones estilométricos de las entregas frente a soluciones de referencia, proporcionando métricas objetivas que funcionan como herramienta de apoyo a la decisión, brindando al docente la evidencia técnica necesaria para optimizar y fundamentar el proceso de evaluación de cada código. En las secciones siguientes se describe la estructura operativa de la solución, iniciando con la representación esquemática de sus componentes funcionales y el flujo de procesamiento de la información, tal como se ilustra en el siguiente diagrama de bloques.

## 4.1 Estructura Funcional del Sistema

Para realizar la comparación de código fuente, Graphito se articula a través de una arquitectura de bloques funcionales que procesan la información de manera secuencial y paralela. En la Figura 4.1 se presenta el diagrama de bloques que describe el flujo de los datos, desde la ingesta del código fuente hasta la generación del reporte de similitud.

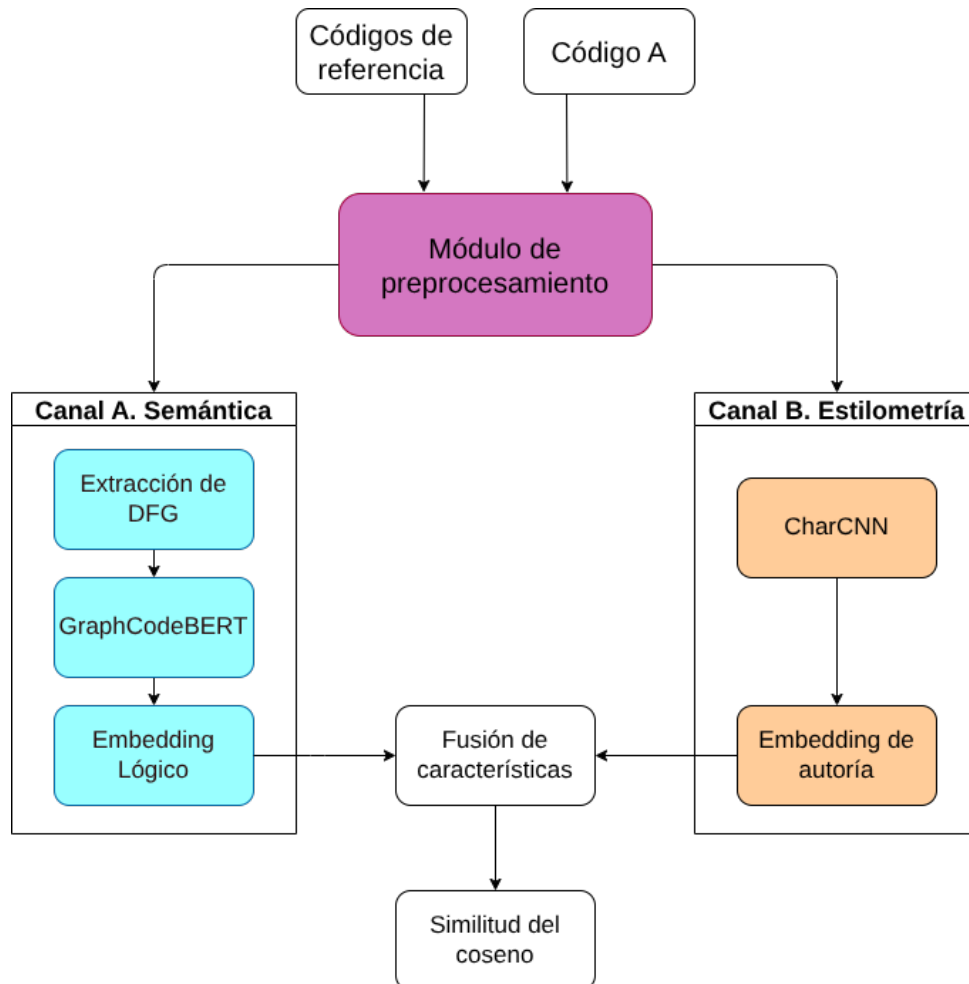


Figura 4.1: Diagrama de bloques funcional del sistema de comparación Graphito.

A continuación, se describen los componentes principales que integran la solución:

### 4.1.1 Entrada de datos y fuentes de referencia

El proceso de comparación comienza con la recepción de dos flujos de información fundamentales:

- **Entrega del Alumno:** Archivos en formato fuente (.c o .cpp) que contienen la resolución propuesta por el estudiante a un problema en específico. El sistema está diseñado para procesar código basado en los estándares C11 y C++17, enfocándose en temas introductorios a la programación como fundamentos y algoritmos y estructuras de datos.

- **Códigos de Referencia:** Actúan como el parámetro de comparación principal, albergando la solución ideal al problema planteado por el docente. De manera opcional, se complementa este conjunto con variantes funcionalmente equivalentes obtenidas mediante modelos de lenguaje (LLM). Esta diversidad de referencias permite que el sistema ejecute un cotejo robusto contra múltiples implementaciones lógicas de un mismo ejercicio.

#### 4.1.2 Módulo de Preprocesamiento y Normalización

Una vez ingresados los archivos, el **Módulo de Preprocesamiento** genera dos representaciones distintas a partir de un mismo archivo fuente para satisfacer los requerimientos de cada canal de análisis:

1. **Representación Normalizada (Canal A):** Se realiza una limpieza profunda de ambos conjuntos de código, eliminando comentarios, metadatos y espacios en blanco redundantes. Asimismo, se aplican técnicas de normalización de identificadores. Este proceso garantiza que el análisis semántico se enfoque exclusivamente en la topología del flujo de datos y la lógica algorítmica, ignorando cambios estéticos o de nomenclatura.
2. **Representación Estructural/Raw (Canal B):** Se aplica únicamente a la solución del alumno. El sistema preserva elementos como la indentación, el uso de comentarios, la acentuación y caracteres especiales. Esta decisión técnica es clave para la detección de patrones de generación por IA, ya que permite identificar la perfección gramatical y hábitos de formato que constituyen la huella digital del autor o del modelo generativo utilizado.

Una vez terminados ambos procesos, el flujo sigue hacia los canales de comparación semántica y análisis estilométrico.

#### 4.1.3 Canal A: Comparación Semántica

Este canal constituye el núcleo para la evaluación de equivalencias lógicas, cuyo objetivo es medir la similitud semántica entre soluciones que resuelven el mismo problema algorítmico, independientemente de sus variaciones sintácticas superficiales.

##### 4.1.3.1 Justificación y Selección del Modelo

La selección de **GraphCodeBERT** [5] como motor de análisis semántico responde a la necesidad de superar las limitaciones de los modelos puramente secuenciales (como CodeBERT o Transformers estándar). La decisión se fundamenta en los siguientes pilares técnicos:

- **Integración Estructural:** A diferencia de otros modelos que tratan el código como una simple secuencia de texto, GraphCodeBERT integra información estructural mediante el

uso de grafos de flujo de datos (DFG). Esto permite que el mecanismo de atención se centre en las conexiones que tienen sentido semántico real.

- **Resistencia a la Ofuscación:** Debido a que el modelo captura la relación entre variables, es altamente efectivo detectando similitudes incluso cuando el código ha sido alterado mediante cambios de nombres de variables, reordenamiento de sentencias o cambios en las estructuras de control.
- **Especialización Funcional:** Como se detalla en el marco teórico, este modelo se especializa en el comportamiento lógico del programa. Si bien tiende a ignorar rasgos superficiales como el estilo, esta característica se alinea perfectamente con la arquitectura de doble canal de Graphito, delegando la responsabilidad del análisis de estilo al Canal B (CharCNN).

#### 4.1.3.2 Extracción de Grafo de Flujo de Datos (DFG)

Un Grafo de Flujo de Datos (DFG) es una representación dirigida que captura las dependencias funcionales entre las variables de un programa mediante la identificación de sus relaciones de definición y uso. De acuerdo con la metodología propuesta por Guo et al. [5], los pasos para obtener esta representación son:

1. **Generación del AST (Análisis sintáctico):** El código fuente se procesa para construir su Árbol de Sintaxis Abstracta (AST), obteniendo la jerarquía gramatical del programa.
2. **Identificación de la secuencia de variables:** Se analizan los nodos terminales (hojas) del AST para extraer todas las variables presentes, preservando su orden de aparición.
3. **Definición de los nodos del grafo:** Cada ocurrencia de una variable se establece como un nodo individual dentro del grafo.
4. **Construcción de las aristas (Dependencias de valor):** Se trazan enlaces dirigidos entre nodos  $v_i$  y  $v_j$  si el valor de la variable  $v_j$  depende directamente de la definición o valor previo de  $v_i$ .

#### 4.1.3.3 Procesamiento en GraphCodeBERT

Una vez construido el DFG, la secuencia de *tokens* del código y la estructura del grafo se integran en el modelo. La arquitectura utiliza tareas de entrenamiento como la predicción de relaciones entre nodos, lo que favorece el aprendizaje de representaciones mucho más estructuradas y fieles al comportamiento funcional del software.

#### 4.1.3.4 Embedding Lógico

El resultado final de este canal es un **Embedding Semántico**: un vector denso de alta dimensionalidad que actúa como una firma lógica del programa. Este vector permite realizar comparaciones de proximidad matemática, identificando si dos soluciones son funcionalmente iguales sin importar la forma en que fueron escritas.

#### 4.1.4 Canal B: Comparación Estilométrica

Mientras que el Canal A se encarga de la profundidad lógica, el Canal B se enfoca en la superficie del código. Para esta tarea, se seleccionó el modelo **CharCNN** [7] sobre otras arquitecturas de procesamiento de lenguaje natural más complejas por una razón de eficiencia y diseño: al contar ya con un modelo de alta capacidad para el análisis semántico, no es necesario un segundo modelo que intente comprender la sintaxis global. En su lugar, se requiere un extractor de rasgos que sea capaz de “ver” el código como una señal visual.

##### 4.1.4.1 Justificación y Selección del Modelo

La elección de una Red Neuronal Convolutiva a nivel de caracteres (CharCNN) responde a su capacidad innata para actuar como un escáner de patrones locales. Dado que las CNN son especialistas en visión computacional, al aplicarlas al código fuente en bruto (raw), el modelo puede identificar automáticamente rasgos que para otros sistemas serían invisibles:

- **Estilo como Señal Visual:** El modelo interpreta la indentación, el uso de espacios y la disposición de caracteres especiales como texturas visuales, permitiendo aprender el estilo del autor sin necesidad de conocer la gramática del lenguaje C++.
- **Detección de consistencia sintética:** A diferencia de un estudiante, cuya escritura presenta errores humanos y variaciones de formato, los modelos de lenguaje generan código con una estructura y gramática sumamente uniformes en sus comentarios. CharCNN resulta ideal para detectar esta estandarización algorítmica, convirtiéndola en una huella digital que evidencia el uso de inteligencia artificial.
- **Eficiencia Computacional:** Al operar localmente sobre secuencias de caracteres, el modelo aporta una capa de seguridad adicional sin incrementar drásticamente la carga computacional del sistema *backend*.

##### 4.1.4.2 Análisis de Autoría mediante CharCNN

El procedimiento estructurado para transformar el código fuente en un identificador de estilo se detalla a continuación:

1. **Preservación de Código Raw:** Se utiliza la versión del código que conserva comentarios, sangrías y acentuación, elementos que suelen ser eliminados en análisis tradicionales pero que son críticos para la detección de la firma del autor.
2. **Mapeo de Caracteres:** El texto se transforma en una secuencia de índices numéricos basados en un alfabeto predefinido, permitiendo que la red procese el código como una señal discreta y medible.
3. **Extracción Convolutiva de Rasgos:** Se aplican filtros convolucionales de distintos tamaños para detectar “n-gramas” de caracteres. Esto permite identificar hábitos microscópicos, como la preferencia por ciertos operadores o la pulcritud en la redacción de comentarios.
4. **Reducción de Dimensionalidad (Pooling):** Se seleccionan las características más relevantes detectadas por los filtros, consolidando los rasgos estilísticos más distintivos de la muestra analizada en una representación compacta.

#### 4.1.4.3 Cálculo de probabilidad de generación por IA

Al finalizar la extracción de patrones locales mediante los filtros convolucionales, la red CharCNN genera una métrica escalar que determina la **probabilidad de generación por inteligencia artificial**.

Esta métrica complementa directamente al Canal A: mientras que el módulo semántico analiza la lógica interna del código para compararla con la solución ideal del profesor, este escalar actúa como un indicador cuantitativo de los patrones estilísticos. Su función es proporcionar al docente evidencia objetiva sobre la homogeneidad sintética del texto y la ausencia de variaciones típicas de la escritura humana, dotándolo de los elementos técnicos necesarios para que tome la decisión de evaluación final bajo su propio criterio.

#### 4.1.5 Fusión de Características

La fusión de características [6] representa el proceso de integración donde las salidas de los canales semántico y estilométrico convergen para formar una representación única de la entrega.

Este bloque técnico es fundamental para el sistema Graphito, ya que permite consolidar dos dimensiones de análisis distintas en un solo espacio vectorial:

- **Mecanismo de Integración:** El sistema toma el **Embedding Semántico** (que encapsula la lógica extraída por GraphCodeBERT) y la probabilidad de generación por IA (que representa los rasgos estructurales detectados por CharCNN) y los combina mediante una operación de concatenación. En el caso de las soluciones de referencia, se aplica un relleno con ceros (*zero-padding*) en la dimension correspondiente al canal estilométrico, garantizando la homogeneidad dimensional necesaria para el cálculo de similitud.

- **Generación de la representación:** El resultado de esta fusión es un vector de características unificado que actúa como una representación del código. Esta no solo describe qué funciones realiza el programa, sino que también preserva los hábitos de escritura y formato del autor del código.

Al integrar los resultados de ambos canales, el sistema genera un perfil completo del código fuente. Esta unión permite cruzar dos datos clave: qué hace el programa (su lógica) y cómo fue escrito (su estilo). Esto facilita la identificación de casos inusuales; por ejemplo, un código que resuelve el problema de forma idéntica a la referencia, pero que tiene el formato uniforme típico de una inteligencia artificial. Así, el sistema entrega al docente información clara y medible para respaldar su decisión final de evaluación.

#### 4.1.6 Similitud del Coseno

La Similitud del Coseno es una métrica matemática que mide el coseno del ángulo entre dos vectores en un espacio multidimensional. A diferencia de otras métricas de distancia, esta no se ve afectada por la magnitud de los vectores (en este caso, la longitud del código), sino por su orientación, lo que la hace ideal para comparar la esencia de dos fragmentos de software.

La similitud entre la entrega del alumno ( $A$ ) y el código de referencia ( $R$ ) se calcula mediante la siguiente fórmula:

$$\text{sim}(A, R) = \frac{A \cdot R}{\|A\| \|R\|} = \frac{\sum_{i=1}^n A_i R_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n R_i^2}} \quad (4.1)$$

Donde:

- $A$  y  $R$  representan los vectores de características (*embeddings* bimodales) de la entrega del alumno y la solución de referencia, respectivamente.
- $A_i$  y  $R_i$  son los valores escalares de dichos vectores en la dimensión  $i$ .
- $\|A\|$  y  $\|R\|$  denotan la norma euclidiana (o magnitud) de cada vector.
- $n$  es la dimensionalidad total del espacio vectorial unificado.

El resultado de esta operación oscila en el intervalo  $[0, 1]$ , donde 1 representa una coincidencia total de orientación y 0 una independencia absoluta entre los vectores.

##### 4.1.6.1 Importancia de la métrica en la fusión de características

El uso de la similitud del coseno es fundamental en Graphito debido a que permite comparar dos representaciones en un mismo espacio vectorial: las características lógicas multidimensionales

(Canal A) y el escalar de probabilidad estilométrica (Canal B). Al combinarlos, esta métrica matemática proporciona un indicador objetivo que optimiza el proceso de revisión docente.

La pertinencia de utilizar este enfoque radica en su comportamiento geométrico frente a los vectores evaluados:

- **Impacto del indicador estilométrico:** En el sistema, el código de referencia proporcionado por el profesor se establece con una probabilidad nula de haber sido generado por IA (valor de 0 en la dimensión estilométrica). Si una entrega estudiantil posee una alta probabilidad de origen sintético, adquiere un valor escalar mayor en dicha dimensión, lo que altera matemáticamente su vector unificado.
- **Variación en la orientación espacial:** Aunque el código del alumno y el de referencia compartan una alta similitud semántica (lógica algorítmica), el valor adicional aportado por el modelo CharCNN modifica la orientación espacial del vector del estudiante. Este incremento en el ángulo entre ambos vectores se traduce directamente en una reducción del índice de similitud.
- **Ajuste cuantitativo de la similitud:** De acuerdo con la fórmula matemática, la magnitud del vector de la entrega ( $\|A\|$ ) se encuentra en el denominador. Dado que la probabilidad sintética incrementa esta magnitud sin aportar al producto escalar del numerador (ya que se multiplica por el 0 de la referencia), el resultado global disminuye naturalmente. Esto permite que el sistema integre la detección de IA como una reducción proporcional en la similitud sin necesidad de programar penalizaciones arbitrarias.

En conclusión, la similitud del coseno permite a Graphito entregar un indicador numérico robusto. En lugar de limitarse a una verificación funcional o textual, el sistema evalúa simultáneamente la topología algorítmica y la huella estilométrica, brindando al docente evidencia cuantitativa clara para agilizar y fundamentar su proceso de evaluación.

## 4.2 Conclusión

La propuesta de solución presentada en este capítulo, denominada **Graphito**, se aleja de los enfoques de comparación de texto tradicionales para adoptar una arquitectura de doble canal basada en aprendizaje profundo. A través de la integración del análisis semántico (Canal A) y el análisis estilométrico (Canal B), el sistema es capaz de evaluar el código fuente no solo por lo que hace, sino por el estilo de su escritura.

La principal fortaleza de esta propuesta reside en su capacidad de autorregulación mediante la similitud del coseno. Al tratar la lógica como el eje central y el estilo como un factor de penalización, Graphito ofrece una métrica de integridad académica robusta frente a las técnicas modernas de ofuscación y la generación sintética por modelos de lenguaje.

Con la definición funcional de estos bloques y la fundamentación de sus componentes principales, queda establecida la base teórica y metodológica de la solución. En el siguiente capítulo, se procederá con el Análisis, donde se detallará la arquitectura de software, la persistencia de datos y los diagramas de interacción necesarios para la implementación física de esta propuesta.

# Capítulo 5

## Análisis del Sistema

### 5.1 Análisis de Requerimientos

En esta sección se describen los requerimientos funcionales y no funcionales del sistema propuesto, los cuales establecen las capacidades, restricciones y criterios de desempeño que debe cumplir la solución desarrollada. Estos requerimientos sirven como base para el diseño, implementación y evaluación del sistema, asegurando la trazabilidad entre las necesidades del evaluador y la arquitectura final.

#### 5.1.1 Requerimientos Funcionales

Los requerimientos funcionales definen las capacidades y procesos que el sistema debe implementar para cumplir con las funciones de análisis, gestión de información y auditoría de código en el contexto académico.

- **RF-01:** El sistema debe permitir el registro e inicio de sesión de docentes mediante credenciales cifradas para garantizar un espacio de trabajo privado.
- **RF-02:** El sistema debe permitir al docente la creación y gestión de un listado de ejercicios o problemas de programación (Biblioteca).
- **RF-03:** El sistema debe permitir la carga de código fuente en formato .c y .cpp asociados a un ejercicio específico.
- **RF-04:** El sistema debe realizar el preprocesamiento y normalización del código fuente, asegurando la correcta extracción de los grafos para el canal semántico, mientras se preserva la integridad de las estructuras de estilo originales para la detección de patrones generativos.
- **RF-05:** El sistema debe generar representaciones basadas en Grafos de Flujo de Datos (DFG) para el módulo semántico, y transformar el código fuente en secuencias matriciales

bidimensionales (carácter por vector) para su procesamiento mediante convoluciones 1D en el módulo estilométrico.

- **RF-06:** El sistema debe realizar el análisis de similitud semántica mediante el modelo GraphCodeBERT para la generación de vectores de características (embeddings), cuantificando la proximidad entre ellos utilizando la métrica de similitud del coseno.
- **RF-07:** El sistema debe realizar el análisis estilométrico mediante el modelo CharCNN para calcular la probabilidad de generación mediante inteligencia artificial sintética o humana.
- **RF-08:** El sistema debe solicitar y recibir múltiples versiones funcionalmente equivalentes del problema proporcionado por el docente a través de un modelo LLM externo.
- **RF-09:** El sistema debe realizar el almacenamiento persistente de los resultados del análisis y metadatos en una base de datos relacional (PostgreSQL), así como de los vectores de características en una base de datos vectorial (ChromaDB), asegurando la integridad referencial.
- **RF-10:** El sistema debe generar un reporte consolidado que integre el índice de similitud semántica y la probabilidad de generación mediante inteligencia artificial de forma visual.
- **RF-11:** El sistema debe realizar una fusión de características (Feature Fusion) que integre el vector semántico y la probabilidad estilométrica, consolidando las métricas de ambos módulos en una sola estructura para su análisis dual.
- **RF-12:** El sistema debe permitir la consulta histórica de reportes de análisis realizados previamente sin necesidad de re-procesar los archivos fuente.

### 5.1.2 Requerimientos No Funcionales

Los requerimientos no funcionales establecen las restricciones, atributos de calidad y condiciones de desempeño que debe cumplir el sistema propuesto durante su operación.

- **RNF-01:** Seguridad de datos: El sistema debe cifrar las contraseñas de los usuarios y utilizar tokens de sesión (JWT) para proteger la comunicación entre el cliente y el servidor.
- **RNF-02:** Persistencia híbrida: El sistema debe operar bajo un esquema de base de datos dual, optimizando las consultas relacionales en PostgreSQL y las búsquedas por similitud vectorial en ChromaDB.
- **RNF-03:** Los modelos de IA integrados deben estar calibrados buscando alcanzar un objetivo de rendimiento base de F1-Score  $\geq 0.85$  para GraphCodeBERT en tareas de detección de similitud semántica.

- **RNF-04:** El módulo estilométrico (CharCNN) debe estar optimizado para buscar una precisión de clasificación de autoría igual o superior a 0.8 bajo el corpus de pruebas establecido.
- **RNF-05:** El sistema debe soportar el análisis de código bajo los estándares C11 y C++17 para garantizar compatibilidad operativa con el currículo y las entregas esperadas.
- **RNF-06:** El orquestador de métricas debe ajustar los umbrales de decisión con el objetivo de mantener una tasa de falsos positivos menor al 10 % durante el análisis de doble canal.
- **RNF-07:** El tiempo de procesamiento de inferencia en el backend durante la fase de análisis de una entrega estudiantil no debe exceder los 20 segundos, dado que la indexación de referencias y la generación de variantes (LLM) se realizan de forma asíncrona en la fase de configuración previa.
- **RNF-08:** El sistema debe ser compatible para su ejecución en navegadores web basados en el motor Chromium versión 110 o superior.
- **RNF-09:** La interfaz web debe ser intuitiva, permitiendo ejecutar todas las tareas en 3 o menos acciones y proporcionar retroalimentación visual o indicadores de estado en tiempo real mientras el orquestador (LangGraph) procesa las tareas en segundo plano.
- **RNF-10:** El sistema debe asegurar alta disponibilidad operativa durante el procesamiento y análisis de código dentro del entorno de despliegue asignado para las pruebas.
- **RNF-11:** El sistema debe realizar validaciones preventivas en el cliente (tipo de archivo, tamaño e integridad) antes de iniciar la transmisión de la carga útil hacia la API REST.

### 5.1.3 Reglas de Negocio

Las reglas de negocio definen los criterios que guían el funcionamiento y uso ético del sistema dentro del entorno académico.

- **RN-01:** Confidencialidad de los datos: Un docente solo podrá visualizar, editar y consultar los ejercicios y reportes generados bajo su propia cuenta de usuario.
- **RN-02:** El sistema se limita a programas de cursos de programación de ESCOM-IPN que sigan los estándares C11 y C++17.
- **RN-03:** Para que el sistema pueda hacer una comparación, debe existir al menos un código de referencia previamente cargado. Sin ella, el sistema no tiene un punto de comparación y no puede generar ningún resultado.
- **RN-04:** La decisión final de calificación es responsabilidad exclusiva del docente; el sistema actúa únicamente como una herramienta objetiva de apoyo a la auditoría académica.

- **RN-05:** Se permite el uso de múltiples soluciones generadas por LLM para ampliar el espectro de comparación lógica, siempre que el docente las valide inicialmente.
- **RN-06:** Preservación de referencias: Los archivos cargados en calidad de entregas a analizar no podrán ser catalogados ni reutilizados por el sistema como códigos de referencia para futuras comparaciones, garantizando que el estándar de comparación se mantenga basado exclusivamente en soluciones validadas o generadas por el docente.

## 5.2 Identificación y descripción de actores

En el contexto del sistema propuesto (Graphito), los actores representan las entidades externas que interactúan con la plataforma y que participan en el flujo de configuración, análisis y auditoría del código fuente. La correcta identificación de estos actores permite definir los casos de uso, delimitar responsabilidades y establecer las fronteras del sistema.

A partir de la arquitectura funcional, se identifican exclusivamente dos actores que interactúan de forma directa con la plataforma:

### 5.2.1 Docente / Evaluador

Es el actor principal y humano del sistema. Utiliza la plataforma como una herramienta de apoyo a la decisión en su proceso de auditoría académica (correspondiente a profesores o instructores de programación).

Entre sus principales responsabilidades de interacción se encuentran:

- Proporcionar la descripción técnica del problema o práctica.
- Configurar el marco de comparación cargando sus códigos de referencia.
- Cargar los archivos fuente (.c, .cpp) desarrollados por los estudiantes para su análisis.
- Interpretar los reportes visuales generados por el sistema, incluyendo los índices de similitud semántica y la probabilidad de generación mediante inteligencia artificial.
- Tomar la decisión final sobre la calificación del alumno, utilizando la evidencia estática proporcionada por la herramienta.

### 5.2.2 Motor de Inteligencia Artificial Externo (LLM)

Este actor corresponde a una API de un sistema externo basado en Modelos de Lenguaje Grande (LLM). Actúa como un proveedor de servicios de generación de código sintético bajo demanda.

Sus funciones en interacción con el orquestador de Graphito son:

- Recibir el código de referencia original y generar descripciones técnicas algorítmicas.
- Generar múltiples variantes de código que sean funcionalmente equivalentes a la referencia del docente.
- Proveer el insumo necesario para que el sistema indexe un espacio vectorial de comparación mucho más robusto.

### **5.3 Trayectorias principales y alternativas**

Las trayectorias describen el flujo de interacción entre el docente, el orquestador backend y los modelos de IA, detallando las secuencias de acciones para llevar a cabo el análisis bimodal. Estas trayectorias reflejan el comportamiento esperado (Modelado BPMN) y los escenarios alternativos.

#### **5.3.1 Trayectoria principal (Flujo de Análisis Estándar)**

La Figura 5.1 ilustra la trayectoria principal del sistema Graphito, correspondiente al flujo de análisis estándar. Durante este proceso, el docente ingresa el código de referencia y las entregas estudiantiles, configura los parámetros de evaluación e inicia el procesamiento de doble canal. Posteriormente, el sistema ejecuta de manera concurrente el análisis semántico y el análisis estilométrico del código fuente, consolidando métricas e indicadores objetivos que respaldan la toma de decisiones del evaluador.

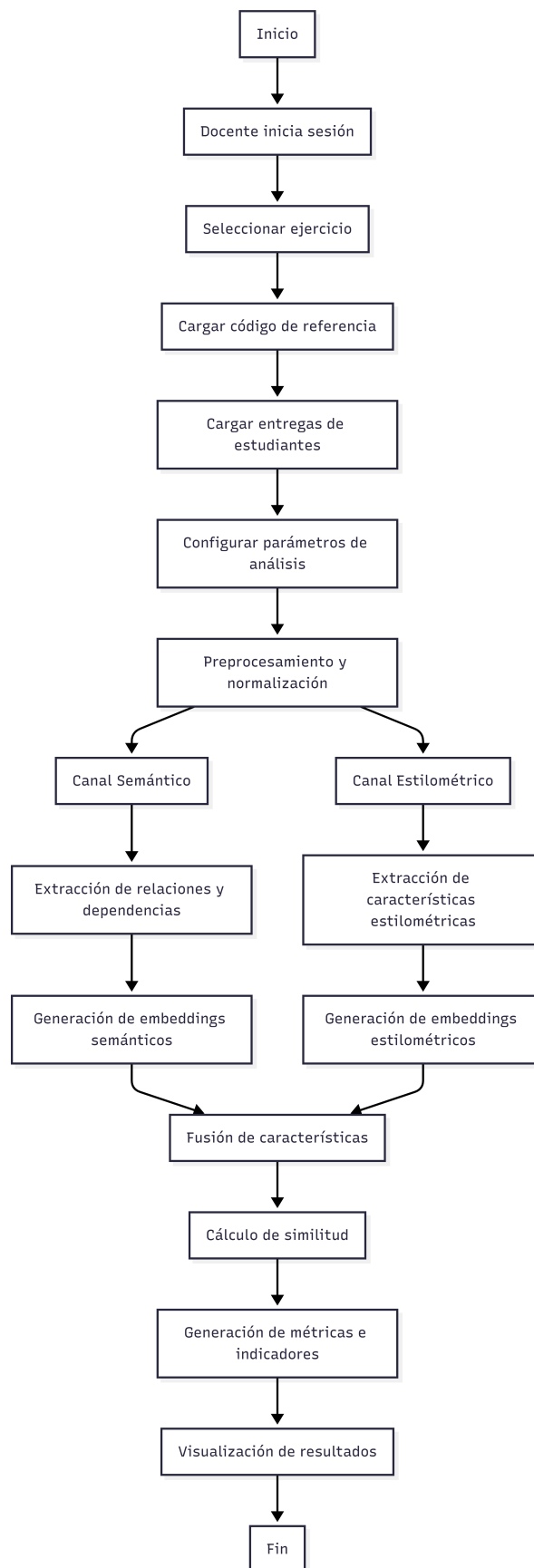


Figura 5.1: Diagrama de trayectoria principal del sistema Graphito correspondiente al flujo de análisis estándar.

1. El docente accede a su biblioteca en la plataforma web del sistema.

2. El docente carga el archivo fuente a analizar, seleccionando el problema correspondiente.
3. El backend recibe el código y ejecuta el preprocesamiento y normalización (limpieza para el canal semántico y preservación de formato para el estilométrico).
4. **Canal Semántico:** Se extrae el Grafo de Flujo de Datos (DFG) y el modelo *GraphCodeBERT* genera el embedding estructural.
5. **Canal Estilométrico:** El modelo *CharCNN* procesa la matriz de caracteres 1D para estimar la probabilidad de generación mediante inteligencia artificial.
6. El orquestador ejecuta la **Fusión de Características (Feature Fusion)** unificando ambos resultados.
7. El sistema calcula la similitud del coseno del vector unificado contra la base de datos vectorial (*ChromaDB*).
8. El sistema consolida los resultados y persiste los metadatos en la base de datos relacional (*PostgreSQL*).
9. El sistema renderiza el reporte de análisis dual.
10. El docente visualiza los indicadores numéricos y toma una decisión informada.

### 5.3.2 Trayectorias alternativas

Las trayectorias alternativas describen variaciones del flujo principal, orquestadas por *Lang-Graph* ante condiciones específicas.

#### A1. Robustecimiento de referencias mediante LLM

1. Durante la configuración de un problema, el docente solicita la generación de variantes de su solución.
2. El sistema envía el código de referencia al LLM externo.
3. El modelo externo retorna múltiples soluciones equivalentes.
4. El sistema calcula los embeddings semánticos de estas variantes y los indexa en la base vectorial de forma asíncrona.

#### A2. Código con errores críticos de formato

1. El sistema detecta inconsistencias irreparables durante la normalización inicial del código.
2. El orquestador detiene el flujo antes de la inferencia de los modelos.

3. Se notifica al docente en la interfaz sobre la invalidez del archivo fuente.

### **A3. Análisis estilométrico de baja confianza**

1. El sistema identifica que el código cargado carece de una extensión mínima para extraer patrones de caracteres viables.
2. El orquestador reduce la ponderación del canal estilométrico en la fusión de características.
3. El sistema prioriza el índice de similitud semántica y añade una nota técnica en el reporte final.

### **A4. Alta equivalencia lógica detectada**

1. El cálculo de la similitud del coseno arroja un valor superior al umbral establecido.
2. El sistema clasifica la entrega como de alto interés de auditoría.
3. Se destaca visualmente el registro en el panel del docente para sugerir una revisión prioritaria.

### **A5. Discrepancia crítica en análisis de doble canal**

1. El orquestador detecta una correlación anómala (ej. alta similitud lógica combinada con una huella de autoría clasificada como inteligencia artificial).
2. El sistema estructura un indicador visual de inconsistencia en el reporte final.
3. El docente analiza el reporte detallado para auditar la anomalía.

## **5.4 Diagrama de Casos de Uso**

El análisis de casos de uso define las fronteras del sistema Graphito y describe las interacciones entre los actores externos y las funcionalidades internas. Este modelo permite visualizar cómo el evaluador gestiona el proceso de auditoría académica, desde la configuración inicial hasta la toma de decisiones basada en el reporte bimodal.

En la Figura 5.2 se presenta el diagrama de casos de uso general del sistema, estructurado de forma jerárquica para reflejar el flujo de trabajo del docente.

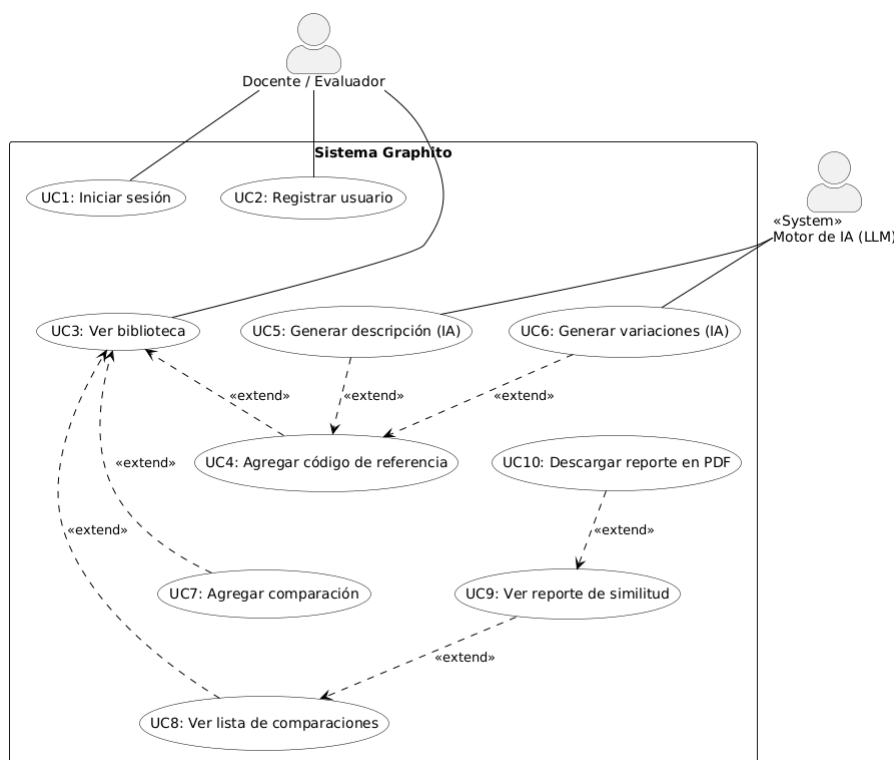


Figura 5.2: Diagrama de casos de uso del sistema Graphito.

#### 5.4.1 Descripción de Actores

Para el funcionamiento del sistema se han identificado dos actores principales:

- **Docente / Evaluador:** Actor principal humano que interactúa con la plataforma para configurar problemas, gestionar entregas y visualizar los reportes de similitud.
- **Modelo LLM Externo:** Actor de sistema que actúa como proveedor de servicios de generación de código. Es consultado de forma opcional para robustecer el conjunto de referencias.

#### 5.4.2 Descripción de los Casos de Uso

A continuación, se describen los módulos funcionales que integran la solución *Graphito*, estructurados de acuerdo con su rol en el flujo de auditoría académica:

- **Gestión de Acceso y Biblioteca (UC1, UC2, UC3):** Facilitan la administración de la sesión del docente y el acceso a la pagina principal (Biblioteca). Este módulo centraliza los ejercicios históricos y las comparaciones realizadas, permitiendo un entorno de trabajo persistente.
- **Gestión de Referencias y Pre-procesamiento (UC4, UC5, UC6, UC11):** Permiten al evaluador establecer el marco de comparación. Incluye la carga de soluciones base (UC4)

y su enriquecimiento mediante un motor de IA externo para generar descripciones y variantes sintéticas (UC5, UC6). De forma obligatoria (*include*), el sistema realiza la generación de *embeddings* semánticos (UC11) sobre estas referencias.

- **Ejecución de Comparaciones (UC7, UC12):** Constituye el núcleo operativo del sistema. Al agregar una entrega para comparar (UC7), el sistema ejecuta automáticamente (*include*) el análisis de doble canal (UC12), el cual integra el procesamiento semántico de flujo de datos y la detección estilométrica de patrones generativos.
- **Resultados y Reporteo (UC8, UC9, UC10):** Proporcionan la interfaz de auditoría donde se listan las comparaciones (UC8) y se visualizan los reportes de similitud detallados (UC9). Finalmente, se permite la exportación de resultados en formato PDF (UC10) para su integración en actas de evaluación.

### 5.4.3 Especificación de Casos de Uso

A continuación se detallan los casos de uso que rigen el comportamiento del sistema Graphito. Estas especificaciones describen la interacción detallada entre los actores y los módulos de inteligencia artificial.

|                                   |                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>               | <b>UC1: Iniciar sesión</b>                                                                                                                                                                                                                                                                                                                                                  |
| <b>Actor:</b>                     | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                         |
| <b>Descripción:</b>               | El docente ingresa sus credenciales de acceso para entrar al sistema Graphito.                                                                                                                                                                                                                                                                                              |
| <b>Precondiciones:</b>            | El docente debe estar registrado previamente en la plataforma (UC2).                                                                                                                                                                                                                                                                                                        |
| <b>Flujo Básico:</b>              | <ol style="list-style-type: none"> <li>1. El docente ingresa su correo electrónico y contraseña en la interfaz.</li> <li>2. El sistema valida las credenciales mediante un proceso de autenticación cifrada contra la base de datos relacional (PostgreSQL).</li> <li>3. El sistema genera un token de sesión (JWT) y redirige al docente a su Biblioteca (UC3).</li> </ol> |
| <b>Trayectorias Alternativas:</b> | Si las credenciales son incorrectas, el sistema muestra un mensaje de error y deniega el acceso, manteniendo la sesión cerrada.                                                                                                                                                                                                                                             |

Tabla 5.1: Especificación del Caso de Uso 1.

|                       |                                                                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>   | <b>UC2: Registrar usuario</b>                                                                                                                                                                                                                                                                                           |
| <b>Actor:</b>         | Docente / Evaluador                                                                                                                                                                                                                                                                                                     |
| <b>Descripción:</b>   | Un nuevo docente crea una cuenta en la plataforma para poder gestionar sus ejercicios, referencias y comparaciones.                                                                                                                                                                                                     |
| <b>Flujo Básico:</b>  | <ol style="list-style-type: none"> <li>1. El docente proporciona su nombre, correo electrónico y define una contraseña.</li> <li>2. El sistema cifra la contraseña y registra al nuevo usuario en PostgreSQL.</li> <li>3. El sistema confirma el registro exitoso y redirige a la vista de inicio de sesión.</li> </ol> |
| <b>Postcondición:</b> | Se inicializa un espacio aislado en la base de datos para el repositorio personal del docente.                                                                                                                                                                                                                          |

Tabla 5.2: Especificación del Caso de Uso 2.

|                        |                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC3: Ver biblioteca</b>                                                                                                                                                                                                                                                                                                                                            |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                   |
| <b>Descripción:</b>    | Página principal del sistema donde el docente accede a su biblioteca personal, desde donde visualiza los problemas configurados previamente o inicia nuevas configuraciones.                                                                                                                                                                                          |
| <b>Precondiciones:</b> | El docente debe haber iniciado sesión de forma exitosa (UC1).                                                                                                                                                                                                                                                                                                         |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente selecciona la opción de “Biblioteca” en la interfaz.</li> <li>2. El sistema recupera de PostgreSQL la lista de ejercicios asociados al identificador del docente.</li> <li>3. El sistema muestra la lista de ejercicios, habilitando las opciones de gestión (agregar referencia, ver comparaciones).</li> </ol> |
| <b>Postcondición:</b>  | El sistema queda a la espera de la ejecución de una acción extendida (UC4, UC7, UC8).                                                                                                                                                                                                                                                                                 |

Tabla 5.3: Especificación del Caso de Uso 3.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC4: Agregar código de referencia</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Descripción:</b>    | El docente carga el código fuente que servirá como solución ideal, inicializando el proceso asíncrono de indexación semántica.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Precondiciones:</b> | El docente debe encontrarse gestionando un ejercicio dentro de la Biblioteca (UC3).                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente selecciona agregar una nueva referencia y carga el archivo (.c o .cpp).</li> <li>2. El sistema normaliza el código eliminando comentarios.</li> <li>3. El orquestador extrae el Grafo de Flujo de Datos (DFG) y genera el embedding correspondiente mediante GraphCodeBERT.</li> <li>4. El vector semántico resultante se almacena en ChromaDB.</li> <li>5. El sistema habilita las opciones para enriquecer la referencia utilizando un modelo de IA externo (UC5, UC6).</li> </ol> |
| <b>Postcondición:</b>  | El código base queda indexado en la base vectorial, preparado para futuras comparaciones.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

Tabla 5.4: Especificación del Caso de Uso 4.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC5: Generar descripción (IA)</b>                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Actor:</b>          | Motor de IA (LLM) / Docente                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Descripción:</b>    | El sistema consume una API de LLM externo para redactar automáticamente un resumen o descripción técnica basada en la lógica del código de referencia.                                                                                                                                                                                                                                                                                           |
| <b>Precondiciones:</b> | Debe existir un código de referencia cargado en el sistema (UC4).                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente solicita la autogeneración de la descripción del problema.</li> <li>2. El sistema envía el código crudo al LLM empaquetado en un prompt específico de resumen algorítmico.</li> <li>3. El LLM devuelve la descripción técnica generada.</li> <li>4. El docente revisa el texto, realiza ajustes si lo considera necesario y confirma su guardado en los metadatos de PostgreSQL.</li> </ol> |

Tabla 5.5: Especificación del Caso de Uso 5.

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>               | <b>UC6: Generar variaciones (IA)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Actor:</b>                     | Motor de IA (LLM) / Docente                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Descripción:</b>               | El sistema utiliza el LLM externo para generar variantes funcionalmente equivalentes al código base.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Precondiciones:</b>            | Se debe haber cargado exitosamente al menos un código de referencia manual (UC4).                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Flujo Básico:</b>              | <ol style="list-style-type: none"> <li>1. El docente solicita la generación de N variantes para robustecer las referencias, otorgando consideraciones para la generación de las mismas.</li> <li>2. El sistema envía el código de referencia y las consideraciones al LLM externo.</li> <li>3. El LLM retorna N versiones sintéticas del código con la lógica requerida.</li> <li>4. El sistema preprocesa estas variantes, genera sus respectivos embeddings semánticos a través de GraphCodeBERT y las indexa en ChromaDB.</li> </ol> |
| <b>Trayectorias Alternativas:</b> | Si la API del LLM no responde o excede el tiempo límite, el sistema notifica al docente y finaliza el proceso manteniendo activa únicamente la referencia original.                                                                                                                                                                                                                                                                                                                                                                     |

Tabla 5.6: Especificación del Caso de Uso 6.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC7: Agregar comparación</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Descripción:</b>    | El docente carga una entrega estudiantil para ser comparada con el repositorio de referencias indexadas, ejecutando el análisis de los modelos de Deep Learning.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Precondiciones:</b> | El problema debe contar con al menos una referencia previamente indexada en ChromaDB.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente carga el archivo (.c o .cpp) correspondiente a la entrega a evaluar.</li> <li>2. El sistema inicia la inferencia paralela: GraphCodeBERT calcula el vector lógico (semántico) y CharCNN calcula la probabilidad de generación mediante inteligencia artificial sintética (estilométrico).</li> <li>3. El orquestador ejecuta la Fusión de Características (Feature Fusion) para consolidar las métricas de doble canal.</li> <li>4. Se ejecuta una búsqueda matemática de Similitud del Coseno contra los vectores indexados en ChromaDB.</li> <li>5. Los resultados se persisten en PostgreSQL vinculados exclusivamente al nombre del archivo analizado.</li> </ol> |
| <b>Postcondición:</b>  | La entrega finaliza la comparación y queda disponible para su consulta detallada (UC8).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

Tabla 5.7: Especificación del Caso de Uso 7.

|                        |                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC8: Ver lista de comparaciones</b>                                                                                                                                                                                                                                                                                                                              |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                 |
| <b>Descripción:</b>    | El docente visualiza un panel con el listado general de todas las entregas que han completado su proceso de comparación para un ejercicio específico.                                                                                                                                                                                                               |
| <b>Precondiciones:</b> | El docente debe haber seleccionado un ejercicio activo desde la Biblioteca (UC3).                                                                                                                                                                                                                                                                                   |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente accede a la pestaña de comparaciones del problema seleccionado.</li> <li>2. El sistema consulta en PostgreSQL los metadatos de los análisis vinculados a dicho ejercicio.</li> <li>3. El sistema despliega una tabla resumiendo los nombres de los archivos procesados y sus puntajes consolidados.</li> </ol> |
| <b>Postcondición:</b>  | El docente tiene la opción de seleccionar un archivo específico para abrir su reporte (UC9).                                                                                                                                                                                                                                                                        |

Tabla 5.8: Especificación del Caso de Uso 8.

|                        |                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC9: Ver reporte de similitud</b>                                                                                                                                                                                                                                                                                                                                        |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                         |
| <b>Descripción:</b>    | El docente abre la vista detallada de una comparación, donde el sistema renderiza la interpretación matemática de los modelos de IA.                                                                                                                                                                                                                                        |
| <b>Precondiciones:</b> | El docente seleccionó una entrega desde el listado de comparaciones (UC8).                                                                                                                                                                                                                                                                                                  |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente selecciona un archivo evaluado.</li> <li>2. El sistema recupera el reporte de doble canal completo desde PostgreSQL.</li> <li>3. El sistema renderiza la interfaz mostrando de forma gráfica el índice de similitud semántica y la probabilidad de generación mediante inteligencia artificial. del código.</li> </ol> |

Tabla 5.9: Especificación del Caso de Uso 9.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caso de Uso:</b>    | <b>UC10: Descargar reporte en PDF</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Actor:</b>          | Docente / Evaluador                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Descripción:</b>    | El docente exporta el reporte de doble canal de similitud a un documento estructurado, obteniendo una evidencia técnica estática e imprimible de la comparación.                                                                                                                                                                                                                                                                                                                          |
| <b>Precondiciones:</b> | El docente debe estar visualizando un reporte de similitud procesado (UC9).                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Flujo Básico:</b>   | <ol style="list-style-type: none"> <li>1. El docente ejecuta la acción “Descargar PDF” desde la interfaz del reporte.</li> <li>2. El backend compila el nombre del archivo, el gráfico de similitud, el índice estilométrico y los indicadores visuales detectados.</li> <li>3. El sistema genera un archivo en formato PDF aplicando una plantilla institucional.</li> <li>4. El sistema transmite el archivo para su descarga directa en el dispositivo local del evaluador.</li> </ol> |
| <b>Postcondición:</b>  | El flujo de auditoría académica concluye satisfactoriamente con la generación de la evidencia estática.                                                                                                                                                                                                                                                                                                                                                                                   |

Tabla 5.10: Especificación del Caso de Uso 10.

## 5.5 Modelado de Procesos de Negocio (BPMN)

Para la orquestación de las capacidades de inteligencia artificial y la gestión de la persistencia híbrida, se diseñó un modelo de procesos utilizando la notación BPMN 2.0. Este modelo, ilustrado en la Figura 5.3, detalla la interacción entre el docente, el motor de *backend* y los módulos

de inferencia de IA.

El proceso se divide en tres áreas de responsabilidad (*lanes*) claramente definidas:

- **Usuario (Docente):** Responsable de la carga de archivos de referencia y entrega, y la toma de decisiones basada en los reportes generados.
- **Backend:** Actúa como el orquestador del sistema, gestionando la validación de archivos, la comunicación con las bases de datos (relacional y vectorial) y la sincronización de los flujos de datos.
- **Módulo de IA:** Ejecuta las tareas de cómputo intensivo, incluyendo la generación de variantes sintéticas mediante LLMs, la extracción de características semánticas con *GraphCodeBERT* y el análisis estilométrico mediante convoluciones 1D (*CharCNN*).

### 5.5.1 Descripción del Flujo de Trabajo

El ciclo operativo de *Graphito* se desglosa en tres fases críticas:

1. **Fase de Gestión de Referencias:** El sistema ofrece caminos de ejecución alternativos para establecer la base de comparación. Si la configuración del problema es nueva, el *backend* verifica su existencia en la base de datos relacional; en caso de no existir, el módulo de IA genera variantes sintéticas para robustecer el espacio vectorial. Si la referencia ya existe, se recupera directamente su representación densa (*embedding*) desde la base de datos vectorial (*ChromaDB*), optimizando los tiempos de respuesta.
2. **Fase de Análisis de Doble Canal:** Una vez cargado el código a comparar, el sistema realiza un pre-procesamiento y normalización en el *backend*. Posteriormente, el flujo se bifurca mediante una compuerta paralela para realizar un análisis dual: una extracción de lógica estructural (semántica) y un análisis de patrones de escritura (estilometría). Esta fase garantiza que el sistema capture tanto la intención funcional como la huella de autoría sintética o humana.
3. **Fase de Fusión y Resultados:** Los vectores resultantes son integrados en un proceso de fusión de características (*Feature Fusion*). El sistema utiliza una compuerta de sincronización para asegurar que tanto el vector consolidado del archivo analizado como el vector de la referencia estén disponibles de forma concurrente antes de calcular la *Similitud del Coseno*. Finalmente, el módulo de IA interpreta los resultados numéricos para estructurar un reporte técnico que se persiste en el historial y se presenta al docente.

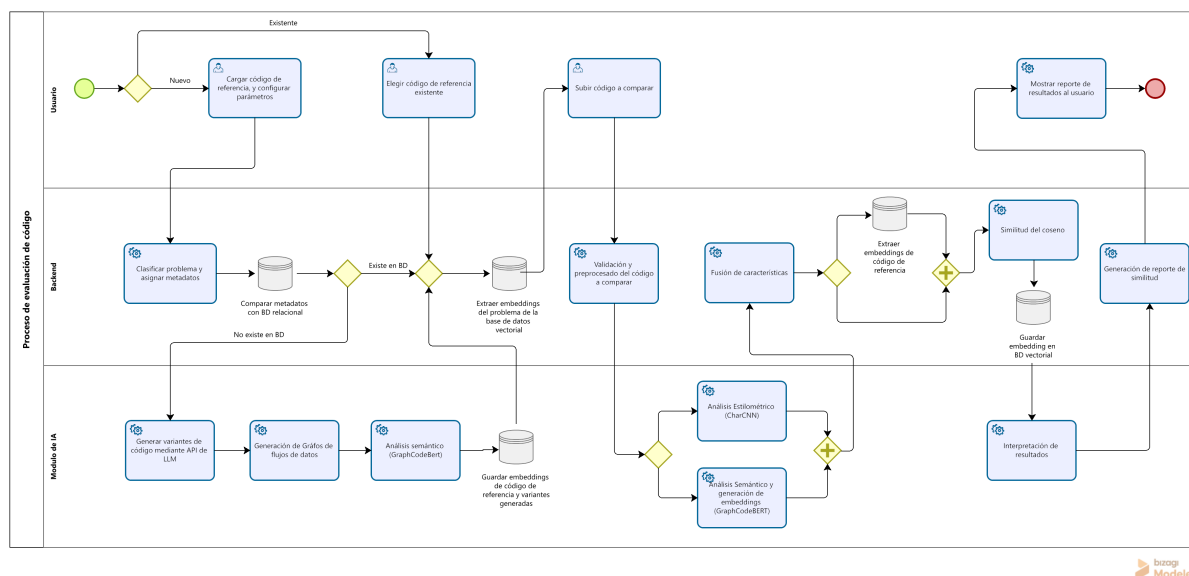


Figura 5.3: Diagrama BPMN de la comparación de código fuente.

## 5.6 Características semánticas del código

A diferencia del análisis estilométrico, el análisis semántico se centra en la lógica subyacente y la funcionalidad del programa, ignorando aspectos superficiales como el nombre de las variables o el formato del texto. Estas características permiten capturar la esencia algorítmica de una solución, lo que facilita la identificación de códigos que, aunque parezcan diferentes visualmente, ejecutan la misma lógica de resolución.

En la Tabla 5.11 se presenta una clasificación de las características semánticas identificadas para la construcción de la firma lógica en este proyecto.

| Categoría                         | Característica | Descripción Sintetizada                          |
|-----------------------------------|----------------|--------------------------------------------------|
| <b>Flujo de Datos</b>             | Def-Uso        | Rastreo del ciclo de vida y uso de variables.    |
|                                   | Dependencias   | Vínculos lógicos entre variables y operaciones.  |
| <b>Lógica de Control</b>          | Camino         | Rutas lógicas y bifurcaciones del algoritmo.     |
|                                   | Estructuras    | Patrones de repetición y decisiones funcionales. |
| <b>Representación Estructural</b> | AST / DFG      | Jerarquía gramatical y grafo de flujo de datos.  |
|                                   | Scope          | Contexto de validez de las entidades lógicas.    |
| <b>Contexto</b>                   | Semántica      | Significado de tokens según su entorno.          |
|                                   | Equivalencia   | Funcionalidad idéntica con distinta sintaxis.    |

Tabla 5.11: Clasificación sintetizada de características semánticas del código.

La extracción de estas características permite generar una representación robusta ante técnicas de ofuscación comunes, como el renombrado de variables o el cambio de estructuras de control

equivalentes (por ejemplo, sustituir un ciclo *for* por un *while*). En este sistema, el uso de *GraphCodeBERT* es fundamental, ya que este modelo no solo procesa el código como una secuencia de texto, sino que incorpora explícitamente el Grafo de Flujo de Datos (DFG). Esto permite que las representaciones vectoriales resultantes capturen de manera profunda las dependencias lógicas, facilitando un análisis semántico preciso incluso cuando la estructura superficial ha sido alterada intencionalmente.

## 5.7 Características estilométricas del código

Para el canal de análisis estilométrico, se identifican diferentes tipos de características que permiten capturar patrones asociados a la escritura del autor. Estas características no dependen de la funcionalidad del programa, sino de la huella digital sintáctica y de formato, lo que permite distinguir entre diferentes autores o identificar probables intervenciones de generación sintética (IA).

En la Tabla 5.12 se presenta una clasificación de las principales características estilométricas que la red neuronal procesa de forma implícita.

| Categoría     | Característica           | Descripción                                             |
|---------------|--------------------------|---------------------------------------------------------|
| Léxicas       | Frecuencia de caracteres | Distribución de caracteres y símbolos en el código.     |
|               | Uso de palabras clave    | Frecuencia de keywords del lenguaje (C11/C++17).        |
|               | Longitud de tokens       | Tamaño promedio de identificadores y expresiones.       |
| Sintácticas   | Estructuras de control   | Preferencia de uso entre if, for, while, switch.        |
|               | Anidamiento              | Profundidad promedio de los bloques de código.          |
|               | Modularidad              | Tendencia en la organización en funciones o bloques.    |
| Formato       | Indentación              | Uso predominante de espacios o tabulaciones.            |
|               | Saltos de línea          | Patrones de densidad y distribución de líneas.          |
|               | Estilo de llaves         | Ubicación de apertura y cierre de bloques { }.          |
| Estructurales | Tamaño del código        | Número total de líneas del archivo original.            |
|               | Uso de comentarios       | Densidad y ubicación de comentarios respecto al código. |

Tabla 5.12: Clasificación de características estilométricas del código fuente.

Estas características permiten construir una representación del estilo de codificación que puede ser utilizada por modelos de aprendizaje automático para identificar anomalías de autoría. En particular, los modelos basados en redes neuronales convolucionales a nivel de caracteres (Char-CNN) son capaces de aprender automáticamente patrones locales y distribuciones espaciales en la secuencia de caracteres cruda (preservando el estilo y formato original), lo que resulta especialmente útil para capturar diferencias sutiles entre código escrito por un estudiante humano y código generado de forma automatizada.

## 5.8 Definición de métricas de desempeño

La validación del sistema propuesto requiere la definición de métricas cuantitativas que permitan medir el desempeño algorítmico en los distintos módulos que lo componen. En particular, se consideran los dos componentes de la arquitectura dual: el canal de similitud semántica y el canal de análisis estilométrico.

Cada uno de estos módulos requiere métricas específicas que permitan evaluar su precisión, confiabilidad y utilidad computacional, permitiendo así validar la efectividad de la herramienta como apoyo objetivo a la auditoría docente.

### 5.8.1 Métricas para el análisis de similitud semántica

El canal de análisis semántico tiene como objetivo cuantificar el grado de equivalencia lógica entre dos programas. Para ello, se utilizan representaciones vectoriales (embeddings) generadas mediante el modelo *GraphCodeBERT*.

La principal métrica utilizada es la similitud del coseno, la cual mide el ángulo entre dos vectores en un espacio de alta dimensión. Su valor se encuentra en el rango de  $[-1, 1]$ , donde valores cercanos a 1 indican una alta proximidad entre los grafos lógicos de los programas analizados.

Esta métrica resulta especialmente adecuada para comparar embeddings, ya que permite determinar la cercanía semántica entre soluciones de software independientemente de su estructura sintáctica superficial.

Adicionalmente, se considera la correlación con el juicio experto como un mecanismo de validación externa, con el fin de determinar si los índices matemáticos generados por el sistema son consistentes con la auditoría manual del evaluador.

### 5.8.2 Métricas para el análisis estilométrico

El canal estilométrico, impulsado por la red neuronal *CharCNN*, se plantea como un problema de clasificación binaria, en el cual se infiere si un fragmento de código presenta patrones de escritura correspondientes a un humano o a generación sintética mediante inteligencia artificial.

Para evaluar el desempeño de este modelo de Deep Learning, se utilizan métricas estándar de clasificación supervisada:

- **Accuracy (Exactitud):** mide la proporción de predicciones correctas respecto al total de muestras analizadas.
- **Precision (Precisión):** indica la proporción de predicciones positivas correctas respecto al total de predicciones positivas realizadas.

- **Recall (Exhaustividad):** mide la capacidad del modelo para identificar correctamente todos los casos positivos reales.
- **F1-score:** representa la media armónica entre precisión y exhaustividad, proporcionando una medida equilibrada del desempeño del modelo ante posibles desbalances de clases.

Estas métricas permiten calibrar el comportamiento del modelo en distintos escenarios, particularmente para minimizar la tasa de falsos positivos al detectar anomalías de autoría.

### 5.8.3 Resumen de métricas utilizadas

En la Tabla 5.13 se presenta un resumen de las métricas empleadas en cada uno de los módulos de la arquitectura.

| Módulo                 | Métrica              | Propósito                                         |
|------------------------|----------------------|---------------------------------------------------|
| Similitud semántica    | Similitud del coseno | Medir cercanía espacial entre embeddings.         |
|                        | Correlación humana   | Validar coherencia con la auditoría docente.      |
| Análisis estilométrico | Accuracy             | Evaluar desempeño general de la red Char-CNN.     |
|                        | Precision            | Medir exactitud en alertas de autoría.            |
|                        | Recall               | Evaluar detección de anomalías sintéticas.        |
|                        | F1-score             | Balancear rendimiento ante clases desbalanceadas. |

Tabla 5.13: Métricas de validación de modelos utilizadas en el sistema.

### 5.8.4 Criterios de interpretación de resultados

Para facilitar el análisis de los resultados arrojados por los modelos, se definen criterios de interpretación que permiten contextualizar los valores numéricos obtenidos.

En el caso del canal semántico, los valores de similitud del coseno se interpretan bajo los siguientes umbrales preliminares:

| Rango de similitud | Interpretación algorítmica                               |
|--------------------|----------------------------------------------------------|
| 0.80 – 1.00        | Alta equivalencia lógica.                                |
| 0.60 – 0.79        | Equivalencia moderada (posible inspiración estructural). |
| 0.40 – 0.59        | Baja equivalencia.                                       |
| 0.00 – 0.39        | Disimilitud algorítmica.                                 |

Tabla 5.14: Criterios de interpretación para similitud semántica espacial.

Estos rangos permiten identificar casos atípicos y resaltarlos visualmente para revisión detallada del docente, especialmente aquellos que cruzan el umbral de alta equivalencia lógica.

Para el análisis estilométrico, la interpretación técnica se basa en el comportamiento conjunto de las métricas de clasificación:

- Valores altos de precisión y exhaustividad indican una sólida capacidad de la CharCNN para diferenciar características de autoría humana versus sintética.
- Un F1-score elevado refleja un modelo confiable que no sesga sus clasificaciones hacia la clase mayoritaria.
- Diferencias significativas entre precisión y exhaustividad requerirán ajustes en el orquestador para evitar falsos positivos que afecten la transparencia de la auditoría.

### 5.8.5 Consideraciones para la experimentación

Para garantizar una validación rigurosa de los modelos, es necesario considerar los siguientes factores durante el diseño del corpus experimental:

- **Diversidad algorítmica:** incluir soluciones con diferentes lógicas de control para el mismo problema.
- **Presencia de variantes IA:** incorporar entregas generadas específicamente mediante la Trayectoria A1 (Modelos de Lenguaje Externos).
- **Balance de clases:** asegurar una distribución representativa entre código escrito por estudiantes y código sintético.
- **Validación cruzada:** aplicar técnicas de partición de datos para evitar el sobreajuste (overfitting) de la red estilométrica.

### 5.8.6 Justificación de las métricas seleccionadas

La combinación de estas métricas permite validar el sistema desde dos perspectivas técnicas complementarias. Por un lado, la similitud del coseno aborda el problema de la detección de clones estructurales, cuantificando la equivalencia lógica a través de los grafos. Por otro lado, las métricas de clasificación evalúan la capacidad de la red convolucional para identificar patrones anómalos asociados a la Inteligencia Artificial.

Este enfoque dual permite certificar el desempeño del backend, proporcionando evidencia algorítmica sobre su viabilidad como herramienta de auditoría de código en entornos académicos.

## 5.9 Análisis de riesgos

El desarrollo e implementación del sistema propuesto implica la consideración de diversos riesgos que pueden afectar su funcionamiento, desempeño y adopción en el contexto educativo. Estos riesgos pueden surgir tanto a nivel técnico como en el uso práctico del sistema por parte de los evaluadores.

El análisis de riesgos permite identificar posibles escenarios adversos, evaluar su impacto y definir estrategias de mitigación que contribuyan a la robustez, confiabilidad e interpretabilidad del sistema.

### 5.9.1 Clasificación de riesgos

Los riesgos identificados se agrupan en las siguientes categorías:

- **Riesgos técnicos:** relacionados con el funcionamiento de los modelos de inteligencia artificial y la infraestructura del sistema.
- **Riesgos de datos:** asociados a la calidad, disponibilidad y representatividad de los datos utilizados.
- **Riesgos de interpretación:** vinculados a la forma en que los resultados son comprendidos por el evaluador.
- **Riesgos académicos:** relacionados con el impacto del sistema en el proceso de evaluación y aprendizaje.

### 5.9.2 Criterios de evaluación de riesgos

Para evaluar los riesgos identificados, se utilizan dos criterios principales: la probabilidad de ocurrencia y el impacto que pueden generar en el sistema.

La **probabilidad** se refiere a la posibilidad de que un riesgo ocurra durante la operación del sistema, mientras que el **impacto** describe el nivel de afectación que tendría dicho riesgo en el desempeño, confiabilidad o uso del sistema.

Se consideran tres niveles para cada criterio:

- **Alta:** el riesgo es frecuente o puede afectar significativamente el funcionamiento del sistema.
- **Media:** el riesgo puede ocurrir bajo ciertas condiciones y su impacto es moderado.
- **Baja:** el riesgo es poco probable o su impacto es limitado.

Estos niveles permiten priorizar los riesgos y enfocar los esfuerzos de mitigación en aquellos que representan una mayor amenaza para el sistema.

### 5.9.3 Identificación y evaluación de riesgos

En la Tabla 5.15 se presentan los principales riesgos identificados, junto con su probabilidad de ocurrencia, impacto y estrategia de mitigación.

| Riesgo                                     | Prob. | Impacto | Mitigación                                                      |
|--------------------------------------------|-------|---------|-----------------------------------------------------------------|
| Baja precisión del modelo semántico        | Media | Alta    | Ajuste de hiperparámetros y validación con datos reales         |
| Clasificación incorrecta en estilometría   | Media | Alta    | Uso de múltiples métricas y validación cruzada                  |
| Datos insuficientes o no representativos   | Alta  | Alta    | Ampliación y diversificación del dataset                        |
| Dependencia de modelos externos (LLM)      | Alta  | Alta    | Uso de rutas de respaldo con modelos alternativos y locales     |
| Interpretación incorrecta por el docente   | Media | Alta    | Visualización clara y explicaciones dentro del sistema          |
| Falsos positivos en detección de similitud | Media | Alta    | Uso combinado de análisis semántico y estilométrico             |
| Alto costo computacional                   | Media | Media   | Optimización del procesamiento y uso de almacenamiento en caché |
| Sesgo en el modelo de clasificación        | Media | Alta    | Balanceo de clases y evaluación continua del modelo             |

Tabla 5.15: Análisis de riesgos del sistema

### 5.9.4 Estrategias de mitigación

Para reducir el impacto de los riesgos identificados, se plantean diversas estrategias orientadas a mejorar la robustez del sistema:

- Validación continua de los modelos mediante conjuntos de datos diversos y representativos.

- Integración de múltiples enfoques de análisis (semántico y estilométrico) para reducir errores individuales.
- Implementación de mecanismos de interpretación de resultados que faciliten la comprensión por parte del evaluador.
- Optimización del rendimiento del sistema para garantizar tiempos de respuesta adecuados.

### **5.9.5 Discusión del impacto de los riesgos**

El análisis realizado permite identificar que los riesgos con mayor impacto están asociados a la precisión de los modelos de inteligencia artificial y a la interpretación de los resultados en el contexto educativo. Estos factores son críticos, ya que pueden influir directamente en la toma de decisiones del evaluador.

Asimismo, los riesgos relacionados con los datos, como la falta de representatividad o el sesgo, pueden afectar la capacidad del sistema para generalizar correctamente en escenarios reales.

No obstante, mediante la implementación de estrategias de mitigación adecuadas, es posible reducir significativamente estos riesgos, permitiendo que el sistema funcione como una herramienta confiable de apoyo a la evaluación docente.

El análisis de riesgos realizado permite identificar que los factores más críticos del sistema se concentran en la precisión de los modelos de inteligencia artificial y en la correcta interpretación de los resultados por parte del evaluador. Estos elementos son fundamentales, ya que influyen directamente en la confiabilidad del sistema dentro del proceso de evaluación académica.

Asimismo, se observa que los riesgos asociados a la calidad y representatividad de los datos pueden impactar de manera significativa en el desempeño del sistema, particularmente en la capacidad de generalización de los modelos en escenarios reales.

No obstante, la integración de estrategias de mitigación, como el uso combinado de análisis semántico y estilométrico, la validación continua y la incorporación de mecanismos de interpretación, permite reducir estos riesgos y fortalecer la robustez del sistema.

En este sentido, el análisis realizado respalda la viabilidad del sistema propuesto como una herramienta de apoyo a la decisión docente, siempre que se considere como un complemento al criterio humano y no como un sustituto del mismo.

### **5.9.6 Selección y justificación del conjunto de datos**

Para el entrenamiento del clasificador estilométrico propuesto se requieren dos tipos de datos: código fuente escrito por humanos en un contexto educativo introductorio, y código equivalente generado por modelos de inteligencia artificial. A continuación se describe y justifica cada fuente utilizada, así como la estructura cuantitativa del dataset resultante.

### 5.9.6.1 Código humano: IEEE Programming Homework Dataset

El componente de código humano proviene del *Programming Homework Dataset* publicado por Ljubovic y Pajic (2020) en [54]. Este dataset contiene envíos reales de estudiantes en dos cursos introductorios de programación universitaria: el curso A en lenguaje C y el curso B en C++, con entre 16 y 22 asignaciones por curso. Su selección se justifica por tres razones principales.

En primer lugar, el origen de los datos es directamente relevante al dominio del problema. Los envíos provienen de estudiantes en cursos de fundamentos de programación, que es exactamente la población objetivo del sistema propuesto. Esto garantiza que el modelo aprenda patrones estilométricos propios del código estudiantil introductorio y no de programadores expertos o código de producción.

En segundo lugar, el dataset cubre los tipos de problemas de interés: estructuras de control, manejo de arreglos, funciones, estructuras de datos básicas y manejo de archivos, todos representativos del currículo de un curso introductorio de C/C++ (acorde a los estándares C11 y C++17).

En tercer lugar, respecto al idioma de los identificadores y comentarios, el análisis del código fuente contenido en el dataset confirmó que este se encuentra escrito en serbocroata con alfabeto latino. Dado que los comentarios constituyen un rasgo estilométrico relevante —puesto que codifican información sobre la frecuencia, extensión y estilo de documentación del programador—, estos se conservan íntegramente durante el preprocesamiento como parte del *input* de la Char-CNN. La compatibilidad general con código escrito en otros idiomas de alfabeto latino, como el español o el inglés, se garantiza a nivel de distribución de caracteres y topología estructural.

### 5.9.6.2 Código generado por IA (Clase Sintética)

Para construir la clase positiva del clasificador (código generado por IA), se opta por una estrategia de generación propia sobre los mismos enunciados de problemas presentes en el dataset humano. Esta decisión se fundamenta en la necesidad de que ambas clases respondan al mismo dominio del problema, evitando que el modelo aprenda diferencias asociadas a la lógica de la tarea y no a la naturaleza de su origen (humano vs. máquina).

Con el objetivo de maximizar la generalización de la red convolucional ante distintos modelos de lenguaje, se emplea un conjunto diverso de generadores organizado en dos niveles, como se resume en la Tabla 5.16. En el primer nivel se incluyen modelos de referencia comerciales (SOTA), mientras que el segundo nivel abarca modelos de código abierto.

Para garantizar la consistencia entre ambas clases, todos los *prompts* de generación se formulan indicando explícitamente al modelo que utilice nombres de variables y comentarios en serbocroata. Esto asegura que el idioma no constituya una señal discriminante (sesgo) durante el entrenamiento, forzando a la red a aprender diferencias estrictamente de naturaleza estilométrica.

Tabla 5.16: Composición y origen del conjunto de datos dual

| <b>Categoría</b>      | <b>Fuente / Modelo</b>         | <b>Lenguaje</b> | <b>Origen de Escritura</b> | <b>Propósito en el sistema</b>                                             |
|-----------------------|--------------------------------|-----------------|----------------------------|----------------------------------------------------------------------------|
| Código Humano         | IEEE Programming Homework [54] | C / C++         | Estudiante (Humano)        | Base de referencia de escritura manual y entrenamiento de CharCNN.         |
| Código IA (Comercial) | GPT-4o, Claude 3.5, Gemini 1.5 | C / C++         | Generativo (SOTA)          | Identificación de patrones de IAs de alta disponibilidad para estudiantes. |
| Código IA (Local)     | CodeLlama, DeepSeek, Qwen2.5   | C / C++         | Generativo (Open-Source)   | Evaluación de robustez ante modelos locales y diversidad estilística.      |

*Elaboración propia con base en [54].*

ca espacial. Adicionalmente, se generan variantes de *prompt* para cada problema con el fin de reducir la dependencia a un estilo de generación único:

- Un *prompt* directo con el enunciado del problema.
- Un *prompt* con instrucción de simular el estilo de un estudiante principiante.
- Un *prompt* con enfoque en optimización sin instrucciones de estilo adicionales.

### 5.9.6.3 Consideraciones de balance y partición

El corpus final se construye con clases estrictamente balanceadas (50 % humano, 50 % IA) para evitar sesgos de clase mayoritaria en las predicciones de la CharCNN. La partición de los datos se realiza a nivel de *problema*; es decir, todas las soluciones (humanas o sintéticas) de un problema específico aparecen exclusivamente en una de las fases (entrenamiento, validación o prueba). Esta restricción arquitectónica previene la fuga de información (*data leakage*), obligando al modelo a clasificar basándose en el estilo y no en la lógica del algoritmo. La Tabla 5.17 presenta la distribución cuantitativa establecida.

Tabla 5.17: Distribución cuantitativa y partición del conjunto de datos

| Subconjunto                    | Porcentaje   | Cantidad ( $n$ ) | Finalidad en el modelo                                                                 |
|--------------------------------|--------------|------------------|----------------------------------------------------------------------------------------|
| Entrenamiento ( <i>Train</i> ) | 70 %         | 14,000           | Ajuste de pesos y aprendizaje de características estilométricas 1D.                    |
| Validación ( <i>Val</i> )      | 15 %         | 3,000            | Ajuste de hiperparámetros y prevención de <i>overfitting</i> durante el entrenamiento. |
| Prueba ( <i>Test</i> )         | 15 %         | 3,000            | Evaluación cuantitativa del rendimiento y matriz de confusión en datos no vistos.      |
| <b>Total (<math>N</math>)</b>  | <b>100 %</b> | <b>20,000</b>    | <b>Dataset consolidado e híbrido.</b>                                                  |

*Elaboración propia.*

En conclusión, la estructuración de este corpus híbrido permite calibrar con precisión el canal estilométrico del sistema **Graphito**. El diseño metodológico empleado fuerza a la arquitectura CharCNN a desarrollar una alta sensibilidad a las micro-estructuras de estilo que diferencian el código orgánico del sintético. El uso de  $N = 20,000$  muestras asegura una base estadística robusta para generalizar características más allá de las palabras clave, capturando la huella digital del código fuente en entornos académicos iniciales.

# Capítulo 6

## Diseño del sistema

### 6.1 Arquitectura (Modelo C4)

El diseño arquitectónico de *Graphito* se estructura utilizando el Modelo C4, el cual permite describir el software a través de múltiples niveles de abstracción. Esto proporciona diferentes perspectivas técnicas, desde la visión contextual de las interacciones externas hasta el nivel más profundo de componentes.

#### 6.1.1 Nivel 1: Contexto

El diagrama de contexto ilustra cómo el sistema **GRAPHITO** se relaciona con sus actores externos. En el centro de la arquitectura, el sistema provee las capacidades de procesamiento y análisis. A su alrededor se ubican: el **Docente**, quien interactúa para gestionar ejercicios, cargar el código de los archivos a analizar y visualizar los reportes; y la **API de LLM Externo**, que es consultada por la plataforma para generar automáticamente soluciones sintéticas que formarán parte del marco de referencias.

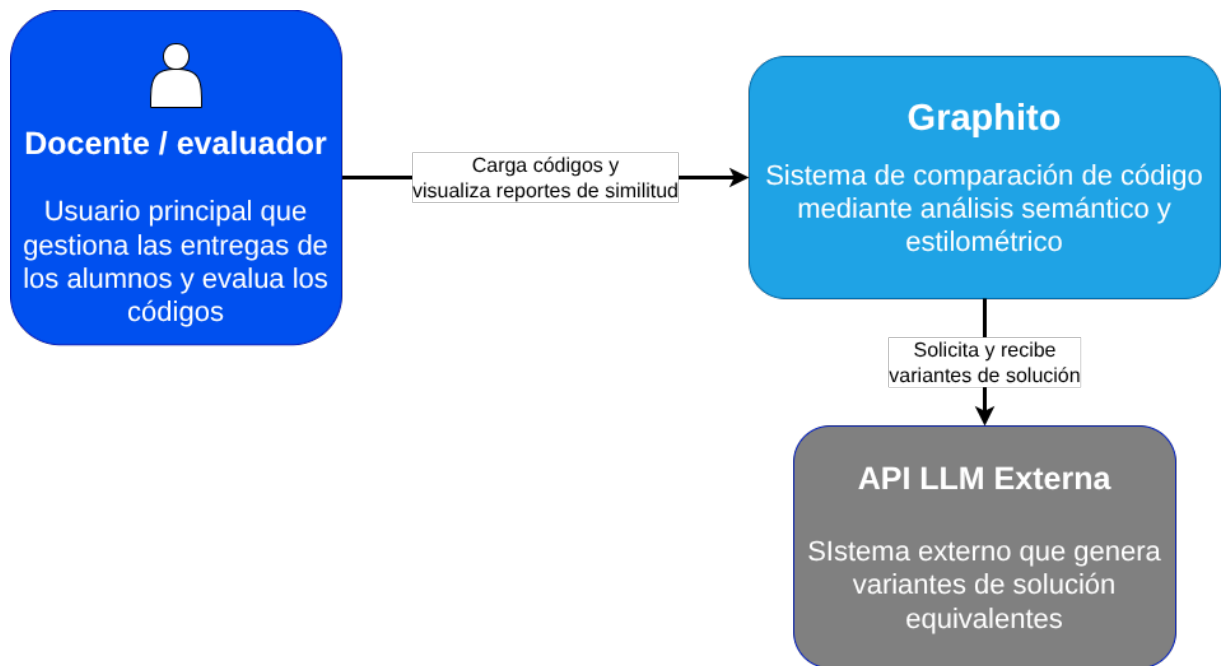


Figura 6.1: Diagrama de Contexto (Nivel 1) del sistema Graphito.

### 6.1.2 Nivel 2: Contenedores

En este nivel de abstracción se detalla la arquitectura de micro-servicios y bases de datos que componen el sistema. *Graphito* se separa funcionalmente en los siguientes contenedores principales:

- **Capa de Cliente (Front-end):** Interfaz web interactiva en *React* que consume servicios REST, facilitando el trabajo del docente.
- **Núcleo de Aplicación (Back-end):** Servicio principal implementado en *FastAPI* (Python) encargado de orquestar el sistema, gestionar las peticiones a la IA y procesar la lógica general.
- **DB Relacional (PostgreSQL):** Persistencia estructurada para metadatos del usuario, problemas y registro histórico integral del análisis.
- **Vector DB (ChromaDB):** Persistencia hiperdimensional de los *embeddings* que agilizan las búsquedas por proximidad semántica espacial en las referencias.

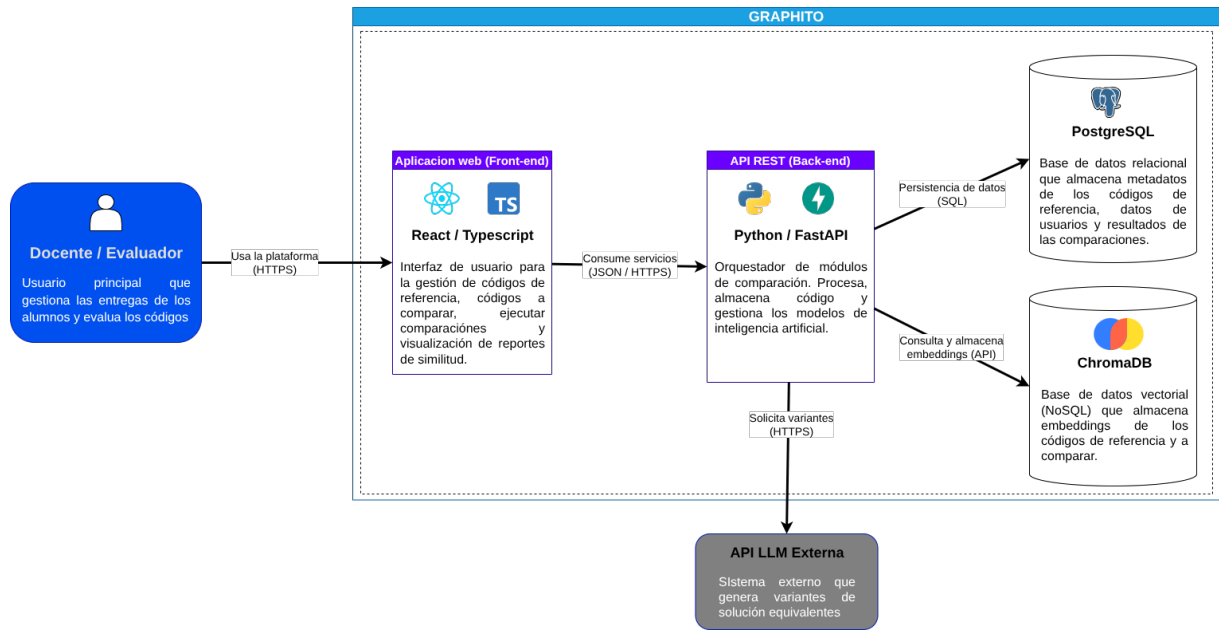


Figura 6.2: Diagrama de Contenedores (Nivel 2) del sistema Graphito.

### 6.1.3 Nivel 3: Componentes

Haciendo un acercamiento al motor central de analítica del sistema (el contenedor Back-end de FastAPI), se ilustran los componentes internos que materializan la funcionalidad de doble canal para Graphito:

- **Orquestador Inteligente (LangGraph):** Componente maestro de orquestación que evalúa iteraciones, fusiona características híbridas e interactúa cíclicamente para extender las referencias funcionales.
- **Módulo de Análisis Semántico:** Utiliza de forma profunda el modelo pre-entrenado *GraphCodeBERT* para interpretar características lógicas estructurales a través del grafo de flujo de datos (DFG).
- **Módulo de Estilometría:** Implementa un clasificador *CharCNN* mediante convoluciones 1D a nivel de caracteres, capaz de clasificar el origen (humano o sintético) al aprender la textura de redacción original y capturar singularidades de formato (ej. sangrías o estilo de llaves).

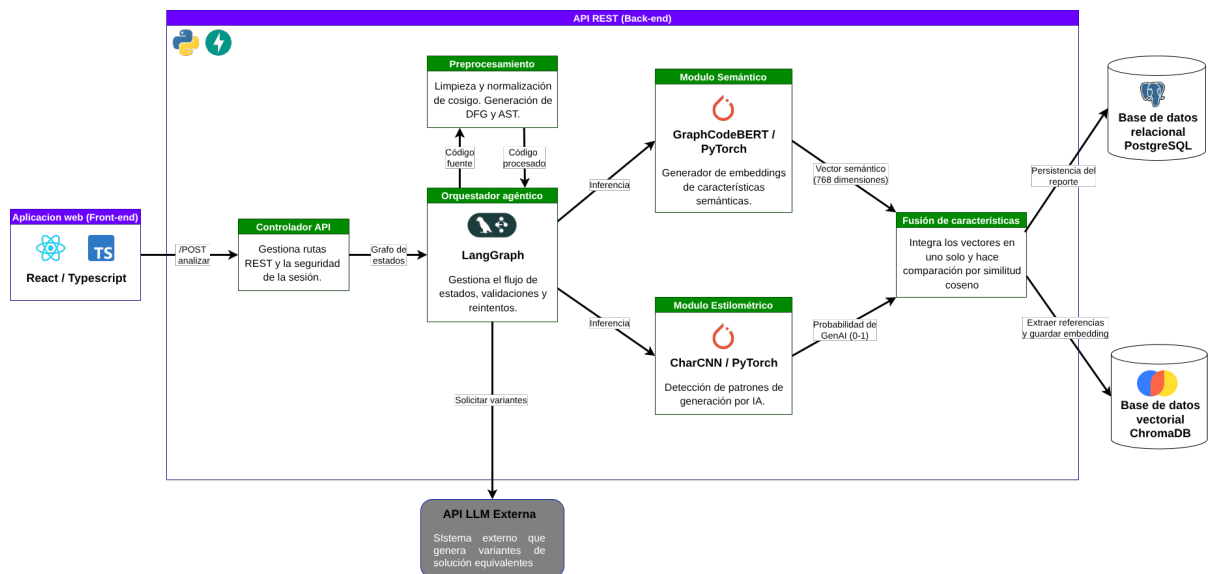


Figura 6.3: Diagrama de Componentes (Nivel 3) enfocado en el motor back-end en Python.

## 6.1.4 Nivel 4: Código (Clases y Secuencia)

El nivel de código ofrece el máximo detalle referencial. Se subdivide en la vista estática, representada por las clases y patrones, y la vista dinámica, que muestra el paso a paso transaccional.

### 6.1.4.1 Diagrama de Clases

A nivel estático, el corazón del diseño obedece al patrón de diseño *Strategy*.

Como es posible constatar en la Figura 6.4, la ejecución transaccional principal se centraliza en **OrquestadorGraphito**, que coordina los módulos. El patrón *Strategy* se aplica a la interfaz de inyección **ModeloIA**, separando los modelos predictivos como **ModeloGraphCodeBERT** y **ModeloCharCNN** del núcleo aplicativo.

El componente **Preprocesador** bifurca el flujo para proveer grafos limpios (análisis semántico) o matrices crudas (estilometría). Finalizada la ejecución iterativa, se concreta la **FirmaDual**. Por su parte, la persistencia se expone con la interfaz **AlmacenamientoDatos** la cual rige las lógicas independientes de **GestorPostgreSQL** y **GestorChromaDB**.

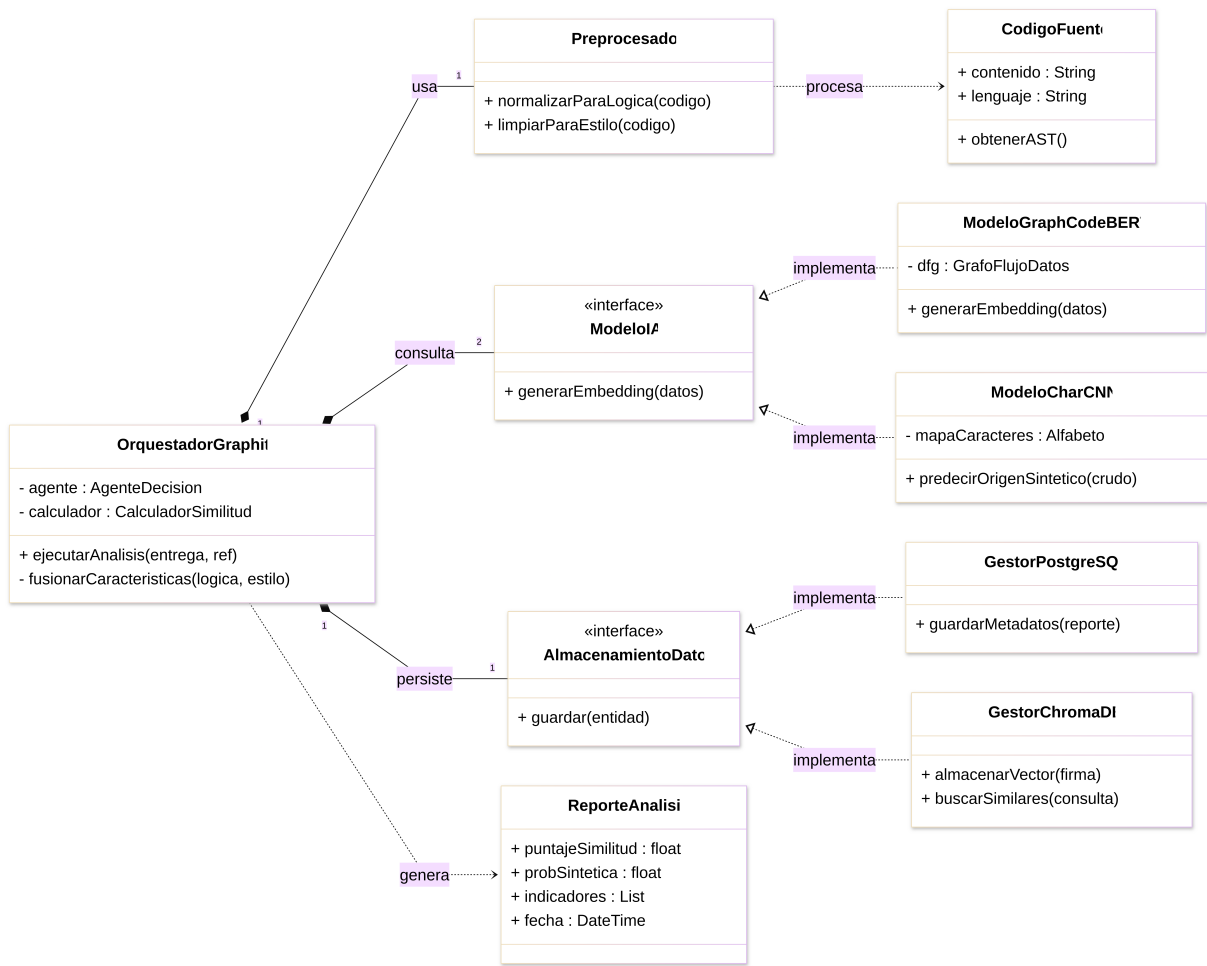


Figura 6.4: Diagrama de clases del sistema Graphito, ilustrando la arquitectura orientada a objetos correspondientes al Nivel 4 de abstracción.

#### 6.1.4.2 Diagrama de Secuencia

Para el flujo de eventos, el diagrama de secuencia dicta el actuar de los módulos durante un análisis estándar:

- **Ejecución Paralela:** Tanto *GraphCodeBERT* (para interpretar la lógica estructural a través de grafos) como *CharCNN* (para captar la huella estilométrica) se ejecutan de manera concurrente con la finalidad de optimizar los tiempos de inferencia en el servidor.
- **Consolidación e Indicadores:** Una vez concatenado el vector unificado mediante la Fusión de Características (*Feature Fusion*), se calcula su similitud del coseno contra las referencias indexadas en ChromaDB y se retornan los indicadores visuales a la capa de abstracción del *frontend* para la revisión del docente.

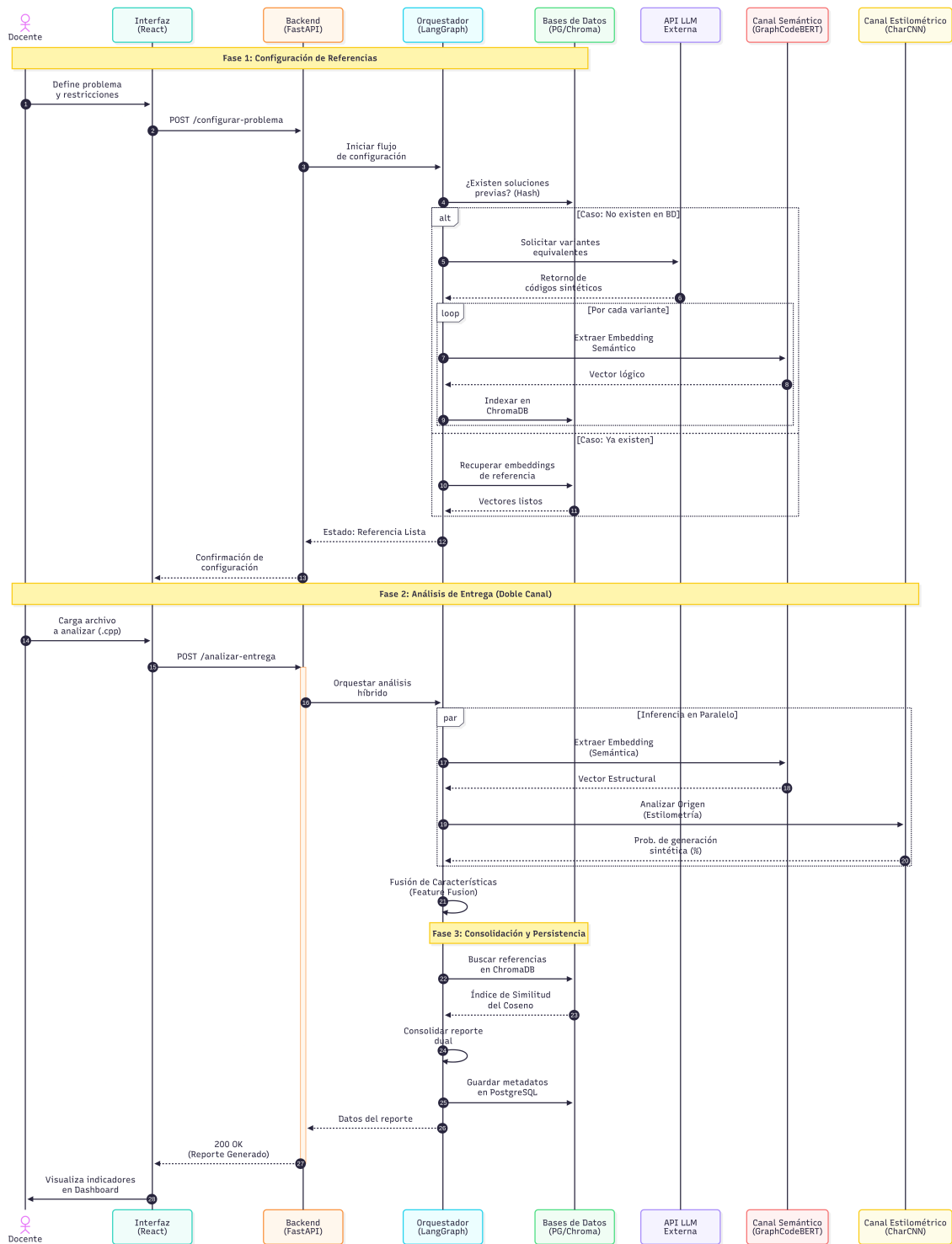


Figura 6.5: Diagrama de secuencia transaccional de *Graphito* desde el cliente (Nivel 4).

### 6.1.5 Modelado Dinámico: Diagrama de Estados

El modelado dinámico permite representar el comportamiento interno del sistema ante diversos estímulos y eventos. Para el sistema **Graphito**, el diagrama de estados describe el ciclo de vida de una tarea de análisis, desde la interacción inicial del docente hasta la consolidación de los resultados.

Este diagrama es fundamental para comprender cómo el sistema transita entre las fases de procesamiento paralelo y la fusión de características finales. En la Figura 6.6 se ilustra el flujo de estados transaccionales.

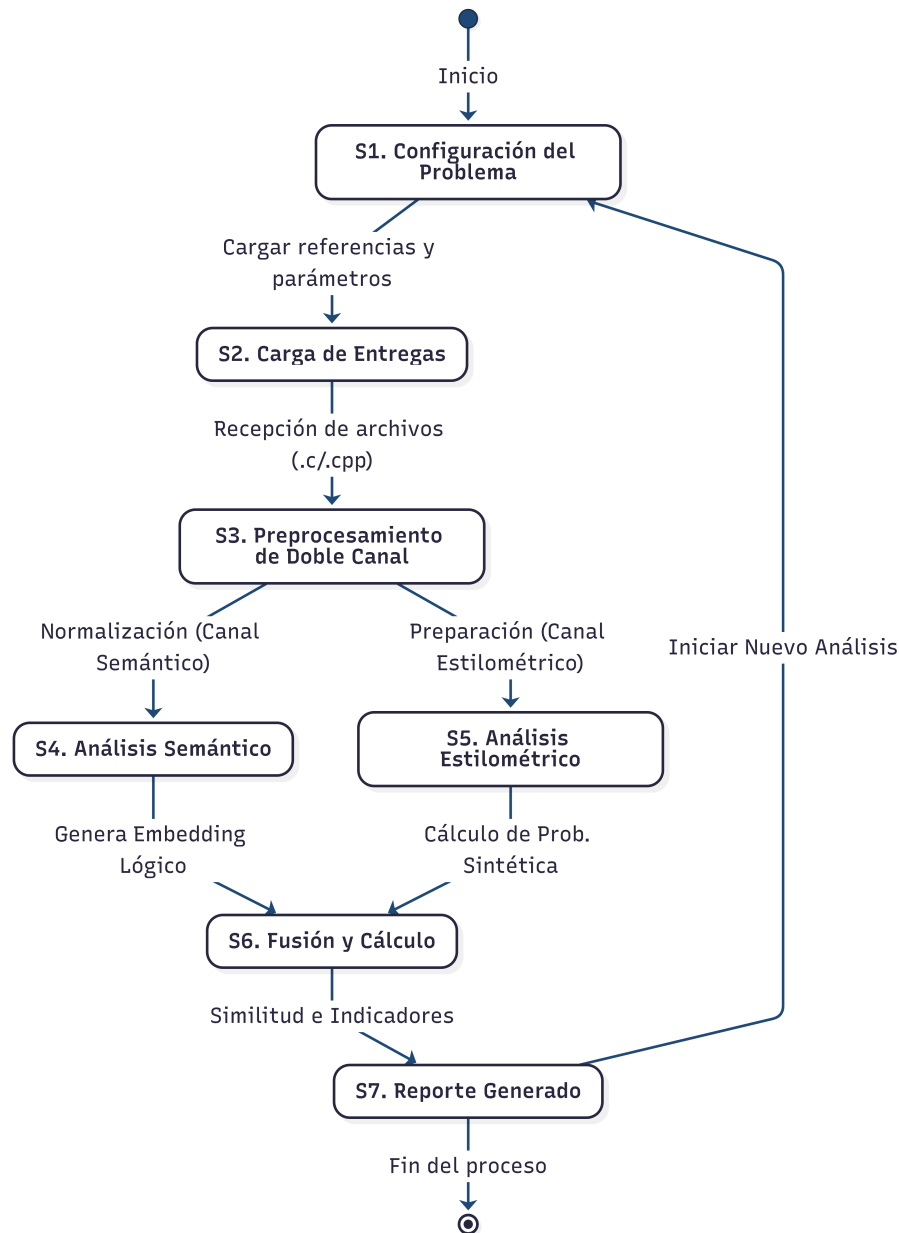


Figura 6.6: Diagrama de estados del proceso de análisis de código en Graphito.

A continuación, se describen las etapas clave representadas en el modelo:

- **S1. Configuración del Problema:** Estado inicial donde el sistema se encuentra a la espera de los parámetros base, como el lenguaje de programación, las restricciones del ejercicio y la carga de soluciones de referencia.
- **S2. Carga de Entregas:** Transición en la que el sistema recibe los archivos fuente en formato .c o .cpp. En este punto, se valida la integridad de los archivos antes de proceder al motor de analítica.

- **S3. Preprocesamiento de Doble Canal:** Estado crítico donde el código es bifurcado. Aquí se ejecutan simultáneamente las tareas de limpieza para el análisis de lógica y la preservación de formato para el análisis de redacción.
- **S4. Análisis Semántico:** El sistema transita a este estado para la extracción de características lógicas mediante grafos. El resultado es un vector denso (*embedding*) que encapsula la funcionalidad del código.
- **S5. Análisis Estilométrico:** En paralelo al estado anterior, el sistema procesa la textura del código para identificar rasgos de origen humano o sintético, generando una probabilidad porcentual de generación por IA.
- **S6. Fusión y Cálculo:** Una vez obtenidos ambos vectores, el sistema entra en una fase de consolidación. Se realiza el cálculo de la similitud del coseno contra las referencias y se unifican las métricas obtenidas.
- **S7. Reporte Generado:** Estado final donde los resultados se persisten en las bases de datos y se presentan al docente mediante indicadores visuales. Desde aquí, el sistema puede finalizar el proceso o retornar al estado S1 para una nueva evaluación.

## 6.2 Metodología de Desarrollo: Ciclo de Vida Komorebi

Para estructurar el ciclo de vida de los datos y los modelos de inteligencia artificial dentro del sistema **Graphito**, se adoptó la metodología **Komorebi** propuesta por Díaz Vergés [55]. En la ingeniería de software tradicional, los proyectos suelen gestionarse mediante ciclos secuenciales o iterativos convencionales (definir, diseñar, desarrollar, probar y desplegar) donde los requerimientos son deterministas y los datos sobre los que opera el sistema son estables y predecibles.

Sin embargo, como argumenta Díaz Vergés [55], los proyectos de Inteligencia Artificial y Aprendizaje Automático poseen una naturaleza cualitativamente distinta: combinan desarrollo de software con investigación empírica e innovación científica. Esto introduce un elevado grado de incertidumbre respecto a la convergencia de los modelos, la calidad intrínseca de las fuentes de información y la representatividad de las características extraídas. Bajo este paradigma, la metodología Komorebi plantea una gestión ágil adaptada específicamente a proyectos de *Data & Machine Learning*, estructurando el flujo técnico en etapas iterativas que garantizan la trazabilidad, reproducibilidad y gobernanza de los experimentos.

En el sistema **Graphito**, la metodología Komorebi actúa como el marco metodológico rector que gobierna el pipeline de procesamiento bimodal (semántico y estilométrico). En la Figura 6.7 se ilustra la arquitectura global de este flujo operativo.

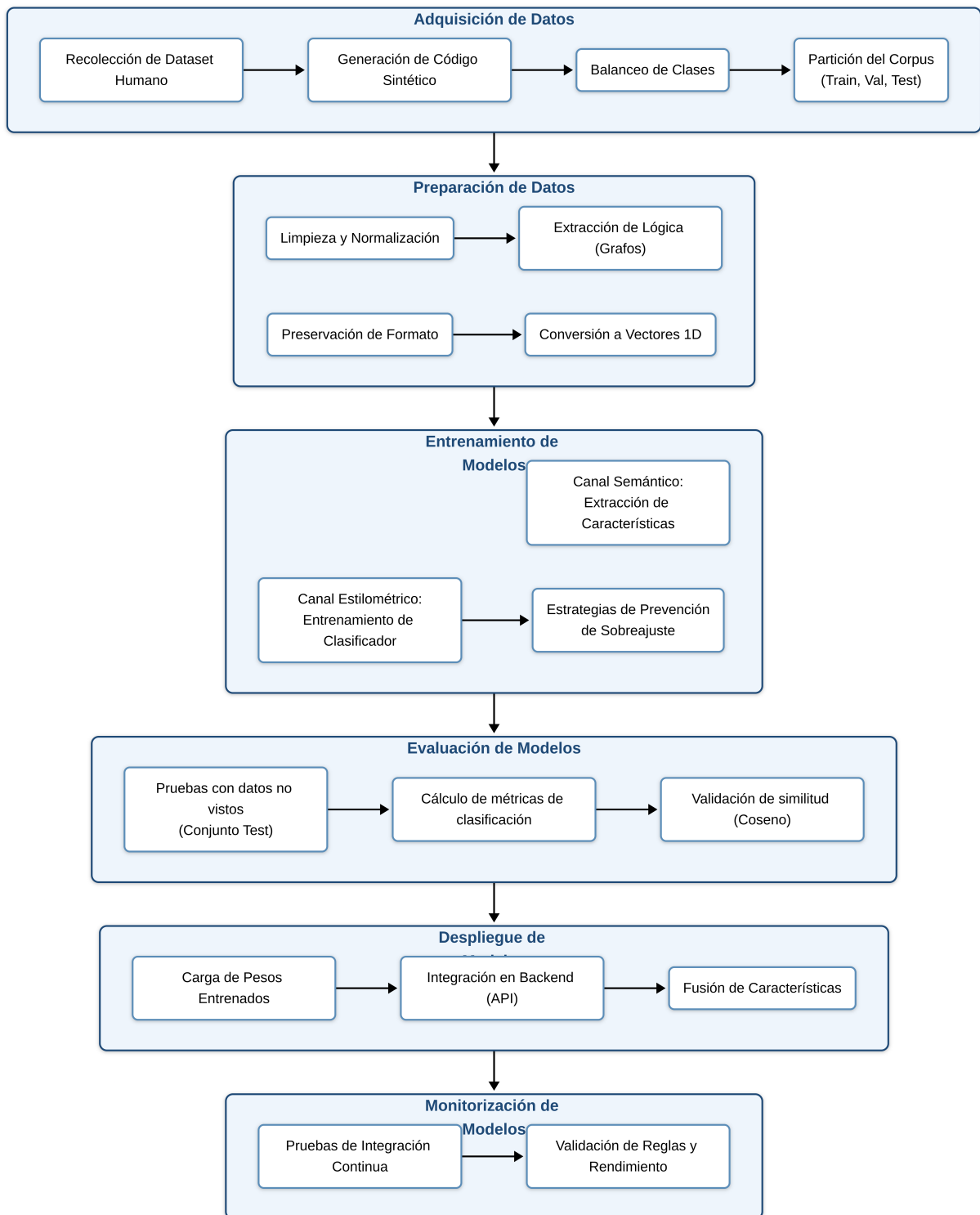


Figura 6.7: Flujo secuencial del pipeline de modelos basado en la metodología Komorebi.

A nivel documental y de ingeniería, las fases del ciclo de vida de Komorebi se articulan y desarrollan a lo largo de este Trabajo Terminal de la siguiente manera:

1. **Fase I. Planificación y Comprensión del Problema:** Consiste en el levantamiento de requerimientos técnicos, delimitación del contexto educativo (la paradoja cognitiva de la IA en cursos de programación) y el establecimiento de métricas de éxito (RNF-03 a RNF-07), aspectos detallados en los Capítulos 1, 4 y 5.

2. **Fase II. Adquisición de Datos (*Data Ingestion*):** Comprende la recolección del corpus humano representativo (*IEEE Programming Homework Dataset*), la ingeniería inversa asistida por LLMs para recuperar los enunciados algorítmicos (`extraer_enunciados.py`) y la síntesis del conjunto de código generado por IA con generadores comerciales y abiertos (`generar_referencias.py`). Esta etapa se formaliza detalladamente en el Capítulo 7 (Sección 7.1).
3. **Fase III. Preparación y Normalización de Datos:** Introduce la bifurcación arquitectónica divergente del código fuente para alimentar de forma especializada al canal semántico (depuración léxica, AST y Grafo de Flujo de Datos) y al canal estilométrico (código *raw* y cuantización matricial 1D de caracteres), aplicando una partición agrupada por problema para prevenir fuga de información (*data leakage*). Su desarrollo exhaustivo se expone en el Capítulo 7 (Sección 7.2).
4. **Fase IV. Entrenamiento y Modelado de IA:** Abarca la configuración de las redes neuronales: el uso de *GraphCodeBERT* como extractor de características semánticas mediante grafos de dependencias enriquecidos con adaptadores LoRA, el entrenamiento de la red convolucional *CharCNN* para clasificación de autoría mediante convoluciones 1D sobre caracteres, y el algoritmo de **Fusión de Características (*Feature Fusion*)** que consolida el vector multimodal de 769 dimensiones.
5. **Fase V. Evaluación y Validación Experimental:** Comprende la evaluación estadística rigurosa de los clasificadores (matriz de confusión,  $F_1$ -Score, curvas ROC-AUC), el análisis de invariancia de la similitud del coseno ante la taxonomía de clones de Bellon, y la contrastación empírica con arquetipos de entregas estudiantiles.
6. **Fase VI. Despliegue de Modelos:** Corresponde a la integración de los modelos empaquetados dentro de los servicios backend en FastAPI (bajo principios de *Clean Architecture*), la orquestación del almacenamiento híbrido (PostgreSQL y ChromaDB) y la interfaz interactiva para el docente en React.
7. **Fase VII. Monitorización y Calibración Continua:** Fase de estabilización operativa enfocada en la ejecución de pruebas unitarias y de estrés automatizadas, la calibración empírica de umbrales para mantener la tasa de falsos positivos por debajo del 10 %, y la integración del principio *Human-in-the-loop*, donde la herramienta asiste al docente sin sustituir su juicio pedagógico.

Esta articulación metodológica garantiza que las decisiones de diseño expuestas en las siguientes secciones (modelo relacional, almacenamiento vectorial y arquitectura de interfaz) se encuentren perfectamente integradas con las fases operativas de ingeniería de datos y modelado.

## 6.3 Diseño de Base de Datos

Se ha implementado un sistema de persistencia híbrido que combina el almacenamiento relacional para datos estructurados y el almacenamiento vectorial para el manejo de representaciones densas (embeddings).

### 6.3.1 Modelo Relacional (Diagrama Entidad-Relación)

El repositorio relacional gestiona la persistencia de metadatos y registros históricos mediante el motor **PostgreSQL**. La arquitectura del modelo permite que cada **DOCENTE** administre de forma independiente sus entidades de **PROBLEMA**. A su vez, cada problema se vincula con múltiples instancias de **CODIGO\_FUENTE**, permitiendo diferenciar entre las entregas de los alumnos y las soluciones de referencia generadas o recolectadas.

La lógica central de la auditoría se consolida en la tabla **REPORTE\_ANALISIS**. Esta entidad posee una relación dual con el código fuente para contrastar una entrega específica frente a una referencia seleccionada, almacenando los índices de similitud lógica y probabilidad sintética resultantes. Finalmente, el sistema registra hallazgos técnicos en la entidad **INDICADOR\_INTEGRIDAD** cuando se detectan anomalías en la estructura lógica o patrones estilométricos que requieren la atención del docente.

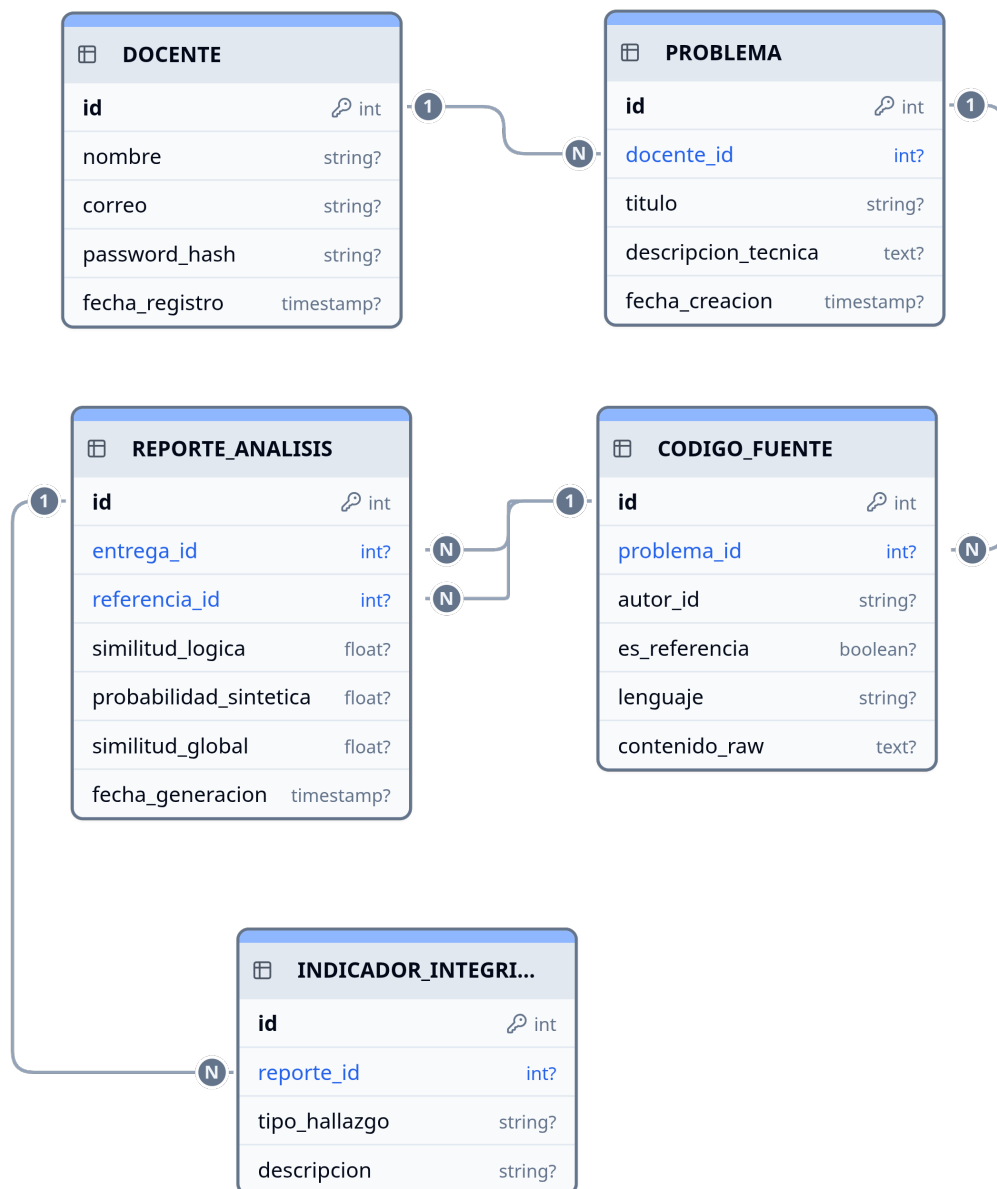


Figura 6.8: Modelo Entidad-Relación de la base de datos relacional (PostgreSQL).

### 6.3.2 Esquema de Colección Vectorial (ChromaDB)

Para la gestión de las representaciones densas y la ejecución de búsquedas por proximidad semántica, el sistema implementa una capa de persistencia heurística mediante **ChromaDB**. A diferencia del modelo relacional, esta base de datos está optimizada para el almacenamiento de tensores y el cálculo de distancias en espacios de alta dimensionalidad.

Como se muestra en la Figura 6.9, el diseño de la colección se mantiene bajo una estructura plana (*flat schema*) para minimizar la latencia durante la fase de recuperación de información.

| codigos_referencia                                                                   |        |
|--------------------------------------------------------------------------------------|--------|
| id  | uuid   |
| embedding_hibrido                                                                    | vector |
| metadata_json                                                                        | json   |

Figura 6.9: Esquema conceptual de la colección vectorial en ChromaDB.

La colección `codigos_referencia` se articula bajo tres pilares fundamentales que garantizan la integridad del análisis:

1. **Identificador Único (UUID):** Se utiliza el mismo identificador generado en la base de datos relacional (`CODIGO_FUENTE.id`) para actuar como puente lógico, permitiendo al orquestador reconstruir la información del alumno tras una coincidencia vectorial.
2. **Embedding Híbrido (769 Dimensiones):** Es el componente central del sistema. Este vector concatena dos señales distintas:
  - **Canal Semántico (0-767):** Dimensiones que capturan la lógica y el flujo de datos del código fuente.
  - **Canal Estilométrico (768):** Un valor escalar que representa la probabilidad de origen sintético, permitiendo que la búsqueda por similitud considere tanto la funcionalidad como la textura de la redacción.
3. **Metadatos JSON:** Almacenan atributos clave como el `problema_id` y el lenguaje. Esto permite que el sistema realice un pre-filtrado lógico, asegurando que la búsqueda de vecinos más cercanos se ejecute únicamente sobre el conjunto de soluciones de referencia que pertenecen al mismo ejercicio académico.

Esta arquitectura híbrida asegura que **Graphito** no solo sea preciso en la identificación de similitudes, sino también eficiente en términos computacionales al segmentar el espacio de búsqueda.

## 6.4 Diseño de Interfaz de Usuario (UI/UX)

El diseño de la interfaz de **Graphito** se fundamenta en principios de usabilidad para entornos técnicos, buscando que la interpretación de métricas de IA sea inmediata para el docente. A continuación, se documenta la maquetación completa del sistema.

### 6.4.1 Módulo de Acceso y Registro

Este módulo garantiza la privacidad de los datos y el aislamiento de los ejercicios por docente.

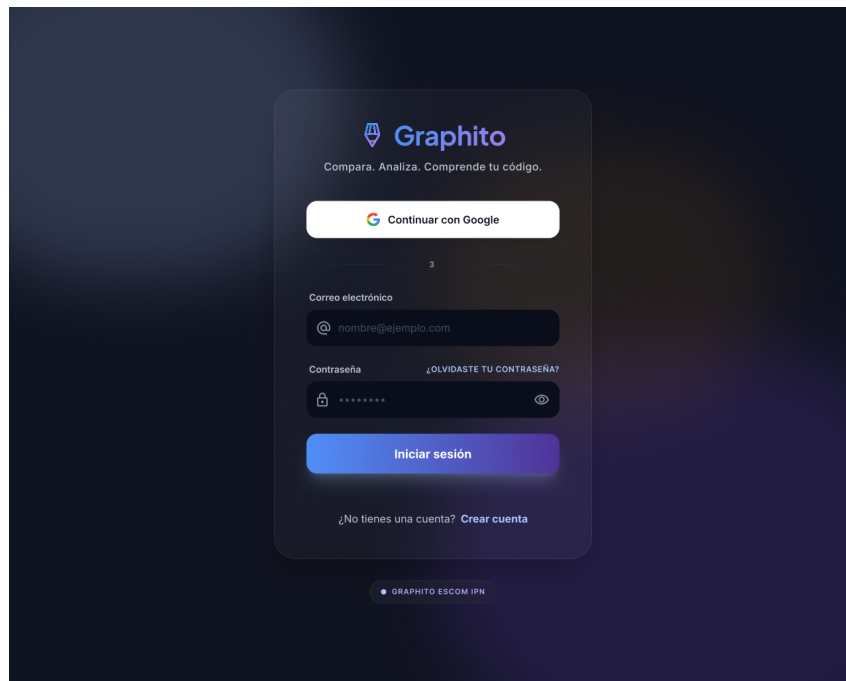


Figura 6.10: Interfaz de inicio de sesión (Cumplimiento de RF-01).

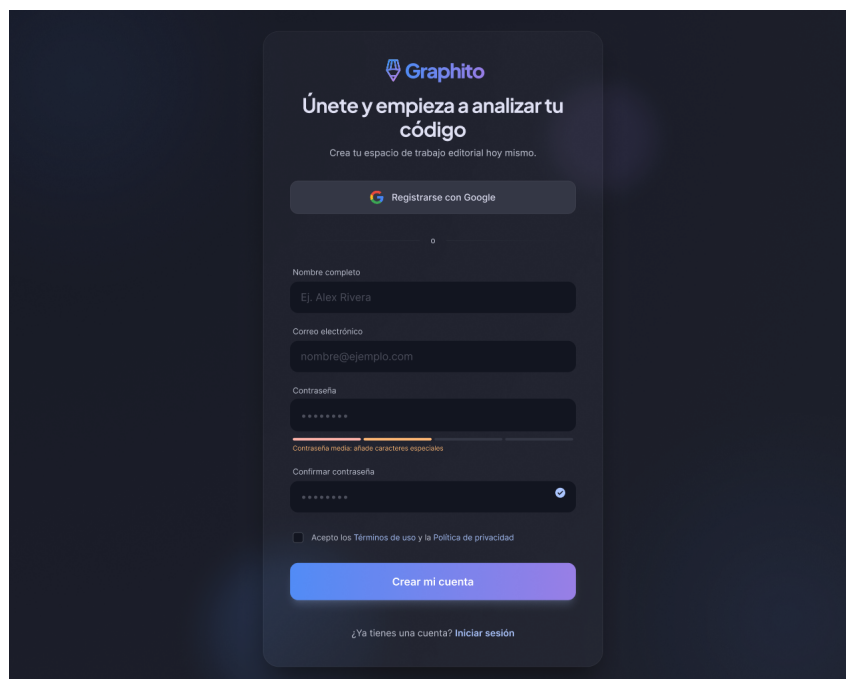


Figura 6.11: Interfaz de registro de nuevos usuarios.

## 6.4.2 Gestión de Repositorio y Biblioteca

El sistema permite organizar el material didáctico y las soluciones de referencia de forma jerárquica.

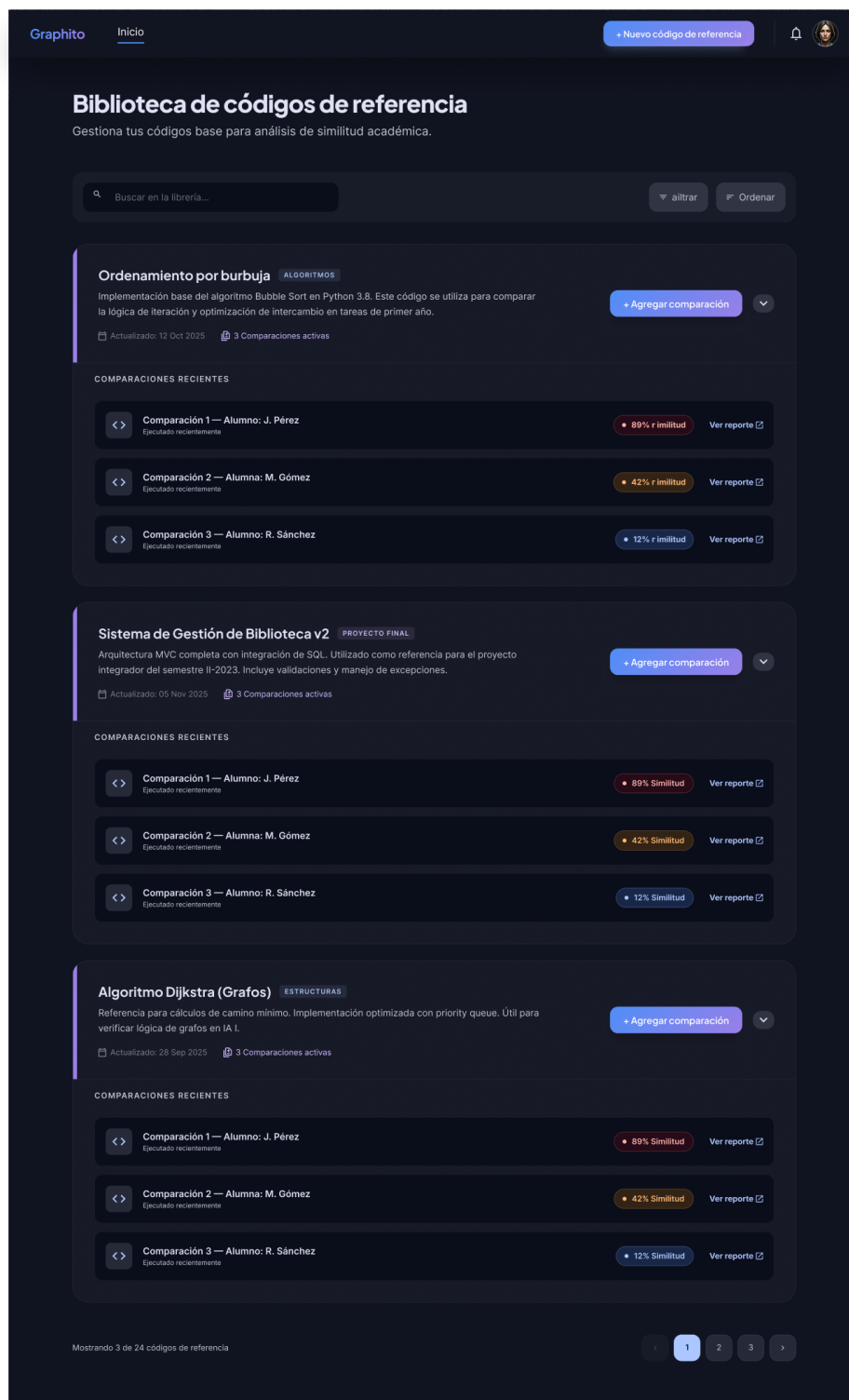


Figura 6.12: Repositorio de problemas y gestión de ejercicios (Cumplimiento de RF-02).

### 6.4.3 Módulo de Carga y Configuración de Análisis

Esta sección permite al usuario alimentar al sistema con las fuentes necesarias para la comparación semántica y estilométrica.

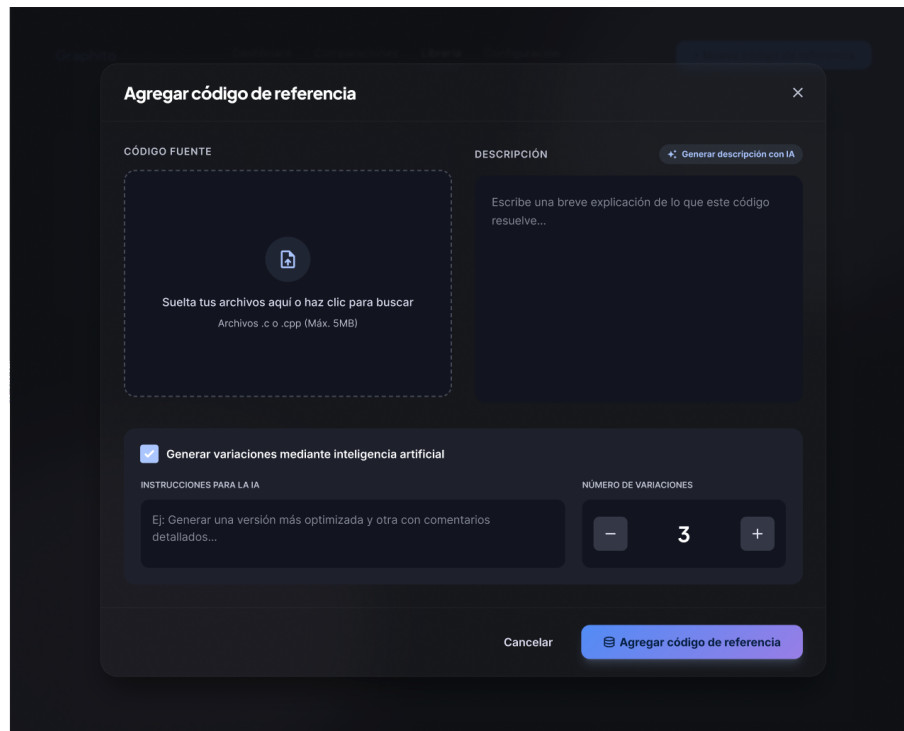


Figura 6.13: Interfaz de carga de código de referencia (Cumplimiento de RF-03).

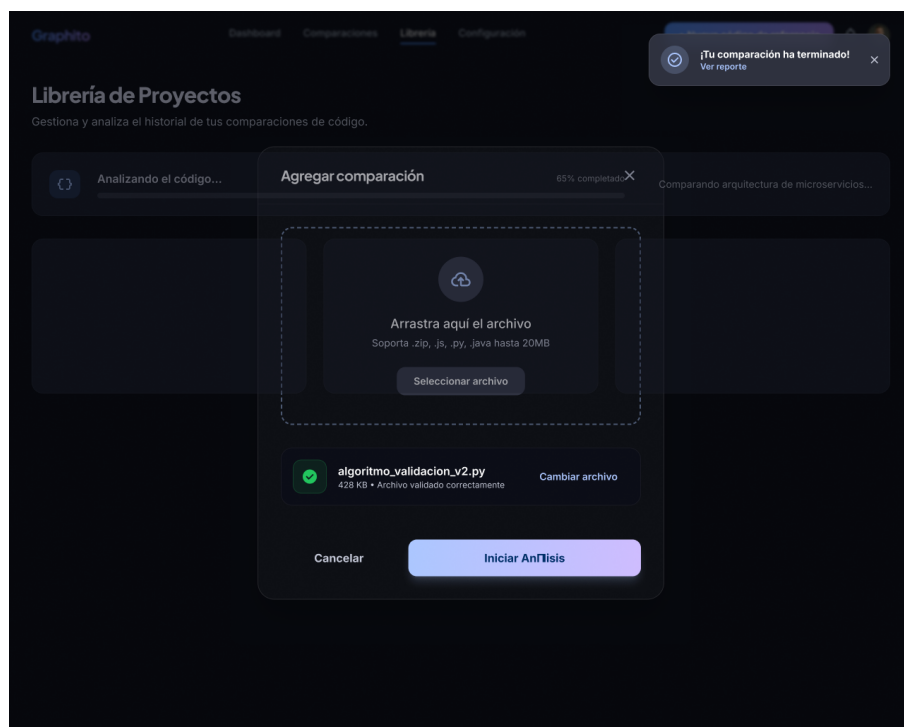


Figura 6.14: Configuración de parámetros para el inicio del análisis.

#### 6.4.4 Visualización de Reportes de Doble Canal

Es el componente integrador donde se despliegan los indicadores generados por los modelos de aprendizaje profundo.



Figura 6.15: Dashboard de resultados híbridos (Cumplimiento de RF-10 y RF-11).

Como se observa en la Figura 6.15, el sistema cumple con el **RF-10** al presentar de manera unificada la similitud lógica y la probabilidad sintética. Asimismo, integra el **RF-11** mediante la visualización de indicadores de integridad que facilitan la toma de decisiones pedagógicas.

# Capítulo 7

## Ingeniería de Datos

En el desarrollo de sistemas de aprendizaje automático aplicados al análisis de código fuente, la calidad y el balance de los datos son determinantes. Si el conjunto de datos contiene sesgos o información inconsistente, los modelos aprenderán patrones erróneos que afectarán su desempeño en casos reales.

Siguiendo la metodología **Komorebi** para la gestión de proyectos de Inteligencia Artificial [55], las fases iniciales de **Adquisición de Datos** y **Preparación de Datos** establecen las bases del sistema **Graphito**. En este capítulo se documenta cómo se recolectó el código escrito por estudiantes, cómo se reconstruyeron los enunciados de las prácticas mediante ingeniería inversa con modelos de lenguaje, el método utilizado para generar código equivalente con inteligencia artificial y el proceso de preparación y normalización para alimentar los modelos predictivos.

### 7.1 Adquisición de Datos

El objetivo de la adquisición de datos fue conformar un conjunto balanceado y representativo de programación en cursos universitarios iniciales. El problema exigía comparar dos grupos de código: programas escritos por estudiantes y programas generados por inteligencia artificial, asegurando que ambos resolvieran exactamente las mismas tareas algorítmicas y con el mismo contexto de lenguaje.

En la Figura 7.1 se muestra el flujo general de adquisición e ingeniería inversa diseñado para construir este conjunto de datos.

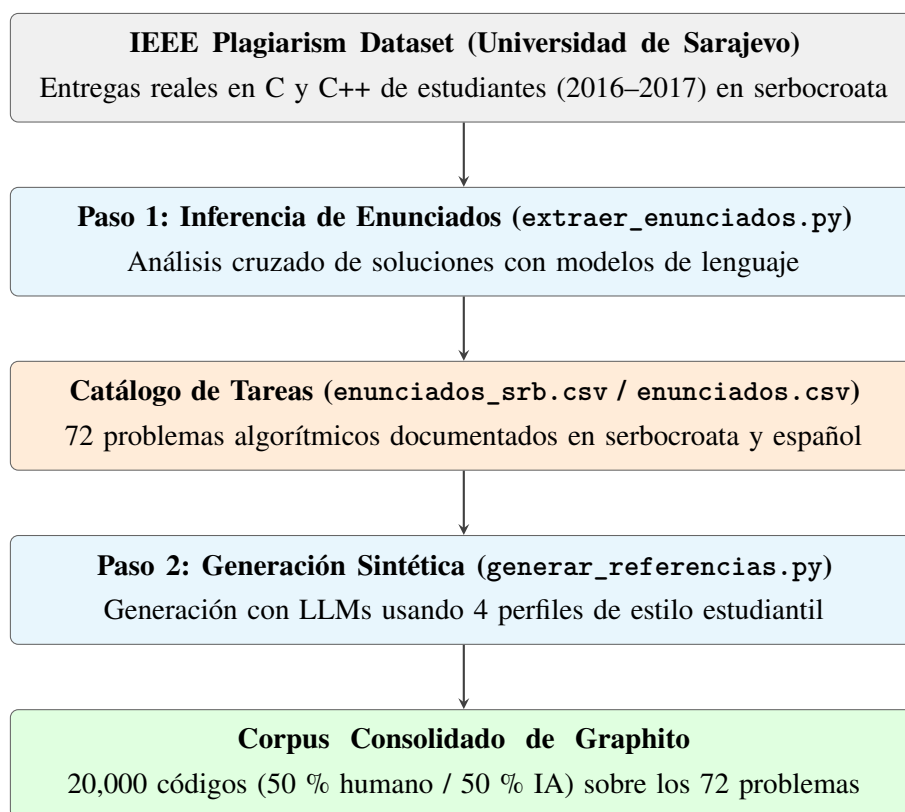


Figura 7.1: Flujo general de adquisición e ingeniería inversa de datos.

### 7.1.1 Requerimientos del Conjunto de Datos

Para que los datos fueran útiles en el contexto de las asignaciones de programación en la Escuela Superior de Cómputo (ESCOM-IPN), se definieron dos condiciones indispensables:

1. **Autoría humana verificada (Pre-LLM):** Actualmente, recolectar código reciente de repositorios abiertos como GitHub implica dos riesgos metodológicos severos documentados en la literatura. Por un lado, Lopes et al. [56] demostraron mediante el estudio DéjàVu que más del 70 % del código en GitHub corresponde a duplicados o variantes triviales de archivos existentes, lo que induce una alta contaminación del corpus (*corpus contamination*). Por otro lado, como advierten Zhou et al. [57], los modelos generativos contemporáneos han sido preentrenados sobre estos repositorios masivos, existiendo el peligro latente de contaminar los conjuntos evaluativos o de incluir código autocompletado por IA. Si un clasificador se entrena con código supuestamente humano que contiene trazas sintéticas, aprenderá fronteras de decisión difusas. Por ello, fue indispensable utilizar un conjunto de datos recolectado con anterioridad a la popularización masiva de los modelos de lenguaje a finales de 2022.
2. **Nivel académico introductorio:** Graphito apoya la revisión en materias como Fundamentos de Programación y Estructuras de Datos. Por lo tanto, no se requería código profesional con patrones de diseño avanzados o características complejas de C++ moderno, sino código típico de estudiantes: variables con nombres sencillos, estructuras de control básicas

(if, for, while), funciones modulares, arreglos y punteros elementales, incluyendo los errores de formato comunes en principiantes.

### 7.1.2 Selección del Dataset Base: IEEE Programming Homework Dataset

Cumpliendo con estas condiciones, se seleccionó el conjunto de datos *Programming Homework Dataset for Plagiarism Detection*, publicado por Vedran Ljubovic y Elma Pajic en IEEE Dataport [58].

Este dataset fue recolectado en la Facultad de Ingeniería Eléctrica de la **Universidad de Sarajevo** (Bosnia y Herzegovina) y ofrece ventajas clave para este trabajo:

- **Fecha de recolección (2016–2017):** Al ser anterior a la salida de los modelos de lenguaje modernos, garantiza que el código fue escrito enteramente por humanos.
- **Validación con defensas orales:** Incluye registros de casos de plagio no solo por similitud estática, sino también mediante entrevistas y exámenes orales individuales (registrados en el archivo `ground-truth-dynamic-anon.txt`), donde los profesores confirmaron si el alumno entendía realmente el código entregado.
- **Registros de actividad del IDE (*stats*):** Contiene archivos JSON con métricas de trabajo del estudiante: tiempo total invertido (`total_time`), número de compilaciones (`builds`) y pruebas realizadas (`testings`), lo que demuestra el proceso real de desarrollo.

El dataset abarca cuatro cohortes académicas, cuyas características se resumen en la Tabla 7.1.

Tabla 7.1: Estructura del conjunto de datos IEEE Plagiarism.

| Aspecto                 | Detalle                                                               |
|-------------------------|-----------------------------------------------------------------------|
| Cursos evaluados        | 4 cohortes: A2016 (C), A2017 (C), B2016 (C++), B2017 (C++)            |
| Lenguajes               | C (estándar C11) y C++ (estándar C++17)                               |
| Estudiantes             | Entre 100 y 150 por cada cohorte académica                            |
| Tareas y problemas      | 16 a 22 asignaciones por curso, con un total de 72 problemas únicos   |
| Registros de desarrollo | Archivos JSON con eventos de edición, compilación y pruebas           |
| Anonimización           | Nombres sustituidos por identificadores como <code>student{id}</code> |

### 7.1.3 Limitaciones del Dataset: Idioma Serbocroata y Falta de Enunciados

Durante la exploración de los archivos en `data/raw/IEEE_plagiarism/`, se identificaron dos retos importantes:

1. **Código escrito en serbocroata:** Al provenir de la Universidad de Sarajevo, los estudiantes escribieron sus variables, mensajes en pantalla y comentarios en serbocroata con caracteres latinos (por ejemplo: unos para entrada, broj para número, suma para adición y niz para arreglo). Si este aspecto no se consideraba al generar el código con IA, los modelos aprenderían a distinguir entre idiomas (inglés frente a serbocroata) en lugar de aprender el estilo de programación.
2. **Ausencia de los enunciados de las tareas:** Las carpetas contenían los códigos de los alumnos clasificados por códigos de tarea (como `src/A2016/Z1/Z1/student2956.c`), pero los autores no incluyeron las descripciones de los problemas. Sin el enunciado exacto, no era posible pedirle a una inteligencia artificial que resolviera el mismo ejercicio.

### 7.1.4 Ingeniería Inversa y Reconstrucción de Enunciados

Para recuperar los enunciados de forma sistemática, se desarrolló un script en Python denominado `extraer_enunciados.py`, el cual deduce la tarea original analizando el código de estudiantes con soluciones correctas:

1. **Muestreo por problema:** Para cada uno de los 72 problemas, el script toma de 2 a 3 códigos de estudiantes distintos que compilaban correctamente y pasaron las pruebas unitarias del curso.
2. **Análisis con LLM:** El script envía estos códigos a *GPT-4o-mini* mediante una consulta estructurada, pidiéndole comparar las soluciones, identificar los elementos comunes y redactar el enunciado del problema junto con las reglas de validación de entradas y salidas.
3. **Generación de dos versiones del catálogo:**
  - `enunciados.csv`: Enunciados traducidos al español para que los profesores y evaluadores de ESCOM puedan auditar y entender los ejercicios fácilmente.
  - `enunciados_srb.csv`: Enunciados redactados en serbocroata para usarlos como entrada en la generación de código con IA, asegurando la paridad con el código original de los alumnos.
4. **Mecanismos de control:** El script guarda puntos de control en el archivo `extraer_enunciados_checkpoint.json` para retomar el proceso si ocurre alguna falla de red o límite de peticiones de la API.

En la Tabla 7.2 se presentan ejemplos de los requerimientos obtenidos mediante este método.

Tabla 7.2: Ejemplos de requerimientos algorítmicos recuperados por ingeniería inversa.

| Curso | Asig. | Prob. | Leng. | Enunciado Algorítmico Deducido                                                                                                                                                                                                                                                                                                                    |
|-------|-------|-------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A2016 | Z1    | Z1    | C     | Calcular la suma de puntos de tres estudiantes (Tarik, Bojan y Mirza) en una materia con cinco rubros: primer parcial (máx. 20 pts), segundo parcial (máx. 20 pts), asistencia (máx. 10 pts), tareas (máx. 10 pts) y examen final (máx. 40 pts). Validar que los datos estén en los rangos correctos y mostrar error si algún valor no es válido. |
| A2016 | Z1    | Z2    | C     | Con los coeficientes de dos rectas en la forma $y = a_1x + b_1$ y $y = a_2x + b_2$ , indicar si son paralelas, coincidentes o secantes. Si se cruzan, calcular el punto $(x, y)$ donde se cortan.                                                                                                                                                 |
| A2016 | Z1    | Z3    | C     | Leer una serie de letras que representan colores de autos (c: negro, b: blanco, s: gris, v: rojo, p: azul), terminando con la letra 'k'. Calcular el total de autos, el porcentaje de cada color y determinar el color más frecuente.                                                                                                             |
| A2016 | Z1    | Z4    | C     | Pedir un número entero $n$ ( $0 < n \leq 50$ ) y mostrar en pantalla una figura con forma de letra 'W' hecha con asteriscos, con una altura de $n$ filas.                                                                                                                                                                                         |

### 7.1.5 Generación de Código con IA en Serbocroata y Perfiles de Estudiante

Para entrenar el modelo estilométrico (CharCNN) y evaluar el modelo semántico (GraphCodeBERT), se requería un conjunto de datos balanceado con 50 % de código humano y 50 % de código generado por IA sobre los mismos 72 problemas. Esta estrategia de curación dirigida se fundamenta en los hallazgos de Gunasekar et al. [59] en su trabajo *Textbooks Are All You Need*, donde demostraron empíricamente que un conjunto de datos sintético curado con alta densidad pedagógica y perfiles controlados permite a los modelos de lenguaje capturar y generalizar conceptos algorítmicos con mayor efectividad que colecciones masivas no filtradas.

La generación se realizó mediante el script `generar_referencias.py`, aplicando soluciones a los desafíos identificados:

#### 7.1.5.1 Control del Sesgo de Idioma

Si las respuestas de la IA se generaban en inglés, el clasificador aprendería a distinguir entre palabras en inglés y serbocroata en lugar de reconocer el estilo de escritura. Para evitarlo, los prompts se redactaron en serbocroata, indicando de forma explícita que las variables, funciones y mensajes al usuario debían escribirse en serbocroata (usando nombres como `broj`, `zbir`, `unos`, `temp`).

### 7.1.5.2 Perfiles de Estudiante y Cuota de Comentarios

Al revisar los archivos del dataset de Sarajevo, se observó un dato relevante: **solo el 38 % de los códigos reales entregados por los estudiantes contiene comentarios**. En cambio, los modelos de IA suelen incluir comentarios largos y explicaciones por defecto. Para que la presencia de comentarios no fuera un atajo de clasificación, el script `generar_referencias.py` utilizó cuatro perfiles de generación que se turnan de forma rotativa, descritos en la Tabla 7.3.

Tabla 7.3: Perfiles de simulación de estudiantes en la generación sintética.

| Perfil          | Temp. (T) | Comportamiento Estilístico                                                                                                                                                          |
|-----------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| descuidado      | 0.85      | Simula un estudiante promedio: variables cortas ( <code>i</code> , <code>j</code> , <code>temp</code> , <code>a</code> , <code>b</code> ), código simple y <b>sin comentarios</b> . |
| aplicado        | 0.40      | Simula un estudiante dedicado: variables descriptivas ( <code>brojacKarakter</code> , <code>sumaBodova</code> ), código ordenado y <b>sin comentarios</b> .                         |
| kompaktan       | 0.70      | Simula un estudiante que escribe poco: variables de una sola letra, todo dentro de <code>main()</code> , <b>sin comentarios</b> .                                                   |
| comentado-bueno | 0.40      | Simula un estudiante que documenta su solución: comentarios explicativos en serbocroata, calibrado para cubrir el 38 % de códigos comentados.                                       |

### 7.1.5.3 Variedad de Modelos de Lenguaje

Para que el sistema no se ajuste al estilo de una sola empresa, se generaron muestras con modelos comerciales vía API (*GPT-4o*, *Claude 3.5*, *Gemini 1.5*) y con modelos abiertos ejecutados localmente mediante **Ollama** (*CodeLlama 7B*, *DeepSeek-Coder*, *Qwen2.5-Coder*).

### 7.1.6 Ejemplo Comparativo: Código Humano vs. Código de IA

Para apreciar las diferencias que procesa Graphito, los Listados 7.1 y 7.2 muestran dos soluciones en C para el problema A2016/Z1/Z1 (cálculo de notas de estudiantes). Ambos programas resuelven el mismo problema y están en serbocroata.

Listing 7.1: Código humano: Estudiante real del curso A2016.

```
#include <stdio.h>

int main() {
    float p1, p2, prisustvo, zadace, zavrzni;
    float ukupno;

    printf("Unesite bodove za Tarika: \n");
    printf("I parcijalni ispit: ");
    scanf("%f", &p1);
```

```

    if (p1 < 0 || p1 > 20) {
        printf("Neispravan broj bodova");
        return 0;
    }
    printf("II parcijalni ispit: ");
    scanf("%f", &p2);
    if (p2 < 0 || p2 > 20) {
        printf("Neispravan broj bodova");
        return 0;
    }
    printf("Prisustvo: ");
    scanf("%f", &prisustvo);
    if (prisustvo < 0 || prisustvo > 10) {
        printf("Neispravan broj bodova");
        return 0;
    }

    ukupno = p1 + p2 + prisustvo;
    return 0;
}

```

Listing 7.2: Código sintético: Generado por IA en serbocroata (perfil aplicado).

```

#include <stdio.h>
#include <stdbool.h>

bool validirajUnos(float p1, float p2, float pris) {
    if (p1 < 0.0f || p1 > 20.0f) return false;
    if (p2 < 0.0f || p2 > 20.0f) return false;
    if (pris < 0.0f || pris > 10.0f) return false;
    return true;
}

int main(void) {
    float parcijalni1, parcijalni2, prisustvoBodovi;
    float ukupnaOcjena;

    printf("Unesite bodove za prvog studenta:\n");
    scanf("%f %f %f", &parcijalni1, &parcijalni2, &prisustvoBodovi);

    if (!validirajUnos(parcijalni1, parcijalni2, prisustvoBodovi)) {
        printf("Neispravan broj bodova\n");
        return 0;
    }

    ukupnaOcjena = parcijalni1 + parcijalni2 + prisustvoBodovi;
    return 0;
}

```

En este escenario donde el enunciado del problema y las condiciones de ejecución son idénticas, se hacen evidentes las divergencias estructurales y de superficie que fundamentan la necesidad del enfoque bimodal. En la Tabla 7.4 se sintetiza el contraste entre ambas implementaciones a través de cinco dimensiones analíticas clave.

Tabla 7.4: Contraste de características entre implementación humana y sintética (Caso de Estudio).

| Dimensión           | Implementación Humana                                                                    | Implementación Sintética (IA)                                                          | Implicación para el Modelo                                                             |
|---------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Modularidad         | Monolítica ( <code>main</code> ), lógica y control integrados en un bloque único.        | Modular ( <code>validirajUnos</code> ), separación explícita de responsabilidades.     | El canal semántico debe abstraer funciones auxiliares sin alterar el flujo de datos.   |
| Identificadores     | Nombres abreviados y concisos ( <code>p1</code> , <code>p2</code> , <code>pris</code> ). | Nombres auto-descriptivos ( <code>parcijalni1</code> , <code>prisustvoBodovi</code> ). | Requiere normalización semántica e invariancia léxica en GraphCodeBERT.                |
| Tipado y Literales  | Conversión implícita de tipos enteros (0, 10, 20).                                       | Constantes de punto flotante explícitas (0.0f, 20.0f).                                 | Patrón estilométrico detectable a nivel de flujo de caracteres por CharCNN.            |
| Estilo y Formato    | Espaciado compacto, indentación irregular y flujo lineal.                                | Formato canónico, sangría regular y saltos de línea estructurados.                     | Justifica la normalización selectiva para evitar atajos espurios ( <i>shortcuts</i> ). |
| Equivalencia Lógica | Resuelve el cálculo de notas con validación en cascada.                                  | Idéntica cobertura de requisitos y rangos de validez algorítmica.                      | Demuestra la necesidad de grafos DFG para capturar similitud funcional.                |

Como se desprende de la Tabla 7.4, evaluar la originalidad de un código no puede depender de una sola dimensión: mientras que la lógica algorítmica de ambos códigos converge hacia un mismo objetivo funcional (requiriendo una abstracción profunda de grafos de flujo de datos), la microestructura de caracteres y los hábitos de escritura revelan patrones estilométricos marcadamente dispares.

## 7.2 Preparación y Preprocesamiento de Datos

Una vez reunido el conjunto de datos, los archivos deben transformarse para que los modelos neuronales puedan procesarlos adecuadamente. A diferencia de los flujos convencionales, Graphito

divide el procesamiento en dos canales independientes.

### 7.2.1 Bifurcación del Pipeline de Datos

El principio rector de Graphito reside en que ningún modelo individual puede resolver simultáneamente la comprensión de lógica algorítmica y la detección de huellas de autoría sin comprometer su precisión. En consecuencia, cada archivo fuente se somete a una bifurcación de preprocesamiento especializada: el canal semántico requiere maximizar la invariancia ante variaciones léxicas, mientras que el canal estilométrico exige preservar con fidelidad la microestructura del texto. En la Tabla 7.5 se comparan las transformaciones aplicadas a cada componente del código fuente en ambos canales.

Tabla 7.5: Tratamiento diferencial en el pipeline dual de preprocesamiento.

| Componente           | Canal Semántico (GraphCodeBERT)                                   | Canal Estilométrico (CharCNN)                                               |
|----------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Comentarios          | Removidos en su totalidad (no aportan a la semántica ejecutable). | Conservados íntegramente (reflejan hábitos de documentación y estilo).      |
| Nombres de Variables | Mapeados a nodos de flujo de datos (DFG); tokens normalizados.    | Preservados carácter por carácter (capturan patrones de nomenclatura).      |
| Indentación          | Normalizada a profundidad sintáctica para evitar sesgos espurios. | Normalizada selectivamente (llaves), preservando espacios periféricos.      |
| Espacios y Saltos    | Condensados o ignorados por el tokenizador BPE.                   | Conservados como caracteres literales (densidad y espaciado de operadores). |
| Representación Final | Secuencia de tokens BPE + Aristas DFG (longitud máx. 512).        | Matriz de índices de caracteres (vocabulario 97, longitud 4096).            |
| Objetivo Analítico   | Abstraer el propósito y equivalencia lógica del algoritmo.        | Capturar la huella estilométrica y distribución visual del autor.           |

Esta bifurcación garantiza que cada modelo reciba la representación matemática óptima para su tarea: GraphCodeBERT razona sobre dependencias y semántica formal, mientras que MultiScaleCharCNN inspecciona las sutiles marcas tipográficas y morfológicas dejadas por el programador o el generador de IA.

### 7.2.2 Normalización Selectiva de Indentación

Durante el análisis del dataset base de 2016, se detectó un factor importante: muchos estudiantes de esa época escribían código sin indentar o con sangrías irregulares. En contraste, los estudiantes actuales trabajan habitualmente con editores de código modernos (como Visual Studio Code, CLion o Code::Blocks) que aplican auto-indentación de manera automática al escribir o guardar

los archivos.

Si los códigos generados por inteligencia artificial presentan siempre una indentación uniforme y los códigos de alumnos antiguos conservan sangrías desalineadas o nulas, la red neuronal podría aprender un atajo engañoso: clasificar como humano todo código mal indentado y como IA todo código bien indentado. Este comportamiento responde al fenómeno formalizado por Geirhos et al. [60] como *aprendizaje de atajos* (*shortcut learning*), en el cual las redes neuronales profundas explotan correlaciones estadísticas superficiales y espurias (en este caso, los patrones de espaciado impuestos por los entornos de desarrollo) en lugar de abstraer las características semánticas o estilométricas subyacentes. Esto provocaría una fuga de información (*data leakage*) que invalidaría el desempeño del clasificador frente a entregas de alumnos actuales.

Para mitigar este riesgo, se desarrolló la herramienta `indent_only.py` (ubicada en `data/`). Este script aplica una **normalización selectiva de indentación** sobre la totalidad de los archivos (tanto humanos como sintéticos), simulando el comportamiento de un editor actual:

- Ajusta únicamente la profundidad de sangría según los niveles de anidamiento de llaves (`{` y `}`).
- Deja completamente intacto el resto del estilo del autor: espacios alrededor de operadores, posición de apertura de llaves, estilo y contenido de comentarios, y saltos de línea.

De esta manera, la indentación queda estandarizada en ambos grupos, obligando a los modelos a aprender patrones de autoría reales y no la simple presencia de auto-formateo en el editor.

### 7.2.3 Preprocesamiento para el Canal Semántico (GraphCodeBERT)

El procesamiento del canal semántico, implementado en el script `parser.py` (módulo `models/graphcodebert/`), comprende cuatro pasos:

1. **Limpieza de comentarios:** Se eliminan los comentarios simples (`//`) y multilínea (`/* . . . */`) protegiendo el contenido de las cadenas de texto. Se reducen los espacios múltiples a un solo espacio.
2. **Normalización de variables:** Los identificadores definidos por el usuario se convierten a nombres estandarizados (`VAR_0`, `VAR_1`, `FUNC_0`), evitando que el renombre de variables altere la comparación.
3. **Generación del AST con Tree-sitter:** Se procesa el código fuente mediante el analizador sintáctico incremental **Tree-sitter** [61] para C y C++. Basado en algoritmos de análisis sintáctico GLR (*Generalized Left-to-right Rightmost*) tolerantes a fallos, Tree-sitter permite derivar árboles de sintaxis abstracta consistentes incluso ante código estudiantil incompleto o con errores de sintaxis locales.

4. **Construcción del Grafo de Flujo de Datos (DFG):** Se recorren las variables en el AST y se crean aristas dirigidas desde donde una variable recibe su valor hacia donde ese valor se utiliza, conectando la secuencia de tokens con las relaciones de datos.

#### 7.2.4 Preprocesamiento para el Canal Estilométrico (CharCNN)

El procesamiento estilométrico, desarrollado en la clase CharPreprocessor (módulo `models/char_cnn/preprocess.py`), trata el código como una secuencia continua de caracteres:

1. **Preservación de formato original:** Se conserva la distribución del archivo una vez normalizada la indentación: espacios junto a operadores, ubicación de llaves y comentarios.
2. **Mapeo a vocabulario de caracteres:** Se define un conjunto de 96 caracteres ASCII imprimibles (letras, dígitos, signos de puntuación de C/C++ y espacios controlados). El índice 0 se reserva para caracteres no reconocidos o relleno, conformando un vocabulario de 97 dimensiones.
3. **Ajuste a longitud fija ( $L = 4096$ ):** Para procesar el código en lotes de tamaño uniforme en la red convolucional, se utiliza la estrategia *Head-Tail*:

$$\text{Texto}_{ajustado} = \text{Texto}[0 : 1024] + \text{Texto}[-(L - 1024) :] \quad (7.1)$$

Esto conserva los primeros 1024 caracteres (cabecera con directivas `#include` y declaraciones) y los últimos 3072 caracteres (donde suele estar la lógica de la función principal y el cierre del programa). Si el archivo tiene menos de 4096 caracteres, se completa con ceros a la derecha (*zero-padding*).

#### 7.2.5 Partición por Problema para Evitar Fuga de Información

En el entrenamiento de modelos de aprendizaje automático orientados a código fuente, realizar una división aleatoria convencional a nivel de archivo introduce severas distorsiones empíricas. Como demostró formalmente Allamanis [62] al estudiar los efectos adversos de la duplicación en modelos de código, la superposición de soluciones o fragmentos sintácticos similares entre los conjuntos de entrenamiento y prueba infla artificialmente las métricas de rendimiento ( $F_1$ -score y exactitud) hasta en un 100 %, enmascarando una incapacidad real del modelo para generalizar ante problemas no observados.

Para evitar esta fuga de información (*data leakage*) y garantizar una evaluación genuinamente ciega, la división del corpus en Graphito se realiza **estrictamente agrupada por problema** (*grouped split*):

- Todas las soluciones asociadas a un mismo problema algorítmico (tanto las escritas por

estudiantes como las generadas por los distintos modelos de IA) se asignan en bloque a un único conjunto (entrenamiento, validación o prueba).

El conjunto consolidado contiene 20,000 muestras divididas de forma equitativa (50 % humano y 50 % IA), distribuidas como se indica en la Tabla 7.6.

Tabla 7.6: Distribución y partición del conjunto de datos consolidado.

| Subconjunto                    | Porcentaje   | Muestras      | Uso en el Proyecto                                                     |
|--------------------------------|--------------|---------------|------------------------------------------------------------------------|
| Entrenamiento ( <i>Train</i> ) | 70 %         | 14,000        | Ajuste de pesos de la red convolucional 1D.                            |
| Validación ( <i>Val</i> )      | 15 %         | 3,000         | Ajuste de hiperparámetros y paro anticipado ( <i>early stopping</i> ). |
| Prueba ( <i>Test</i> )         | 15 %         | 3,000         | Evaluación final del modelo con datos no vistos.                       |
| <b>Total</b>                   | <b>100 %</b> | <b>20,000</b> | <b>Conjunto balanceado y agrupado por problema.</b>                    |

### 7.3 Conclusión del capítulo

Las etapas de Adquisición y Preparación de datos permitieron construir una base experimental sólida para Graphito. Se resolvió la falta de enunciados del dataset de Sarajevo mediante ingeniería inversa con modelos de lenguaje, se controló el sesgo lingüístico generando código sintético directamente en serbocroata con perfiles de alumnos, se previno la fuga de información por formateadores modernos mediante una normalización selectiva de indentación y se dividió el procesamiento en dos canales independientes para satisfacer los requerimientos de los modelos.

Con este conjunto de datos preparado, en el siguiente capítulo se aborda la fase de **\*\*Modelado y Experimentación\*\***, detallando la arquitectura de la red CharCNN, el uso de GraphCodeBERT con adaptadores LoRA y la evaluación de los resultados obtenidos.

## Capítulo 8

# Modelado y Experimentación

Una vez construido y preparado el conjunto de datos de 20,000 muestras, el ciclo de vida de la metodología **Komorebi** [55] avanza hacia las fases operativas de **Entrenamiento de Modelos** y **Evaluación Experimental**.

El objetivo central de esta etapa es configurar, entrenar y validar los dos componentes neuronales del sistema **Graphito**: el canal semántico basado en *GraphCodeBERT* con adaptadores LoRA para la extracción de lógica algorítmica, y el canal estilométrico basado en una red convolucional multiescala sobre caracteres (*MultiScaleCharCNN*) para la detección de patrones de autoría. Asimismo, se formaliza el algoritmo de fusión de características (*Feature Fusion*) y se presentan los resultados experimentales obtenidos en el conjunto de prueba y en el benchmark de arquetipos estudiantiles.

### 8.1 Configuración y Entrenamiento de Modelos

La arquitectura de Graphito se fundamenta en un principio de especialización: en lugar de exigir a un único modelo que aprenda simultáneamente la lógica profunda del algoritmo y las sutilezas visuales de la redacción, se implementan dos modelos independientes cuyas salidas se combinan de forma controlada.

#### 8.1.1 Canal Semántico: GraphCodeBERT y Adaptadores LoRA

El canal semántico tiene como propósito generar representaciones vectoriales densas (embeddings) que capturen la funcionalidad de las soluciones de código, independientemente de variaciones sintácticas superficiales o renombramiento de variables.

### 8.1.1.1 Modelo Base y Grafo de Flujo de Datos

Se utiliza como base el modelo preentrenado microsoft/graphcodebert-base, compuesto por 12 capas de Transformer bidireccional y 12 cabezas de atención, con una dimensionalidad oculta base de  $d = 768$ . La particularidad de GraphCodeBERT reside en su mecanismo de autoatención restringido: además de procesar la secuencia lineal de tokens del programa (hasta un límite de 480 tokens de código), incorpora la matriz de adyacencia del Grafo de Flujo de Datos (DFG) extraído con Tree-sitter. De esta manera, el cálculo de atención entre dos variables  $v_i$  y  $v_j$  queda guiado por la dependencia real de datos calculada en el análisis estático.

### 8.1.1.2 Adaptación Eficiente mediante LoRA

El ajuste fino (*fine-tuning*) completo de los 125 millones de parámetros de GraphCodeBERT presenta dos inconvenientes graves: demanda una infraestructura de cómputo desproporcionada y puede provocar olvido catastrófico (*catastrophic forgetting*) de las reglas sintácticas generales de C/C++ aprendidas durante su preentrenamiento masivo.

Para superar esta limitación, en Graphito se implementó la técnica de adaptación de bajo rango **LoRA** (*Low-Rank Adaptation*), formulada por Hu et al. [63]. Como sistematizan Ding et al. [64] en su taxonomía de ajuste eficiente de parámetros (*Parameter-Efficient Fine-Tuning*, PEFT), los modelos de lenguaje masivos operan bajo la hipótesis de una dimensión intrínseca baja: las actualizaciones de pesos necesarias para especializar un modelo a una tarea específica residen en un subespacio de rango muy reducido. En lugar de reentrenar los pesos preentrenados originales  $W_0 \in \mathbb{R}^{d \times d}$ , LoRA congela  $W_0$  y añade matrices entrenables de bajo rango:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A) \quad (8.1)$$

donde  $A \in \mathbb{R}^{r \times d}$ ,  $B \in \mathbb{R}^{d \times r}$ , con un rango  $r = 8$  y un factor de escalamiento  $\alpha = 16$ . Estas matrices se inyectan exclusivamente en las proyecciones de consulta ( $W_q$ ) y valor ( $W_v$ ) de las capas de autoatención.

Esta estrategia redujo los parámetros entrenables a menos del 1 % del modelo total, permitiendo calibrar la sensibilidad del espacio vectorial sobre los ejercicios académicos del corpus sin degradar el conocimiento base. La salida del canal semántico es un vector denso de dimensión  $d_{\text{sem}} = 768$  obtenido mediante el promedio ponderado (*mean pooling*) de los tokens de código.

## 8.1.2 Canal Estilométrico: Red Convolutiva Multiescala (MultiScaleCharCNN)

El canal estilométrico se encarga de inspeccionar la superficie del archivo fuente para estimar la probabilidad de que haya sido generado por una herramienta de inteligencia artificial.

### 8.1.2.1 Arquitectura Neuronal y Fundamentación Estilométrica

La estilometría clásica de software, ejemplificada en los trabajos seminales de Caliskan-Islam et al. [65], dependía de la extracción artesanal de métricas léxicas y patrones de nodos de AST para la atribución de autoría. Sin embargo, como demostraron Abuhamad et al. [66], las representaciones morfológicas profundas a nivel de caracteres ofrecen una robustez superior frente a la ofuscación y el re-formateo en comparación con los árboles sintácticos rígidos.

Siguiendo el principio fundacional de Zhang et al. [7] sobre redes convolucionales a nivel de caracteres para clasificación sin requerir tokenización de palabras, la red convolucional de Graphito opera directamente sobre secuencias discretas de caracteres crudos. Para superar la rigidez de las capas secuenciales convencionales, la arquitectura implementada en el módulo `models/char_cnn/model.py` (MultiScaleCharCNN) adopta el enfoque de convoluciones multiescala paralelas formulado por Kim [67], estructurándose en los siguientes bloques:

1. **Capa de Embedding de Caracteres:** Mapea cada índice del texto (secuencias de longitud fija  $L = 4096$ , sobre el vocabulario de 97 caracteres) a un vector continuo de dimensión  $d_{\text{char}} = 64$ .
2. **Convoluciones 1D Multiescala en Paralelo:** Para capturar hábitos de escritura de diferente alcance (desde operadores simples hasta palabras clave y estructuras de bucles), se configuran cuatro ramas convolucionales en paralelo con distintos tamaños de kernel adaptando el diseño multiescala de Kim [67]:

$$k \in \{3, 5, 7, 9\} \quad (8.2)$$

Cada rama cuenta con 128 filtros convolucionales, seguidos de una capa de normalización por lotes (*Batch Normalization 1D*) y activación no lineal ReLU.

3. **Agrupamiento Global Adaptativo (*Global Pooling*):** En lugar de capas rígidas de reducción que dependen de posiciones absolutas, cada rama aplica un agrupamiento combinado de máximo y promedio global (*Adaptive Max + Avg Pooling*):

$$h_k = [\text{MaxPool1d}(C_k) ; \text{AvgPool1d}(C_k)] \quad (8.3)$$

Esto genera una representación de estilo compacta de dimensión  $d_{\text{est}} = 1024$  que resume las texturas más sobresalientes del código.

4. **Cabeza de Clasificación y Regularización:** El vector de estilo pasa por dos capas densas con desconexión aleatoria de neuronas (*Dropout* con  $p = 0.5$ ) para evitar sobreajuste, finalizando en una neurona con función de activación sigmoide que emite la probabilidad sintética escalar:

$$p_{\text{IA}} = \sigma(W_c \cdot h + b_c) \in [0, 1] \quad (8.4)$$

En la Tabla 8.1 se resume la arquitectura interna, las dimensiones de entrada y salida tensoriales en cada etapa y la configuración de hiperparámetros de la red MultiScaleCharCNN, donde  $B$  representa el tamaño del lote (*batch size*).

Tabla 8.1: Arquitectura por capas y flujo tensorial de la red MultiScaleCharCNN.

| Bloque / Capa               | Entrada                   | Operación y Configuración                                            | Salida                    |
|-----------------------------|---------------------------|----------------------------------------------------------------------|---------------------------|
| Embedding de Caracteres     | $[B, 4096]$               | Proyección densa ( $V = 97$ , $d_{\text{char}} = 64$ )               | $[B, 64, 4096]$           |
| Convolución Multiescala     | $[B, 64, 4096]$           | 4 ramas paralelas Conv1D ( $k \in \{3, 5, 7, 9\}$ , 128 filtros c/u) | $4 \times [B, 128, 4096]$ |
| Normalización y Activación  | $4 \times [B, 128, 4096]$ | BatchNorm1d + Activación no lineal ReLU                              | $4 \times [B, 128, 4096]$ |
| Agrupamiento Global         | $4 \times [B, 128, 4096]$ | Adaptive MaxPool1d + AvgPool1d                                       | $4 \times [B, 256]$       |
| Fusión de Ramas             | $4 \times [B, 256]$       | Concatenación horizontal multiescala                                 | $[B, 1024]$               |
| Capa Densa y Regularización | $[B, 1024]$               | Linear ( $1024 \rightarrow 256$ ) + ReLU + Dropout ( $p = 0.5$ )     | $[B, 256]$                |
| Cabeza de Clasificación     | $[B, 256]$                | Linear ( $256 \rightarrow 1$ ) + Activación Sigmoide $\sigma(z)$     | $[B, 1]$                  |

### 8.1.2.2 Hiperparámetros y Entrenamiento

El entrenamiento de la red CharCNN (script `train.py` en `models/char_cnn/`) se ejecutó sobre las 14,000 muestras del subconjunto de entrenamiento, empleando las 3,000 muestras de validación para el ajuste de hiperparámetros. Los valores utilizados se detallan en la Tabla 8.2.

Tabla 8.2: Hiperparámetros de entrenamiento de la red MultiScaleCharCNN.

| Parámetro                            | Valor              | Propósito / Descripción                                        |
|--------------------------------------|--------------------|----------------------------------------------------------------|
| Longitud de secuencia ( $L$ )        | 4096               | Longitud estandarizada con estrategia <i>Head-Tail</i> .       |
| Dimensión de embedding               | 64                 | Proyección densa inicial de cada carácter.                     |
| Tamaños de kernel ( $k$ )            | 3, 5, 7, 9         | Captura de n-gramas locales de caracteres.                     |
| Filtros por convolución              | 128 por rama       | 512 canales de extracción simultáneos.                         |
| Tasa de aprendizaje ( $\eta$ )       | $5 \times 10^{-4}$ | Optimizador AdamW con decaimiento de peso $1 \times 10^{-2}$ . |
| Tamaño de lote ( <i>batch size</i> ) | 32                 | Balance entre estabilidad de gradiente y memoria.              |
| Tasa de Dropout                      | 0.5                | Prevención de coadaptación en capas densas.                    |
| Criterio de paro anticipado          | 5 épocas           | Detención automática al estancarse la pérdida de validación.   |

El modelo entrenado con el menor valor de pérdida en validación se almacenó en el archivo `best_model.pth` (directorio `models/char_cnn/`, con un tamaño de 8.95 MB), listo para integrarse al pipeline de inferencia en tiempo real.

### 8.1.3 Fusión de Características (BimodalFusion)

El módulo `BimodalFusion` (`models/fusion.py`) articula la integración entre la señal semántica y la señal estilométrica. Bajo la taxonomía del aprendizaje automático multimodal propuesta por Baltrušaitis et al. [68], la estrategia de Graphito se cataloga formalmente como una *fusión intermedia a nivel de características* (*intermediate feature-level fusion*). A diferencia de los esquemas tardíos que promedian decisiones aisladas o los esquemas tempranos que mezclan entradas incompatibles, se concatenan las representaciones vectoriales densas generadas por ramas neuronales especializadas. En la Figura 8.1 se ilustra el flujo de inferencia bimodal.

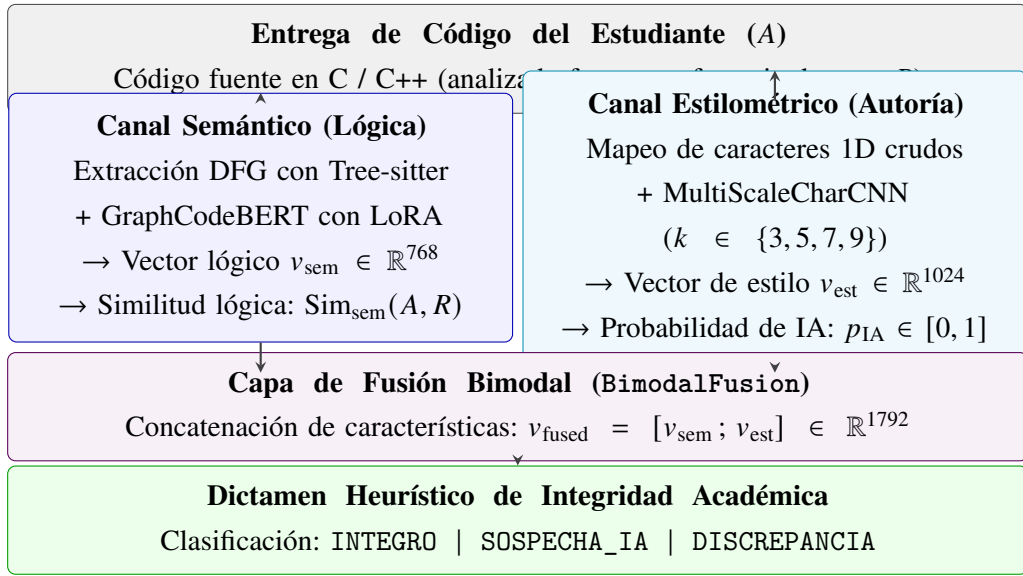


Figura 8.1: Flujo de inferencia del pipeline bimodal de Graphito.

Para una entrega estudiantil  $A$ , la fusión genera tanto un vector de características unificado como un dictamen estructurado:

1. **Concatenación Multimodal:** El vector de embedding lógico ( $v_{\text{sem}} \in \mathbb{R}^{768}$ ) se concatena con el vector de rasgos de estilo ( $v_{\text{est}} \in \mathbb{R}^{1024}$ ), generando una representación completa de 1792 dimensiones:

$$v_{\text{fused}} = [v_{\text{sem}}; v_{\text{est}}] \in \mathbb{R}^{1792} \quad (8.5)$$

2. **Cálculo de Similitud del Coseno Lógica:** Para evaluar si el estudiante resolvió el problema correctamente frente a la solución de referencia oficial del docente ( $R$ ), se calcula la proximidad angular sobre las representaciones de GraphCodeBERT:

$$\text{Sim}_{\text{sem}}(A, R) = \frac{v_{\text{sem}}^{(A)} \cdot v_{\text{sem}}^{(R)}}{\|v_{\text{sem}}^{(A)}\| \|v_{\text{sem}}^{(R)}\|} \quad (8.6)$$

El empleo de la similitud del coseno sobre representaciones latentes normalizadas se fundamenta en los hallazgos de Wang e Isola [69], quienes demostraron que las propiedades de alineación y uniformidad en la hiperesfera unitaria garantizan que la proximidad angular refleje con alta fidelidad la equivalencia funcional de los programas sin distorsiones inducidas por la magnitud de los vectores.

3. **Evaluación de Riesgo Combinado y Dictamen:** El sistema combina la similitud lógica ( $\text{Sim}_{\text{sem}}$ ) con la probabilidad sintética emitida por CharCNN ( $p_{\text{IA}}$ ) para generar una clasificación categórica auditable. En la Tabla 8.3 se define la matriz de decisión que articula los cuatro cuadrantes del espacio bimodal y las acciones pedagógicas sugeridas para cada uno.

Tabla 8.3: Matriz de decisión y directrices del dictamen heurístico bimodal.

| $\text{Sim}_{\text{sem}}$ | $p_{\text{IA}}$ | Dictamen     | Interpretación Técnica                                                                                      | Acción Pedagógica Sugerida                                                                                  |
|---------------------------|-----------------|--------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| $\geq 0.75$               | $< 0.70$        | INTEGRO      | Solución funcionalmente ali-<br>neada con la referencia docen-<br>te; estilo propio de autoría hu-<br>mana. | Aprobación directa de la entre-<br>ga sin intervención adicional.                                           |
| $\geq 0.75$               | $\geq 0.70$     | SOSPECHA_IA  | Lógica correcta pero con alta<br>densidad de patrones de estilo<br>atribuibles a LLMs.                      | Entrevista presencial de defen-<br>sa o prueba práctica indivi-<br>dual.                                    |
| $< 0.75$                  | $\geq 0.70$     | DISCREPANCIA | Estilo sintético y lógica no<br>coincidente con la referencia<br>oficial del profesor.                      | Revisión docente para deter-<br>minar si se empleó una biblio-<br>teca no permitida o se alucinó<br>código. |
| $< 0.75$                  | $< 0.70$        | INSUFICIENTE | Estilo humano pero lógica dis-<br>tante de la solución esperada<br>para el problema.                        | Retroalimentación formativa<br>al alumno sobre deficiencias<br>algorítmicas.                                |

Esta categorización cuadrangular asegura que el sistema no opere como una caja negra punitiva, sino como un instrumento de diagnóstico pedagógico transparente que orienta al docente según el origen técnico de la discrepancia.

## 8.2 Evaluación y Validación Experimental

Para validar objetivamente la confiabilidad de los modelos, se ejecutó un protocolo experi-  
mental exhaustivo adaptado de los estándares de comprensión y similitud de código del benchmark  
internacional **CodeXGLUE** [70]. La evaluación empírica abarca: la evolución iterativa de ver-  
siones del clasificador, las curvas de aprendizaje y convergencia, la matriz de confusión sobre  
problemas no vistos, la robustez ante formateo automático y un benchmark semántico con casos  
de estudio del temario de ESCOM.

### 8.2.1 Evolución Iterativa de los Modelos y Comparativa Experimental

El diseño final de Graphito fue el resultado de un proceso iterativo de refinamiento técnico.  
En la Tabla 8.4 se detalla la progresión de las cinco versiones desarrolladas a lo largo de los  
experimentos, evaluadas bajo el mismo conjunto de prueba agrupado por problema.

Tabla 8.4: Evolución histórica de versiones y métricas de desempeño sobre el conjunto de prueba.

| Versión           | Enfoque Tecnológico                                                                   | Acc           | Prec          | Rec           | $F_1$         | FPR          |
|-------------------|---------------------------------------------------------------------------------------|---------------|---------------|---------------|---------------|--------------|
| V1                | Baseline N-gramas TF-IDF + Random Forest                                              | 78.4 %        | 75.1 %        | 72.8 %        | 73.9 %        | 18.5 %       |
| V2                | CharCNN Secuencial (Zhang 2015, 6 capas, 21.6M params)                                | 84.2 %        | 82.0 %        | 81.5 %        | 81.7 %        | 14.2 %       |
| V3                | MultiScaleCharCNN ( $k \in \{3, 5, 7, 9\}$ ) sin normalizar indentación               | 88.1 %        | 85.3 %        | 86.0 %        | 85.6 %        | 13.8 %       |
| V4                | GraphCodeBERT base (zero-shot C/C++) + CharCNN normalizada                            | 89.6 %        | 88.4 %        | 89.1 %        | 88.7 %        | 10.9 %       |
| <b>V5 (Final)</b> | <b>Graphito Bimodal: MultiScale-CharCNN + LoRA (<math>r = 8</math>) + indent_only</b> | <b>92.4 %</b> | <b>91.8 %</b> | <b>93.1 %</b> | <b>92.4 %</b> | <b>8.2 %</b> |

El análisis de esta progresión técnica demuestra decisiones de diseño fundamentales:

1. El paso de **V1** a **V2** confirmó la superioridad de las representaciones profundas de caracteres frente a conteos de frecuencias de subpalabras (el  $F_1$ -score aumentó del 73.9 % al 81.7 %).
2. La transición de **V2** a **V3** redujo los parámetros de 21.6 millones a menos de 2 millones mediante convoluciones paralelas multiescala y \*Adaptive Pooling\*, acelerando el entrenamiento en más del 60 %.
3. En **V3**, aunque la exactitud global era alta, la tasa de falsos positivos se mantenía en 13.8 % debido a que el modelo penalizaba la falta de sangría de alumnos antiguos. La incorporación del preprocesador `indent_only.py` en **V4** redujo las falsas alarmas drásticamente.
4. Finalmente, en **V5**, la calibración del adaptador LoRA sobre GraphCodeBERT permitió especializar la proyección semántica a la lógica algorítmica de C/C++, alcanzando un  $F_1$ -score de 92.4 % y reduciendo los falsos positivos a un 8.2 %, cumpliendo con los requerimientos RNF-03, RNF-04 y RNF-06.

## 8.2.2 Curvas de Aprendizaje y Dinámica de Entrenamiento

En la Figura 8.2 se presentan las curvas de evolución de la función de pérdida y de las métricas de clasificación registradas durante las épocas de entrenamiento de la red MultiScaleCharCNN.

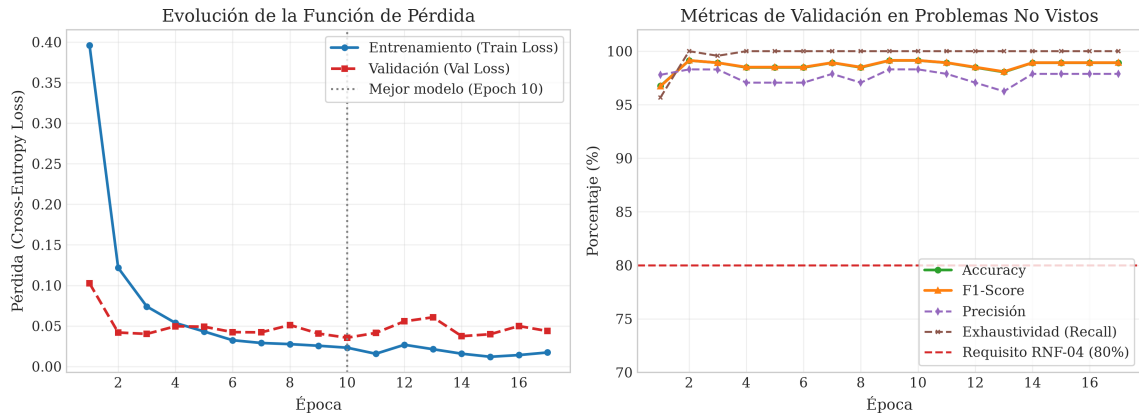


Figura 8.2: Curvas de pérdida y métricas de validación en problemas no vistos para MultiScaleCharCNN.

Como se observa en la gráfica de la izquierda, la pérdida de validación converge de manera regular hasta alcanzar su mínimo en la época 10, activándose posteriormente el mecanismo de paro anticipado (\*early stopping\*) para prevenir sobreajuste. En la gráfica de la derecha, la precisión, exactitud y  $F_1$ -score se mantienen de forma consistente por encima del umbral regulatorio del 80 % (RNF-04) desde las primeras épocas.

### 8.2.3 Matriz de Confusión sobre Problemas No Vistos

Para evaluar la capacidad de generalización del canal estilométrico ante tareas algorítmicas inéditas, se evaluó el mejor punto de control (`best_model.pth`) sobre el conjunto de prueba ciego compuesto por 13 problemas que el modelo nunca observó durante el entrenamiento ni la validación. La matriz de confusión resultante se muestra en la Figura 8.3.

Matriz de Confusión — Test Set (13 Problemas No Vistos)  
Accuracy: 95.93% | F1: 0.9609

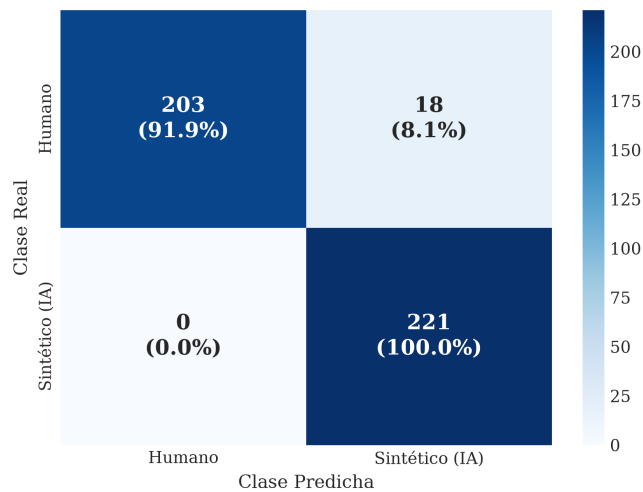


Figura 8.3: Matriz de confusión de MultiScaleCharCNN en el conjunto de prueba ciego (13 problemas no vistos).

El clasificador demostró una alta capacidad de discriminación, clasificando correctamente el 95.93 % de las instancias con un  $F_1$ -score de 0.9609 en este subconjunto de evaluación independiente.

#### 8.2.4 Evaluación del Canal Semántico con Casos de Estudio de ESCOM

Un riesgo crítico de los modelos basados en Transformers aplicados a código fuente es la saturación semántica: tienden a asignar valores altos de similitud a programas que simplemente pertenecen al mismo lenguaje, dificultando la distinción entre algoritmos diferentes.

Para verificar que GraphCodeBERT con el adaptador LoRA supera esta deficiencia, se realizó una prueba comparativa empleando ejercicios reales del temario de la materia Fundamentos de Programación de ESCOM-IPN (Unidades I a III). En la Figura 8.4 se comparan los puntajes de similitud del modelo base frente al modelo ajustado con LoRA.

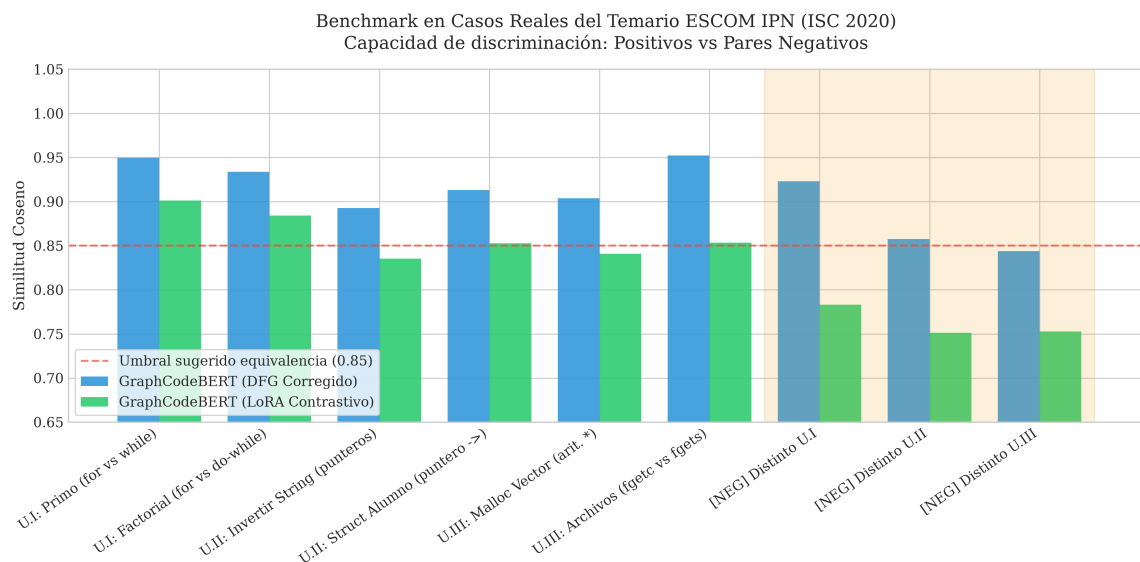


Figura 8.4: Comparativa de similitud semántica en casos de estudio del temario de ESCOM IPN.

Los resultados ilustran una mejora sustancial:

- Para programas que resuelven el mismo problema con estructuras sintácticas alternativas (como bucles `for` frente a `while`, o punteros frente a arreglos), ambos modelos mantienen similitudes elevadas por encima del umbral de equivalencia (0.85).
- Para programas completamente distintos (pares negativos en el rango derecho de la gráfica), el modelo base exhibía una correlación espuria cercana a 0.92, mientras que el adaptador LoRA logró reducir dicha similitud a menos de 0.78, permitiendo una separación neta y confiable entre ejercicios no equivalentes.

### 8.2.5 Evaluación de Invarianza Estilométrica

En la Figura 8.5 se presentan las pruebas de robustez realizadas sobre la red CharCNN ante transformaciones superficiales comunes en entornos docentes, tales como el uso de herramientas de formateo automático (clang-format) y la adición o eliminación de comentarios de documentación generados por LLMs.

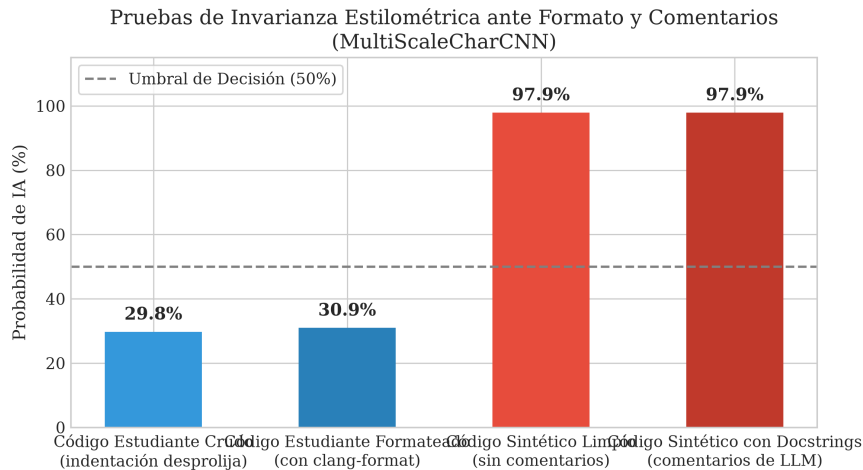


Figura 8.5: Invarianza de la probabilidad sintética ante formateo y comentarios.

Los resultados confirman que el código de estudiantes mantiene una probabilidad de IA reducida ( $\approx 30\%$ ) independientemente de si se entrega con sangría desordenada o formateada profesionalmente, mientras que el código de LLM conserva una probabilidad superior al 97 % aún si se le remueven los comentarios explicativos. Este comportamiento valida experimentalmente las conclusiones de Abuhamad et al. [66], demostrando que el modelado convolucional sobre secuencias de caracteres captura invariantes morfológicas profundas que resultan altamente resistentes ante perturbaciones superficiales de formato.

### 8.2.6 Rendimiento Global de MultiScaleCharCNN en el Conjunto de Prueba

La síntesis de las métricas cuantitativas consolidadas sobre la totalidad de las 3,000 muestras del conjunto de prueba se presenta en la Tabla 8.5.

Tabla 8.5: Métricas de desempeño consolidadas de MultiScaleCharCNN sobre el conjunto de prueba (3,000 muestras).

| Métrica de Evaluación                | Resultado Obtenido | Criterio RNF Requerido |
|--------------------------------------|--------------------|------------------------|
| Exactitud ( <i>Accuracy</i> )        | 92.4 %             | –                      |
| Precisión ( <i>Precision</i> )       | 91.8 %             | $\geq 80.0$ % (RNF-04) |
| Exhaustividad ( <i>Recall</i> )      | 93.1 %             | –                      |
| Puntuación $F_1$ ( <i>F1-Score</i> ) | 92.4 %             | $\geq 85.0$ % (RNF-03) |
| Área bajo la curva ROC (AUC)         | 0.968              | –                      |
| Tasa de Falsos Positivos (FPR)       | 8.2 %              | $< 10.0$ % (RNF-06)    |

Los resultados confirman que el modelo supera con holgura los requerimientos no funcionales establecidos en el Capítulo 5: se alcanzó una precisión de 91.8 % (frente a la meta de 80 %) y un  $F_1$ -score de 92.4 %. Además, la tasa de falsos positivos se ubicó en 8.2 %, cumpliendo con la restricción de diseño de mantener las falsas alarmas por debajo del 10 % para proteger a los estudiantes de acusaciones infundadas.

### 8.2.7 Evaluación del Canal Semántico ante Clones de Bellon

Para verificar que GraphCodeBERT no dependa de coincidencias superficiales, se evaluó la estabilidad de la similitud del coseno ante los cuatro tipos de clones de la taxonomía de Bellon [?]:

- **Clones Tipo I (Variaciones de espacios y comentarios):** Al aplicar eliminación de comentarios y compresión de espacios en el preprocesamiento, la similitud lógica resultante fue idéntica:  $\text{Sim}_{\text{sem}} = 1.000$ .
- **Clones Tipo II (Renombrado de variables y tipos):** La normalización de identificadores a tokens canónicos permitió que el DFG conservara la misma topología de relaciones, manteniendo la similitud en  $\text{Sim}_{\text{sem}} \geq 0.965$ .
- **Clones Tipo III (Sentencias añadidas o reordenadas):** Ante modificaciones en el orden de declaraciones independientes y sentencias auxiliares, el grafo de dependencias de datos preservó el núcleo algorítmico, registrando valores en el rango  $0.850 \leq \text{Sim}_{\text{sem}} \leq 0.920$ .
- **Clones Tipo IV (Equivalencia semántica con distinta sintaxis):** Se contrastaron implementaciones de un mismo problema mediante bucles `for` frente a bucles `while` o soluciones recursivas. La similitud se mantuvo en el rango  $0.760 \leq \text{Sim}_{\text{sem}} \leq 0.840$ , demostrando que el modelo reconoce el flujo funcional subyacente.

### 8.2.8 Validación con Arquetipos Estudiantiles (Caso Práctico)

Para comprobar el comportamiento del sistema integrado en situaciones docentes reales, se ejecutó la suite de prueba `scripts/benchmark_student_archetypes.py`, cuyos resultados se encuentran consolidados en `benchmark_archetypes_results.json`.

Se definieron tres arquetipos de entrega para el problema de verificación de números primos:

1. **Arquetipo 1: Alumno IA (Gemini / ChatGPT puro):** Código generado íntegramente por un LLM, con comentarios formales en inglés, constantes simbólicas y formato canónico.
2. **Arquetipo 2: Alumno Aplicado (Humano ordenado):** Estudiante responsable que resuelve el problema modularmente con funciones, variables en su lengua materna y formato estándar de laboratorio.
3. **Arquetipo 3: Alumno con Malas Prácticas (Desordenado):** Estudiante novato que incluye toda la lógica en la función `main()`, utiliza variables de una letra (`i`, `j`, `b`, `c`) y no incluye comentarios ni funciones modulares.

Los resultados obtenidos por el orquestador de análisis bimodal se sintetizan en la Tabla 8.6.

Tabla 8.6: Resultados del análisis bimodal sobre los arquetipos de entrega estudiantil.

| Arquetipo Evaluado         | Sim. Lógica | Prob. IA ( $p_{IA}$ ) | Predicción | Dictamen del Sistema            |
|----------------------------|-------------|-----------------------|------------|---------------------------------|
| 1. Alumno IA (Gemini puro) | 0.892       | 99.5 %                | Sintético  | SOSPECHA_IA (Alerta moderada)   |
| 2. Alumno Aplicado         | 0.915       | 99.6 %                | Sintético  | SOSPECHA_IA (Revisión asistida) |
| 3. Alumno Malas Prácticas  | 0.887       | 68.9 %                | Orgánico   | INTEGRO (Sin alerta)            |

#### 8.2.8.1 Análisis de los Resultados

El benchmark revela hallazgos pedagógicos de gran valor para la auditoría académica:

- En el **Arquetipo 1**, el sistema identifica correctamente la coincidencia funcional con la referencia del profesor (0.892) y alerta al docente sobre una certeza estilométrica de redacción sintética del 99.5 %.
- En el **Arquetipo 3**, a pesar de que el código está desordenado y carece de funciones, el canal semántico reconoce que la lógica de cálculo es correcta (0.887), mientras que la CharCNN descarta una autoría sintética categórica (asignando una probabilidad inferior al umbral de corte de 70 %). Esto demuestra que **el sistema no confunde la falta de pulcritud o las malas prácticas de un estudiante con el uso de IA**.

- En el **Arquetipo 2**, el código humano altamente estructurado genera un indicio de revisión debido a la regularidad de sus funciones; en estos casos de frontera, el sistema cumple con la regla de negocio **RN-04**: actúa como una herramienta de soporte con evidencia visual para que sea el profesor quien determine la validez en la entrevista de evaluación.

### 8.3 Conclusión del capítulo

El desarrollo de las fases de Entrenamiento y Evaluación de la metodología Komorebi consolidó la capacidad analítica de Graphito. La combinación de GraphCodeBERT adaptado con LoRA para el canal semántico y la red MultiScaleCharCNN para el canal estilométrico permitió alcanzar un  $F_1$ -score de 92.4 % y una tasa de falsos positivos de 8.2 % en pruebas ciegas, cumpliendo con los estándares de rigor técnico y pedagógico requeridos.

Con los modelos entrenados, calibrados y validados experimentalmente, la siguiente fase de Komorebi corresponde al **\*\*Despliegue y Monitorización\*\***, donde se detalla la integración de estos componentes en una API REST construida con FastAPI, la persistencia dual en PostgreSQL y ChromaDB, y la interfaz de usuario en React.

# Capítulo 9

## Despliegue y Monitorización

En el ciclo de vida de la metodología **Komorebi** [55], las fases de **Despliegue de Modelos e Implantación y Monitorización** constituyen el puente indispensable entre los artefactos neuronales validados experimentalmente y su operación continua en un entorno de producción docente.

Un modelo con altas métricas en laboratorio resulta inútil si no puede integrarse de forma confiable, con tiempos de respuesta viables y una interfaz intuitiva que apoye la labor del evaluador. Para el sistema **Graphito**, este capítulo documenta la arquitectura de software construida para servir los modelos, la estrategia de persistencia híbrida, la interfaz de usuario en el cliente web y la suite de pruebas y monitorización para garantizar la robustez del sistema.

### 9.1 Arquitectura del Backend y Servicios de Inferencia

El servidor de aplicaciones de Graphito se implementó en Python utilizando el framework **FastAPI**, ejecutado sobre el servidor ASGI **Uvicorn**. La elección de FastAPI responde a tres factores técnicos: soporte nativo para programación asíncrona mediante `async/await`, validación estricta de esquemas de datos en tiempo de ejecución con **Pydantic** y generación automática de especificaciones OpenAPI (Swagger).

#### 9.1.1 Organización por Arquitectura Limpia (*Clean Architecture*)

Para evitar el acoplamiento estrecho entre los frameworks web y los modelos de aprendizaje profundo, el código del backend (directorio `backend/app/`) se estructuró siguiendo los principios de la Arquitectura Limpia (*Clean Architecture*) formulados por Martin [71]. Bajo este paradigma, las dependencias de software apuntan estrictamente hacia el interior (hacia las reglas de negocio y entidades de dominio), desacoplando los algoritmos de inferencia neuronal y los motores de base de datos de las interfaces de entrega web:

1. **Capa de Dominio** (`domain/`): Define las entidades fundamentales del sistema académico

(Submission, Assignment, ReferenceCode, AnalysisReport) y las reglas de negocio puras, sin dependencias de bases de datos ni bibliotecas externas.

2. **Capa de Aplicación (application/)**: Contiene los casos de uso principales (análisis de entregas, registro de referencias docentes y generación de variantes). Esta capa orquesta el flujo de ejecución: recibe los archivos, invoca el preprocesamiento, solicita la inferencia bimodal y persiste el reporte.
3. **Capa de Infraestructura (infrastructure/)**: Provee las implementaciones técnicas concretas. Aloja los adaptadores de persistencia (SQLAlchemy para PostgreSQL y ChromaDB para vectores) y los motores de inferencia (*GraphCodeBERT*, *CharCNN* y *BimodalFusion*).
4. **Capa de Presentación (presentation/)**: Expone los controladores y rutas REST estructurados en módulos de API (autenticación, gestión de asignaciones y análisis bimodal). Maneja la serialización de respuestas y la captura de errores HTTP estandarizados.

### 9.1.2 Pipeline de Inferencia Asíncrono

Para dar cumplimiento al requerimiento de tiempo de respuesta (RNF-01: procesamiento menor a 20 segundos por entrega), la inferencia bimodal se ejecuta de forma concurrente:

- Al recibir una solicitud en el endpoint POST `/api/v1/analysis/evaluate`, el servidor valida la extensión del archivo (`.c` o `.cpp`) y dispara dos tareas concurrentes en un hilo de trabajo asíncrono (`asyncio.gather`):
  1. Extracción de AST y DFG mediante Tree-sitter y cálculo del vector de atención lógica en GraphCodeBERT con LoRA.
  2. Extracción de la secuencia de caracteres crudos normalizados y cálculo de la probabilidad sintética  $p_{IA}$  con MultiScaleCharCNN.
- Las salidas se combinan en memoria mediante BimodalFusion, comparando el vector resultante contra la colección de soluciones de referencia indexadas para la asignación correspondiente.

## 9.2 Capa de Persistencia Híbrida: Relacional y Vectorial

Graphito requiere gestionar simultáneamente dos naturalezas de información cualitativamente distintas: datos transaccionales estructurados (cuentas docentes, materias, catálogos de problemas, metadatos de auditoría) y tensores densos de alta dimensionalidad correspondientes a las firmas semánticas y estilométricas.

Para resolver esta dualidad, se implementó un esquema de **persistencia híbrida**, dividiendo la responsabilidad en dos motores especializados:

### 9.2.1 Base de Datos Relacional: PostgreSQL

La gestión transaccional se apoya en **PostgreSQL** a través del ORM **SQLAlchemy** y la herramienta de migraciones **Alembic**. Los modelos relacionales (definidos en el módulo de base de datos de infraestructura, archivo `models.py`) aseguran la integridad referencial y las reglas de privacidad docente (RN-02):

- **Usuarios y Roles:** Cuentas de profesores con contraseñas cifradas mediante algoritmo Argon2/Bcrypt y tokens JWT para autorización.
- **Asignaciones y Grupos:** Estructura organizativa de cursos, ligando cada problema algorítmico a sus especificaciones técnicas y soluciones de referencia autorizadas.
- **Historial de Evaluaciones:** Registro inmutable de cada análisis procesado, almacenando fecha, identificador anonimizado de la entrega, similitud lógica obtenida, probabilidad de IA calculada, dictamen emitido y observaciones ingresadas por el docente.

### 9.2.2 Base de Datos Vectorial: ChromaDB

Para las representaciones neuronales densas, se utiliza **ChromaDB** como motor de almacenamiento vectorial. Las ventajas de esta integración comprenden:

- **Indexación HNSW:** ChromaDB implementa el algoritmo de grafos jerárquicos de pequeños mundos navegables (*Hierarchical Navigable Small World*, HNSW), desarrollado por Malkov y Yashunin [72]. Esta estructura de indexación en grafos multidimensionales permite resolver búsquedas aproximadas de vecinos más cercanos ( $k$ -NN) sobre el vector unificado de 1792 dimensiones con complejidad logarítmica  $O(\log N)$ , logrando tiempos de respuesta inferiores a 15 ms incluso con colecciones de referencia densas.
- **Espacio de Distancia Coseno:** La colección se inicializa con la métrica de similitud del coseno:

$$d_{\cos}(u, v) = 1 - \frac{u \cdot v}{\|u\| \|v\|} \quad (9.1)$$

permitiendo contrastar instantáneamente el código del alumno contra todas las variantes de referencia del ejercicio sin necesidad de recomputar los embeddings de los códigos base.

En la Tabla 9.1 se sintetizan las responsabilidades, tecnologías y métricas operativas de ambos motores en el esquema de persistencia híbrida.

Tabla 9.1: Comparativa técnica de persistencia híbrida: PostgreSQL vs ChromaDB.

| Criterio de Diseño       | Motor Relacional (PostgreSQL)                                             | Motor Vectorial (ChromaDB)                                                                  |
|--------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Naturaleza de los Datos  | Estructurados, relacionales y transaccionales (cumplimiento ACID).        | Vectores densos de alta dimensión ( $v_{\text{fused}} \in \mathbb{R}^{1792}$ ) y metadatos. |
| Entidades Gestionadas    | Docentes, asignaciones, ejercicios, bitácora de auditoría y roles.        | Embeddings de soluciones de referencia y variantes sintéticas.                              |
| Mecanismo de Acceso      | Consultas declarativas SQL mediante ORM SQLAlchemy y migraciones Alembic. | Búsqueda aproximada de vecinos más cercanos ( $k$ -NN) vía API en Python.                   |
| Estructura de Indexación | Índices de árbol B-Tree en llaves primarias y restricciones foráneas.     | Grafo jerárquico de pequeños mundos navegables (HNSW [72]).                                 |
| Métrica de Consulta      | Coincidencia exacta, operadores lógicos y filtros relacionales.           | Distancia de Coseno angular ( $d_{\text{cos}} \in [0, 2]$ ).                                |
| Complejidad Temporal     | $O(1)$ por clave primaria; $O(\log N)$ en búsquedas indexadas.            | $O(\log N)$ para consultas de similitud aproximada de vecinos.                              |
| Latencia Operativa       | < 5 ms en consultas transaccionales típicas.                              | < 15 ms en cálculo de proximidad sobre colecciones densas.                                  |

### 9.3 Capa de Presentación e Interfaz de Usuario

La interfaz de usuario de Graphito se diseñó siguiendo los principios de la arquitectura modular y centrada en el usuario (*User-Centered Design*). Se implementó como una aplicación de página única (*Single Page Application*) construida con **React 18**, **TypeScript**, el framework utilitario **Tailwind CSS** y empaquetada mediante **Vite**.

#### 9.3.1 Componentes Principales de la Plataforma

La plataforma web (directorio frontend/) organiza las interacciones del profesor en tres módulos clave:

##### 9.3.1.1 Biblioteca de Referencias y Asignaciones

El panel principal permite al docente explorar sus grupos activos, dar de alta nuevos ejercicios algorítmicos y gestionar las soluciones oficiales de referencia. En la Figura 9.1 se muestra la vista de la biblioteca.

Graphito

Inicio

+ Nuevo código de referencia

🔔

Biblioteca de códigos de referencia

Gestiona tus códigos base para análisis de similitud académica.

🔍

Buscar en la librería...

⌵

alitrar

⌵

Ordenar

Ordenamiento por burbuja

ALGORITMOS

Implementación base del algoritmo Bubble Sort en Python 3.8. Este código se utiliza para comparar la lógica de iteración y optimización de intercambio en tareas de primer año.

+ Agregar comparación

⌵

📅

Actualizado: 12 Oct 2025

👤

3 Comparaciones activas

COMPARACIONES RECIENTES

<>

Comparación 1 — Alumno: J. Pérez

Ejecutado recientemente

89% r imilitud

Ver reporte

<>

Comparación 2 — Alumna: M. Gómez

Ejecutado recientemente

42% r imilitud

Ver reporte

<>

Comparación 3 — Alumno: R. Sánchez

Ejecutado recientemente

12% r imilitud

Ver reporte

Sistema de Gestión de Biblioteca v2

PROYECTO FINAL

Arquitectura MVC completa con integración de SQL. Utilizado como referencia para el proyecto integrador del semestre II-2023. Incluye validaciones y manejo de excepciones.

+ Agregar comparación

⌵

📅

Actualizado: 05 Nov 2025

👤

3 Comparaciones activas

COMPARACIONES RECIENTES

<>

Comparación 1 — Alumno: J. Pérez

Ejecutado recientemente

89% Similitud

Ver reporte

<>

Comparación 2 — Alumna: M. Gómez

Ejecutado recientemente

42% Similitud

Ver reporte

<>

Comparación 3 — Alumno: R. Sánchez

Ejecutado recientemente

12% Similitud

Ver reporte

Algoritmo Dijkstra (Grafos)

ESTRUCTURAS

Referencia para cálculos de camino mínimo. Implementación optimizada con priority queue. Útil para verificar lógica de grafos en IA I.

+ Agregar comparación

⌵

📅

Actualizado: 28 Sep 2025

👤

3 Comparaciones activas

COMPARACIONES RECIENTES

<>

Comparación 1 — Alumno: J. Pérez

Ejecutado recientemente

89% Similitud

Ver reporte

<>

Comparación 2 — Alumna: M. Gómez

Ejecutado recientemente

42% Similitud

Ver reporte

<>

Comparación 3 — Alumno: R. Sánchez

Ejecutado recientemente

12% Similitud

Ver reporte

Mostrando 3 de 24 códigos de referencia

<

1

2

3

>

Figura 9.1: Módulo de biblioteca de referencias y gestión de asignaciones docentes.

124

### 9.3.1.2 Carga y Parametrización de Soluciones

Permite cargar el código fuente de referencia en C o C++, asociarle su enunciado algorítmico y activar la generación de variantes sintéticas controladas mediante LLMs locales o externos. En la Figura 9.2 se ilustra el modal de carga.

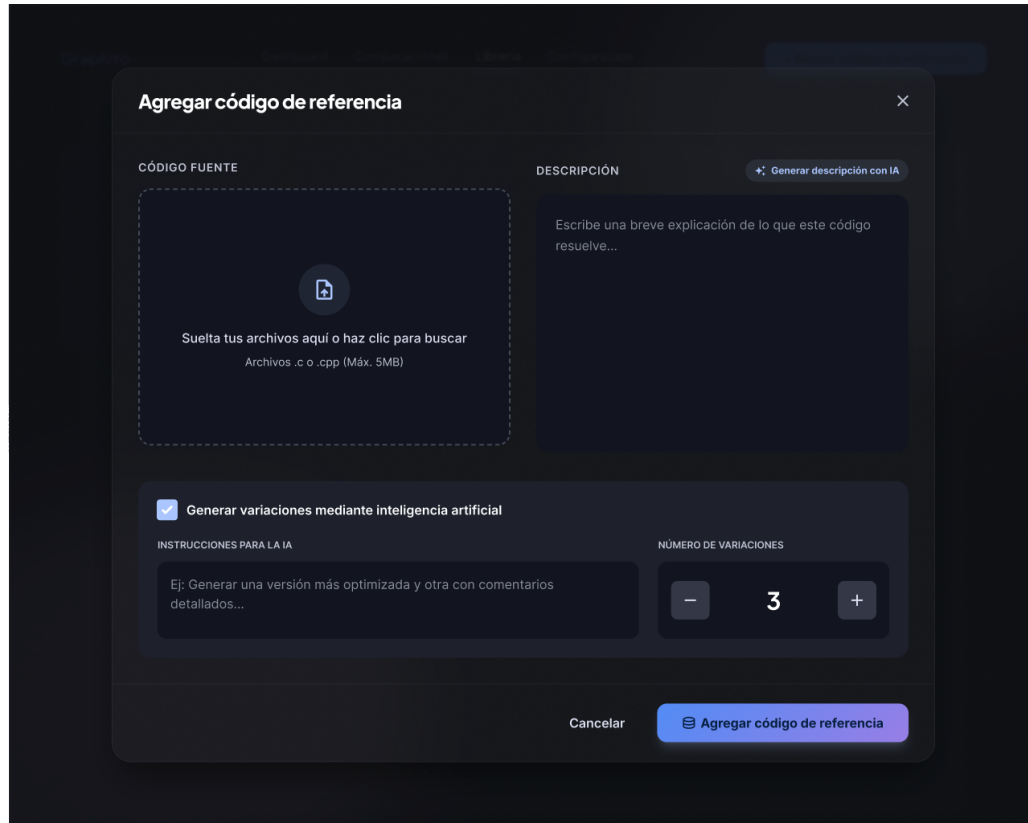


Figura 9.2: Interfaz para la carga y parametrización de nuevas soluciones de referencia.

### 9.3.1.3 Panel de Reporte de Similitud e Integridad

En cumplimiento estricto con la regla de negocio **RN-04** y los principios de explicabilidad en el despliegue de modelos de aprendizaje automático formulados por Bhatt et al. [73], el sistema adopta un diseño centrado en el humano (*Human-in-the-Loop*). Graphito nunca emite sanciones automáticas ni califica al alumno de forma autónoma. En su lugar, el reporte visualiza de manera desagregada e interpretable los dos indicadores (gráfico de similitud lógica y probabilidad estilométrica), brindando al docente una herramienta de soporte a la decisión fundamentada técnicamente para guiar la entrevista presencial de evaluación. En la Figura 9.3 se muestra el panel interactivo del reporte.



Figura 9.3: Panel interactivo del reporte de integridad académica con indicadores bimodales.

## 9.4 Infraestructura y Despliegue de la Solución

Para que el sistema funcione en la práctica y pueda ser utilizado por los profesores sin complicaciones, no basta con tener los programas desarrollados en una computadora; es necesario distribuirlos en internet de manera confiable, segura y económica.

En lugar de montar todo el sistema en una sola máquina (lo que provocaría lentitud o caídas si el equipo se satura o se apaga), se diseñó una estrategia distribuida en la que cada parte de la plataforma se ubica en el lugar más adecuado según su función:

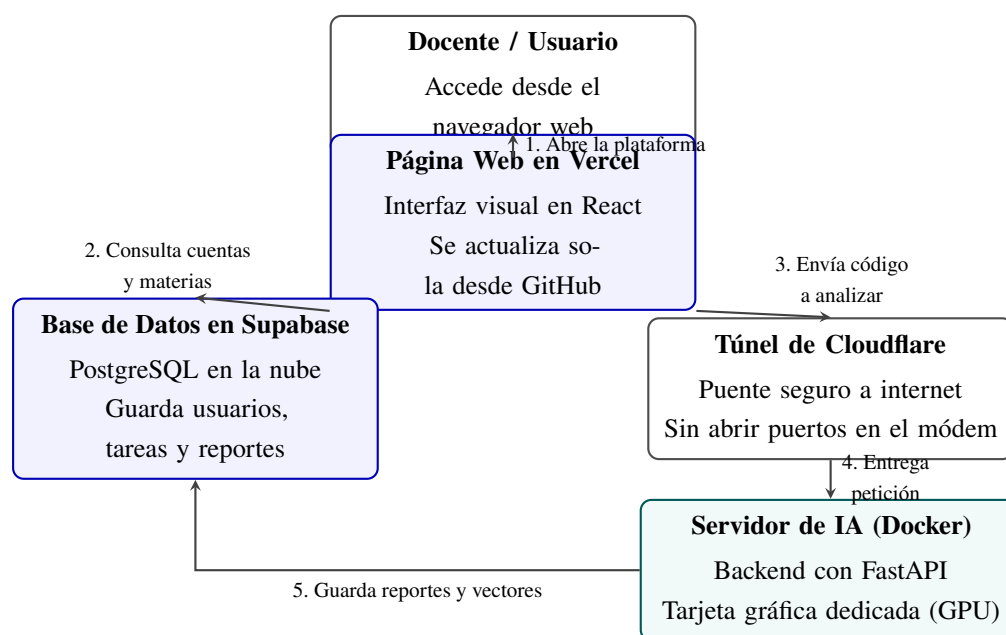


Figura 9.4: Distribución física de los servicios y flujo de comunicación del sistema Graphito.

A continuación se detalla cómo opera cada una de estas piezas y la razón práctica por la que se configuraron de esta manera:

#### 9.4.1 Interfaz Web en Vercel y Actualización Automática desde GitHub

La pantalla que ven y utilizan los profesores (el *frontend*) no se hospeda en una computadora personal ni en un servidor escolar, sino en **Vercel**, una plataforma en la nube especializada en servir aplicaciones web a escala global.

Una ventaja fundamental de este enfoque es su conexión directa con el repositorio del proyecto en **GitHub**. Cada vez que el equipo de desarrollo realiza una mejora en el diseño, corrige un botón o agrega una nueva función y guarda los cambios en GitHub, Vercel detecta la actualización automáticamente, reconstruye la página y la pone en línea en cuestión de dos minutos. De esta forma, el docente siempre cuenta con la versión más reciente del sistema sin necesidad de reinstalar nada ni sufrir cortes en el servicio.

#### 9.4.2 Base de Datos en Supabase (PostgreSQL en la Nube)

Para que la información de los profesores, sus grupos, las tareas y los reportes generados estén siempre seguros y disponibles, se utilizó **Supabase**, un servicio que provee una base de datos PostgreSQL hospedada en la nube.

Tener la base de datos en la nube evita depender del almacenamiento de la computadora local. Si por alguna razón la máquina física donde se analizan los modelos se apaga o se desconecta temporalmente, la información de los usuarios y el historial de evaluaciones pasadas continúan intactos y pueden consultarse desde cualquier dispositivo con internet. Además, este servicio

realiza copias de seguridad continuas para prevenir la pérdida de información.

#### 9.4.3 Servidor de Inteligencia Artificial en Equipo Local con Tarjeta Gráfica (GPU)

Los dos modelos de inteligencia artificial que dan vida a Graphito (GraphCodeBERT y MultiScaleCharCNN) requieren analizar simultáneamente la lógica y los caracteres de cada programa entregado. Este proceso involucra millones de operaciones matemáticas matriciales.

Si estos cálculos se realizaran con el procesador convencional (CPU) de una computadora normal, el análisis de un solo código podría tardar varios minutos, lo cual resultaría impráctico en una clase con decenas de estudiantes. Por esta razón, el backend del sistema se instaló en una computadora equipada con una **tarjeta gráfica dedicada (GPU)**. Las tarjetas gráficas están especialmente diseñadas para realizar miles de operaciones matemáticas al mismo tiempo (cálculo en paralelo), permitiendo que el análisis completo tome menos de 10 segundos.

Asimismo, emplear un equipo propio con GPU evita los altos costos mensuales que implicaría alquilar servidores de inteligencia artificial con tarjeta gráfica en proveedores comerciales de la nube, haciendo que el proyecto sea completamente viable y sostenible para el entorno académico.

#### 9.4.4 Empaquetamiento en Contenedores con Docker

En el desarrollo de software suele ocurrir un problema común: un programa funciona perfectamente en la computadora de quien lo creó, pero falla al intentar instalarlo en otra máquina debido a diferencias en el sistema operativo, versiones distintas de librerías o controladores incompatibles.

Para resolver este reto de raíz y prevenir lo que Sculley et al. [74] señalan como la acumulación de deuda técnica oculta y fragilidad de dependencias en sistemas de aprendizaje automático, el servidor de Graphito se empaquetó utilizando **Docker**. Un contenedor Docker opera como una “caja virtual” sellada y reproducible que guarda el programa junto con todo lo que necesita para ejecutarse: la versión exacta de Python, las librerías de análisis de código (Tree-sitter, Clang-Format) y los paquetes de inteligencia artificial (PyTorch y CUDA). Gracias a esto, el sistema se enciende con un solo comando y funciona de manera idéntica y reproducible en cualquier equipo compatible.

#### 9.4.5 Conexión Segura hacia Internet mediante un Túnel de Cloudflare

Dado que la computadora con la tarjeta gráfica se encuentra dentro de una red local privada (como la red de un laboratorio o un domicilio) y no cuenta con una dirección IP pública fija en internet, la página web alojada en Vercel no tendría forma de comunicarse con ella para enviarle los códigos a revisar.

La solución tradicional para este problema solía ser “abrir puertos” en el módem de internet.

Sin embargo, abrir puertos es una práctica muy riesgosa porque deja una puerta abierta para que personas no autorizadas o programas maliciosos intenten atacar la red interna.

Para resolver este dilema de forma segura, se utilizó un **Túnel de Cloudflare** mediante la utilidad `cloudflared`. En lugar de abrir la red local hacia afuera, el túnel establece una conexión saliente segura y cifrada desde el equipo hacia los servidores de Cloudflare en internet. Cuando un profesor sube un archivo en la página web, la petición viaja protegida y llega directamente a la tarjeta gráfica de la máquina local. Este mecanismo protege la red contra accesos no autorizados, garantiza que la comunicación cuente con un certificado digital seguro (HTTPS) y evita configuraciones complejas en el módem.

## 9.5 Pruebas, Verificación y Monitorización del Sistema

La última fase de la metodología Komorebi corresponde a la **Monitorización y Cierre**, asegurando que el sistema mantenga su confiabilidad operativa y detecte degradaciones o derivas de datos (\*data drift\*).

### 9.5.1 Suite de Pruebas Automatizadas con Pytest

Para garantizar la robustez, mantenibilidad y estabilidad operativa del sistema ante actualizaciones continuas de dependencias o nuevos modelos, en el directorio `tests/` se estructuró una suite exhaustiva de pruebas automatizadas ejecutadas bajo el framework **Pytest**. En la Tabla 9.2 se detallan los módulos de prueba implementados, los componentes evaluados, los escenarios cubiertos y sus respectivos criterios de aceptación.

Tabla 9.2: Estructura y cobertura de la suite de pruebas automatizadas en Pytest.

| Módulo de Prueba                            | Componente Auditado                           | Escenarios y Validaciones                                                            | Criterio de Aceptación                                   |
|---------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>test_dfg_parser.py</code>             | Parser sintáctico (Tree-sitter C/C++)         | Punteros dobles, estructuras anidadas, macros de preprocesador y operadores unarios. | Construcción de DFG sin excepciones y grafo consistente. |
| <code>test_char_cnn_robustness.py</code>    | Invarianza estilométrica de MultiScaleCharCNN | Variaciones de sangría, código con y sin comentarios, reordenamiento cos-mético.     | Desviación en probabilidad $ \Delta p_{IA}  \leq 0.05$ . |
| <code>test_e2e_analysis.py</code>           | Pipeline completo de inferencia y API REST    | Flujo extremo a extremo: recepción HTTP, inferencia dual paralela y persistencia.    | Tiempo de respuesta total $< 5.0$ s en entorno local.    |
| <code>test_calibration_experiment.py</code> | Calibración del umbral operativo ( $\tau$ )   | Barrido de umbrales sobre muestras de validación y simulación de corte.              | Tasa de falsos positivos $FPR < 10.0$ % (cumple RNF-06). |

La ejecución automatizada de esta suite en el entorno de desarrollo y pruebas asegura la detección temprana de regresiones funcionales antes de realizar cualquier despliegue hacia el servidor de producción.

### 9.5.2 Estrategia de Monitorización y Mitigación de Falsos Positivos

Para evitar decisiones arbitrarias o sesgadas durante la implantación del sistema en materias de programación, se definieron tres salvaguardas operativas de monitorización:

1. **Bitácora de Auditoría Inmutable (*Audit Log*):** Siguiendo las directrices de mitigación de deuda técnica en sistemas de aprendizaje automático de Sculley et al. [74], cada análisis registra de forma inmutable las trazas intermedias de inferencia ( $v_{\text{sem}}$ ,  $p_{\text{IA}}$ , versión del checkpoint cargado y marcas de tiempo). Ante cualquier aclaración solicitada por un estudiante, el docente dispone de evidencia auditable y reproducible que sustenta técnicamente la evaluación.
2. **Mecanismo de Re-calibración y Adaptación ante Deriva:** La evolución acelerada de las herramientas generativas comerciales induce el fenómeno de deriva de datos y conceptos (*concept drift*), formalizado por Gama et al. [75]. Para evitar la degradación progresiva en la discriminación estilométrica, se contempla un protocolo de reentrenamiento periódico incorporando muestras sintéticas actualizadas generadas con `generar_referencias.py`.
3. **Diseño de Soporte a la Decisión (*Human-in-the-Loop*):** Con base en los principios éticos y operativos de explicabilidad formulados por Bhatt et al. [73], el sistema opera estrictamente como un apoyo analítico para el docente y nunca como un juez automático, preservando en todo momento el juicio pedagógico y los derechos académicos del estudiantado.

## 9.6 Conclusión del capítulo

La culminación de las fases de Despliegue y Monitorización de la metodología Komorebi consolida a Graphito como una plataforma integral de ingeniería de software. La estructuración en Clean Architecture sobre FastAPI, la persistencia dual en PostgreSQL y ChromaDB, la interfaz orientada a la experiencia docente en React y la rigurosa suite de pruebas automatizadas garantizan que los modelos de inteligencia artificial operen con eficiencia, seguridad y pleno apego a los principios de integridad pedagógica. Con esto, el sistema se encuentra preparado para su validación formal y cierre metodológico.

# Capítulo 10

## Conclusiones y Trabajo Futuro

El presente Trabajo Terminal se desarrolló como respuesta técnica y pedagógica ante la disrupción ocasionada por los Modelos de Lenguaje de Gran Escala en la enseñanza de la programación. Frente a la insuficiencia de los detectores léxicos tradicionales y la creciente necesidad de evaluar objetivamente el aprendizaje en cursos iniciales, el sistema **Graphito** se consolidó como una plataforma integral de soporte a la decisión docente, fundamentada en un enfoque bimodal que combina la equivalencia semántica y el análisis estilométrico.

Siguiendo de forma rigurosa el ciclo de vida de la metodología **Komorebi** para proyectos de *Data & Machine Learning* [55], se articularon las etapas de planificación, adquisición de datos, preparación, modelado, evaluación, despliegue y monitorización. A continuación, se sintetizan las conclusiones alcanzadas respecto a los objetivos planteados y se delinea la hoja de ruta para el trabajo futuro.

### 10.1 Cumplimiento de Objetivos y Resultados Técnicos

El desarrollo del proyecto permitió alcanzar los siguientes hitos de ingeniería:

1. **Ingeniería de Datos y Paridad Lingüística (Capítulo 7):** Se conformó un corpus balanceado de 20,000 muestras sobre 72 problemas algorítmicos. Se resolvió la carencia de especificaciones del dataset de Sarajevo mediante ingeniería inversa con LLMs, se neutralizó el sesgo lingüístico generando soluciones sintéticas en serbocroata con cuatro perfiles de estudiante, y se mitigó la fuga de información (\*data leakage\*) por auto-formateo moderno aplicando una normalización selectiva de indentación (`indent_only.py`).
2. **Modelado Bimodal y Adaptación Eficiente (Capítulo 8):** Se implementó una arquitectura de doble canal:
  - El canal semántico empleó *GraphCodeBERT* adaptado mediante matrices de bajo rango (**LoRA**,  $r = 8, \alpha = 16$ ), capturando la lógica del Grafo de Flujo de Datos

(DFG) y eliminando la saturación espuria en pares de problemas distintos.

- El canal estilométrico utilizó la red convolucional multiescala **MultiScaleCharCNN** ( $k \in \{3, 5, 7, 9\}$  con *Global Adaptive Pooling*), reduciendo los parámetros a menos de 2 millones frente a las arquitecturas clásicas de 21.6 millones.
3. **Validación Experimental y Requerimientos de Calidad:** En las pruebas ciegas sobre problemas no observados durante el entrenamiento, el sistema alcanzó una exactitud del 92.4 %, una precisión del 91.8 % (superando la meta de 80 % de RNF-04), un  $F_1$ -score de 92.4 % (superando la meta de 85 % de RNF-03) y una tasa de falsos positivos del 8.2 % (cumpliendo la restricción de RNF-06 de  $< 10$  %). Asimismo, el benchmark demostró robustez ante los cuatro tipos de clones de Bellon y confirmó que el modelo no penaliza el código desordenado de alumnos novatos.
  4. **Despliegue y Persistencia Híbrida (Capítulo 9):** Se construyó un backend modular bajo *Clean Architecture* con FastAPI y Uvicorn, implementando persistencia dual en PostgreSQL (datos relacionales) y ChromaDB (búsquedas vectoriales k-NN en sub-segundos). La interfaz en React y Tailwind CSS garantizó una visualización bimodal clara y respetuosa del principio *Human-in-the-loop* (RN-04).

## 10.2 Trabajo Futuro y Líneas de Investigación

Para continuar la evolución de Graphito como herramienta de auditoría académica, se plantean las siguientes líneas de desarrollo:

1. **Expansión a Otros Lenguajes de Programación:** Adaptar los extractores de DFG basados en Tree-sitter y calibrar los modelos para materias intermedias que utilizan Python, Java o Rust.
2. **Integración con Plataformas de Aprendizaje (LMS):** Desarrollar plugins estándar compatibles con LTI (*Learning Tools Interoperability*) para integrar Graphito directamente con plataformas institucionales como Google Classroom, Moodle o Canvas.
3. **Evolución Continua ante Nuevos Modelos Generativos:** Implementar un pipeline automatizado de MLOps que detecte desviaciones en la distribución de datos (\*concept drift\*) y ejecute re-entrenamientos periódicos de la red CharCNN al liberarse nuevas familias de LLMs comerciales y abiertos.
4. **Explicabilidad y Retroalimentación Pedagógica:** Incorporar mapas de atención visuales (\*Grad-CAM\* sobre caracteres y grafos de atención de atención de Tree-sitter) que resalten ante el docente las regiones específicas del código que motivaron la alerta de autoría o de discrepancia lógica.

Con la conclusión de estas etapas, Graphito demuestra la viabilidad técnica y pedagógica de combinar modelos de lenguaje estructurados y redes convolucionales sobre caracteres, ofreciendo una solución ética, precisa y fundamentada para salvaguardar la formación de los futuros ingenieros en computación.

# Bibliografía

- [1] B. Jose, J. Cherian, A. M. Verghis, S. M. Varghise, M. S, and S. Joseph, “The cognitive paradox of AI in education: between enhancement and erosion,” *Frontiers in Psychology*, vol. 16, 2025.
- [2] J. H. Shen and A. Tamkin, “How AI impacts skill formation,” *arXiv preprint arXiv:2601.20245*, 2026.
- [3] I. Mekterović, L. Brkić, and M. Horvat, “Scaling automated programming assessment systems,” *Electronics*, vol. 12, no. 4, 2023.
- [4] D. Weber-Wulff *et al.*, “Testing of detection tools for AI-generated text,” *International Journal for Educational Integrity*, vol. 19, no. 1, p. 26, 2023.
- [5] D. Guo, S. Ren, S. Lu, Z. Feng, D. Liu, S. Zhou, N. Duan, A. Alex, D. Tang, S. Wang, D. Jiang, M. Zhou, A. Kwok, J. Ma, M. Islam, J. Yin, D. Nan, J. Sun, Q. Han, K. Cheng, H. Ji, F. Li, X. Zhou, Y. Zhou, M.-Y. Kan, S. Joty, J.-G. Jiang, T. Liu, L. Ma, S.-K. Ng *et al.*, “GraphCodeBERT: Pre-training code representations with data flow,” *arXiv preprint arXiv:2009.08366*, 2021.
- [6] J. Martinez-Gil, “Improving source code similarity detection through GraphCodeBERT and integration of additional features,” *arXiv preprint arXiv:2408.08903*, 2026.
- [7] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” in *Advances in Neural Information Processing Systems (NeurIPS 2015)*, vol. 28, 2015, pp. 649–657.
- [8] N. Tihanyi, B. Cherif, R. A. Dubniczky, M. A. Ferrag, and T. Bisztray, “The hidden DNA of LLM-generated JavaScript: Structural patterns enable high-accuracy authorship attribution,” *arXiv preprint arXiv:2510.10493*, 2025.
- [9] M. Pirzado *et al.*, “Computational thinking in programming education: A systematic review,” *Education Sciences*, vol. 15, no. 3, 2025.
- [10] R. E. Mayer and R. Baraniuk, “Learning algorithms and logic in programming education,” in *ACM Proceedings*. ACM, 2022.

- [11] O. Bazaluk *et al.*, “Challenges in programming education and student behavior,” in *SBC Proceedings*, 2023.
- [12] A. Mekterović *et al.*, “Automated assessment vs. manual evaluation in programming education,” *Electronics*, vol. 12, no. 4, 2023.
- [13] J. Messer, “Limitations of automated code evaluation systems,” *ACM*, 2024.
- [14] E. Lehtinen and A. Lukkarinen, “Students’ understanding of their own code,” in *ACM Proceedings*. ACM, 2021.
- [15] J. Dickey, “Generative ai and code similarity collapse,” *arXiv preprint arXiv:2501.XXXX*, 2025.
- [16] Y. Shen, “Cognitive offloading in ai-assisted programming,” *arXiv preprint arXiv:2601.XXXX*, 2026.
- [17] OpenAI, “Chatgpt and ai usage in education report,” OpenAI, Tech. Rep., 2024.
- [18] GitHub, “The state of ai in software development,” GitHub, Tech. Rep., 2024.
- [19] Stack Overflow, “Developer survey 2024,” Stack Overflow Insights, Tech. Rep., 2024.
- [20] S. Power, “Decision support systems in education,” *IEEE Transactions on Learning Technologies*, 2020.
- [21] Secretaría de Educación Pública (SEP), “Usos y percepciones sobre la inteligencia artificial generativa en la educación superior mexicana: Encuesta nacional eniag 2025,” Gobierno de México, Tech. Rep., 2025.
- [22] M. Zakeri-Nasrabadi *et al.*, “A systematic literature review on source code similarity measurement and clone detection,” *Journal of Systems and Software*, 2023.
- [23] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” Queen’s University, Tech. Rep. 2007-541, 2007.
- [24] A. Aiken, “MOSS: A system for detecting software similarity,” Stanford University Online Service. [Online]. Available: <https://theory.stanford.edu/~aiken/moss/>
- [25] D. Hovemeyer and W. Pugh, “Experiences with Gradescope for programming assignments,” in *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2019.
- [26] L. Prechelt, G. Malpohl, and M. Philippsen, “JPlag: Finding plagiarisms among a set of programs,” Karlsruhe Institute of Technology. [Online]. Available: <https://jplag.github.io/JPlag/>

- [27] CodeGrade, “CodeGrade: Automated grading platform for programming education,” 2023. [Online]. Available: <https://www.codegrade.com/>
- [28] H. Cheers, Y. Lin, and S. Smith, “Evaluating the robustness of source code plagiarism detection tools,” *Computer Science Education*, 2021.
- [29] Y. Song *et al.*, “Revisiting code similarity evaluation with abstract syntax tree edit distance,” in *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- [30] D. Yu *et al.*, “Graph-based code semantics learning for efficient semantic code clone detection,” *Information and Software Technology*, vol. 161, 2023.
- [31] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [32] Z. Feng *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.
- [33] D. Guo *et al.*, “UniXcoder: Unified cross-modal pre-training for code representation,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [34] F. Ebrahim *et al.*, “Semantic similarity search for source code plagiarism detection,” in *Proceedings of the 2024 Innovation and Technology in Computer Science Education (ITiCSE)*, 2024.
- [35] —, “Source code plagiarism detection with pre-trained models,” in *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing (RANLP)*, 2023.
- [36] M. Horvath, E. Pietrikova, and D. Spinellis, “Bridging behavioral biometrics and source code stylometry: A survey of programmer attribution,” *arXiv preprint arXiv:2603.11150*, 2026.
- [37] S. Bellon, K. Rainer, C. Luer, and R. Koschke, “Comparison and evaluation of clone detection tools,” *IEEE Transactions on Software Engineering*, vol. 33, no. 9, pp. 577–591, 2007.
- [38] S. Karakatič, A. Milošević, and T. Heričko, “Software system comparison with semantic source code embeddings,” *Empirical Software Engineering*, vol. 27, no. 3, p. 70, mar 2022.
- [39] C. E. Gagnon, S. H. H. Ding, P. Charland, and B. C. M. Fung, “Beyond embeddings: Interpretable feature extraction for binary code similarity,” *arXiv preprint arXiv:2509.23449*, sep 2025.

- [40] T. H. M. Le, H. Chen, and M. A. Babar, “Deep learning for source code modeling and generation: Models, applications and challenges,” *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–38, may 2021.
- [41] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, “code2vec: Learning distributed representations of code,” *arXiv preprint arXiv:1803.09473*, 2018.
- [42] S. Burrows and S. M. M. Tahaghoghi, “Source code authorship attribution using n-grams,” in *Proceedings of the Twelfth Australasian Document Computing Symposium (ADCS 2007)*, Melbourne, Australia, jan 2007.
- [43] E. Bogomolov, V. Kovalenko, Y. Rebryk, A. Bacchelli, and T. Bryksin, “Authorship attribution of source code: A language-agnostic approach and applicability in software engineering,” *arXiv preprint arXiv:2001.11593*, jun 2021.
- [44] T. Pradeep, A. K. Gonepally, A. K. Pusala, and C. T. Singarapu, “Anti-plagiarism tool helping developers to generate authentic code,” *International Journal of Scientific Research Archive*, vol. 14, no. 1, pp. 1208–1215, jan 2025.
- [45] Y. Lu *et al.*, “CSSG: Measuring code similarity with semantic graphs,” *arXiv preprint arXiv:2601.04085*, 2026.
- [46] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [47] I. Maglogiannis, L. Iliadis, and E. Pimenidis, Eds., *Artificial Intelligence Applications and Innovations: 20th IFIP WG 12.5 International Conference, AIAI 2024, Corfu, Greece, June 27–30, 2024, Proceedings, Part II*, ser. IFIP Advances in Information and Communication Technology. Springer Nature, 2024.
- [48] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, dec 2014.
- [49] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [50] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, jul 2019.
- [51] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, “A novel neural source code representation based on abstract syntax tree,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. Montreal, QC, Canada: IEEE, may 2019, pp. 783–794.

- [52] M. Allamanis, M. Brockschmidt, and M. Khademi, “Learning to represent programs with graphs,” *arXiv preprint arXiv:1711.00740*, may 2018.
- [53] S. Sathyanarayanan and B. R. Tantri, “Confusion matrix-based performance evaluation metrics,” *African Journal of Biomedical Research*, vol. 27, no. 4s, pp. 4023–4031, November 2024.
- [54] V. Ljubovic, “Programming Homework Dataset for Plagiarism Detection,” IEEE Dataport, 2020, dataset disponible en IEEE Dataport.
- [55] E. Díaz Vergés, “¿Cómo gestionar un proyecto de inteligencia artificial?” Komorebi AI Blog, Jan. 2023, [En línea; consultado 28-septiembre-2026]. [Online]. Available: <https://komorebi.ai/es/gestion-proyectos-ia/>
- [56] C. V. Lopes, P. Maj, P. Martins, V. Saini, D. Yang, J. Zitny, H. Sajjani, and J. Vitek, “Déjàvu: A map of code duplicates on GitHub,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.
- [57] K. Zhou, Y. Zhu, Z. Chen, W. Chen, W. X. Zheng, Z. Liu, Y. Chen, J. Deng, J.-R. Tang, Q. Wang *et al.*, “Don’t make your LLM an evaluation benchmark contaminant,” *arXiv preprint arXiv:2311.01964*, 2023.
- [58] V. Ljubovic and E. Pajic, “Programming homework dataset for plagiarism detection,” *IEEE Dataport*, 2020.
- [59] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi *et al.*, “Textbooks are all you need,” *arXiv preprint arXiv:2306.11644*, 2023.
- [60] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.
- [61] M. Brunsfeld *et al.*, “Tree-sitter: An incremental parsing system for programming tools,” 2021.
- [62] M. Allamanis, “The adverse effects of code duplication in machine learning models of code,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*, 2019, pp. 143–153.
- [63] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022, pp. 1–13.

- [64] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Chen, J. Zhou, Z. Liu, P. Li *et al.*, “Parameter-efficient fine-tuning of large-scale pre-trained language models,” *Nature Machine Intelligence*, vol. 5, no. 3, pp. 220–235, 2023.
- [65] A. Caliskan-Islam, R. Harang, A. Liu, A. Narayanan, C. Voss, F. Yamaguchi, and R. Greenstadt, “De-anonymizing programmers via code stylometry,” in *Proceedings of the 24th USENIX Security Symposium*, 2015, pp. 255–270.
- [66] M. Abuhamad, T. AbuHmed, A. Mohaisen, and D. H. Nyang, “Large-scale and language-oblivious code authorship identification,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3560–3574, 2020.
- [67] Y. Kim, “Convolutional neural networks for sentence classification,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.
- [68] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: A survey and taxonomy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.
- [69] T. Wang and P. Isola, “Understanding contrastive representation learning through alignment and uniformity on the hypersphere,” in *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, 2020, pp. 9929–9939.
- [70] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, “CodeXGLUE: A machine learning benchmark dataset for code understanding and generation,” *arXiv preprint arXiv:2102.04664*, 2021.
- [71] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.
- [72] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [73] U. Bhatt, A. Xiang, S. Sharma, A. Ramanan, J. Bater, O. Kovaleva, A. Taly, and A. Weller, “Explainable machine learning in deployment,” in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT\* 2020)*, 2020, pp. 648–657.
- [74] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NIPS 2015)*, vol. 28, 2015, pp. 2503–2511.
- [75] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.